# Financial Statement Forecasting from Annual Reports

A Structural No-Plug Approach with Machine Learning

---

JPMorganChase — MLCOE TSRL Internship Project (Question 1)

Jaebum (Albert) Chung

April 6, 2026

**Abstract**

This report implements a structural financial statement forecasting model based on the no-plug framework of Pareja(09), where the balance sheet is derived endogenously from the cash budget and income statement without circular references. Policy variables are trained from historical data using gradient-based optimisation, and Bayesian extensions provide confidence intervals via Monte Carlo simulation. The framework is extended with LLM-based extraction pipelines for automated ingestion of annual reports and strategic financial analysis.

# Contents

# 1 Introduction

## 1.1 Literature Review

*The Limitations of "Plug" Variables in Financial Forecasting*

Financial statement forecasting is a foundational practice in corporate finance, yet traditional methodologies often compromise the integrity of the double-entry accounting system. As reviewed in Pareja et al. (2007) and Pareja (2009), a significant portion of existing literature relies on the use of "plugs": balancing variables forced to reconcile the difference between Assets and Liabilities plus Equity. Typically, analysts assign a specific account (often Cash, Debt or Equity [Pareja(09)]) to absorb any discrepancies in the forecast.

While this approach ensures a balanced sheet mathematically, it is methodologically flawed for robust forecasting. By using a plug, the analyst effectively overrides the natural evolution of the balance sheet, obscuring the causal relationships between operating decisions and financial health. More critically, the use of a plug erodes the primary validation mechanism of the double-entry principle. If the balance sheet is forced to balance, it becomes impossible to detect logical errors or inconsistencies in the underlying code. As noted in the critiques within the Pareja framework, a model that balances by force rather than by logic fails to serve as a reliable tool for stress testing or error detection.

Financial statements are governed by strictly defined accounting identities and deterministic valuation rules, such as the First-In, First-Out (FIFO) inventory method [IASB(21)]. Consequently, the direct application of machine learning algorithms, such as Long Short-Term Memory (LSTM) networks, to forecast individual line items is often methodologically flawed. Because these 'black box' models treat variables as probabilistic time series rather than components of an integrated system, they fail to enforce the structural interconnectedness required by double-entry bookkeeping. This limitation frequently results in forecasts that violate the fundamental accounting equation (Assets = Liabilities + Equity), thereby rendering the model incapable of accurately assessing a company's overall financial health.

## 1.2 The Double-Entry Principle

Double-entry bookkeeping is the foundation of modern accounting. Its central rule is that every financial transaction must be recorded as two equal and opposite entries across ledger accounts, so that the fundamental accounting equation is preserved at all times:

$$\text{Assets} = \text{Liabilities} + \text{Equity}$$

To see why, consider two scenarios and how they flow through the three core financial statements: the Income Statement, the Statement of Cash Flows, and the Balance Sheet.

**Example 1: A credit sale and subsequent collection (Income Statement → Balance Sheet → Cash Flow Statement).** A company sells \$1,000 of goods on credit. At the point of sale, the Income Statement records \$1,000 in Revenue, which, after taxes and expenses, flows into Net Income and ultimately into Retained Earnings (a component of Equity). On the Balance Sheet, Accounts Receivable (an asset) increases

by \$1,000. The equation remains balanced: assets rose by \$1,000 (AR) and equity rose by the same amount (via Retained Earnings absorbing Net Income).

When the customer later pays the \$1,000 in cash, the Cash Flow Statement records a \$1,000 operating cash inflow. On the Balance Sheet, Cash increases by \$1,000 while Accounts Receivable decreases by \$1,000. One asset replaced another, and Total Assets are unchanged. The equation still holds because this is a pure asset swap. Notice how a single economic event (a sale) creates an accrual on the Income Statement first, then resolves through the Cash Flow Statement later, with the Balance Sheet tracking both stages.

**Example 2: Debt-financed equipment purchase (Cash Flow Statement → Balance Sheet → Income Statement).** A company borrows \$2,000 in long-term debt and immediately uses the proceeds to purchase equipment. The Cash Flow Statement records a \$2,000 inflow under financing activities (the borrowing) and a \$2,000 outflow under investing activities (the capital expenditure), so the net cash impact is zero. On the Balance Sheet, Non-Current Assets (equipment) increase by \$2,000 and Long-Term Debt (a liability) increases by \$2,000. Both sides of the equation grew by the same amount, so it remains balanced.

In subsequent periods, the transaction continues to ripple through the statements. The Income Statement records Depreciation expense (reducing Net Income as the asset is consumed) and Interest expense on the debt (reducing Net Income further). Both expenses flow back to the Balance Sheet: Depreciation reduces Non-Current Assets, and the lower Net Income reduces Retained Earnings. Meanwhile, the Cash Flow Statement captures the interest and principal payments as outflows, each reducing Cash on the Balance Sheet while simultaneously reducing the outstanding Debt liability. At every step, the equation holds.

**Connection to our model.** These examples reveal why double-entry bookkeeping makes *driver-based* financial forecasting without plugs or circularity possible. Because every value flow is recorded on both sides of the ledger (every expense has a matching cash outflow, every credit sale has a matching receivable, every depreciation charge has a matching reduction in Non-Current Assets), the financial state at time $t + 1$ is fully determined by the state at time $t$ together with the flows between them. Nothing escapes the ledger.

### 1.3 Proposed Methodology: A Structural "No-Plug" Approach

To address the limitations of plug-based financial forecasts, this report implements a structural forecasting model based on the framework proposed by Pareja (2009) with the revenue as the primary driver. The core distinction of this method is the rejection of circular references and balancing plugs. Instead, the model derives next year's Balance Sheet and Income Statement endogenously from this year's state through a *Cash Budget* and the projected revenue.

**The three financial statements.** A company's financial position at any point in time is captured by three interconnected statements:

- **Balance Sheet**: A snapshot of what the company owns (Assets: non-current assets, receivables, inventory, cash, market securities) and owes (Liabilities: payables, short-term debt, long-term debt) plus the residual claim of owners (Equity).
- **Income Statement**: The flow of revenues and expenses over a period, producing Net Income. Key elements include Sales, Cost of Goods Sold (COGS), Operating Expenses (OpEx), Depreciation, Interest Expense, and Income Tax.
- **Cash Budget** (Statement of Cash Flows): The flow of cash in and out of the company, decomposed into operating, investing, financing, and owner transaction activities. The ending cash balance is the *result* of these flows, not an input.

**Forecast mechanism: driver + policies + Cash Budget.** The model uses **Sales Revenue** as the single primary forecast driver. Given a sales forecast, a set of *policy ratios* determine every other element:

- Sales drives Revenue on the Income Statement, which determines COGS (via a cost ratio), Receivables, Inventory, Payables, and Advance Payments on the Balance Sheet.
- OpEx, Depreciation, and Interest Expense complete the Income Statement, producing Net Income.
- The Cash Budget routes every revenue, expense, and accrual change into the appropriate net liquidity balance (operating, capital expense, financing, owner transactions, discretionary). The sum of these flows determines the new Cash and Investments in Market Securities balances.
- Debt rollover, equity financing, dividends, and buybacks are computed from the Cash Budget's financing and owner transaction modules, updating Liabilities and Equity.

Because every cash inflow and outflow is explicitly routed through the Cash Budget, the next year's Balance Sheet is fully determined: no element is left as a residual plug. The accounting identity Assets = Liabilities + Equity must emerge from the correctness of the logic, which is what makes the model auditable and testable.

**Policy variables and training.** In Pareja (2009)'s framework, all policy variables are assumed known. In our case, we do not know the policy variables but know a company's past financial statements. We train the policy ratios (e.g., %AR, %AP, %TL, %Cash, cost ratio, payout ratio, depreciation rate) from historical data using gradient-based optimization. Simple policies use static ratios; advanced policies use logit-linear time trends or Bayesian variational inference to capture evolving firm behavior and quantify uncertainty.

**Refinements to Pareja (2009).** We have further refined the framework to better reflect the treasury strategies of modern multinational corporations like Apple or General Motors. Where the original model might treat Marketable Securities solely as a reservoir for excess cash, our implementation models these securities as a distinct portion of the company's liquidity strategy [Almeida(14)]. This adjustment allows for a more realistic depiction of how large firms manage capital allocation between operational cash and interest-bearing assets.

Moreover, Pareja (2009) assumes long-term debt to always have 10-year maturity. In reality, various debt products with varying maturity schedules exist. We, therefore, use an average maturity value as a variable to be also trained.

Section 2 details the time evolution equations for every financial element, showing precisely how Sales, the policy ratios, and the Cash Budget connect to produce next year's Balance Sheet and Income Statement without any plug.

### 1.4 Testing and Validation Plan

Because the model eschews plugs, the fundamental accounting identity (*Assets = Liabilities + Equity*) becomes a *falsifiable* test of correctness rather than a guaranteed outcome. A nonzero delta at any forecast step constitutes a genuine failure, exposing structural errors that plug-based models would silently absorb. This property is the foundation of our validation strategy, which combines the accounting identity invariant with a comprehensive programmatic test suite. The full testing methodology is detailed in Section 5.

The first three subsections of Part 1 (*The Balance Sheet Elements*, *Time Evolution of Financial Elements*, and *Time Series Representation of the Balance Sheet*) answer questions 1 and 2; the submitted codes answer questions 3 and 4; and the remaining subsections (*Training the Model*, *Applying Machine Learning*, and *Forecast Results*) answer questions 5 through 8. For Part 2, the answers to questions a) through i) are directly mapped to its subsection headings.

## 2 Part 1

### 2.1 The Balance Sheet Elements

The elements of a balance sheet from Pareja(09) are as follows:

Assets

Net fixed assets (NFA)

Advance payments to suppliers for purchases (AdvPP)

Account receivables (AR)

Inventory (Inventory)

Short-term investments (Invest)

Cash equivalent (Cash)

Liabilities

Account payables (AP)

Advance payments from customers for sales (AdvPS)

Short-term debt (STDebt)

Long-term debt (LTDebt)

Equity

Equity invested (EI)

Cumulated retained earnings (CRE)

Cumulated buyback of equity (CBB)

Net Income (NI)

On the balance sheets of real companies, there are likely to be more elements. For example, there could be current and non-current 'other liabilities' other than the 4 mentioned above, and there could be various other assets including deferred tax, non-current account receivables, Goodwill, and other intangible assets. For the purpose of this simple modeling exercise, we first identify the elements listed in the above balance sheet, and lump all unaccounted-for elements as shown below, using Apple's balance sheet as example:

- Assets
  - All unaccounted-for non-interest-bearing elements are lumped with "non-current assets" along with net PPE.
    - Deferred tax, non-current account receivables, goodwill, other assets
    - Depreciation rate based on net fixed asset will be reduced accordingly.
  - We use other current assets as the proxy for advance payments to suppliers for purchases.
  - Short-term and long-term investments in market securities are lumped into "investments in market securities."
    - Earn a combined average interest.
  - We also combine cash equivalents with investments in market securities to for "total liquidity" because companies do not just hold onto a pile of cash, but rather invest a significant portion of cash in securities to earn interest, which is different from Pareja(09)'s logic of only investing in excess cash after meeting a minimum cash balance.
    - Total liquidity = cash equivalent + investments in market securities
- Liabilities
  - Advance payments from customers to be fulfilled next year are represented as deferred revenue in company balance sheets.
  - All unaccounted-for elements are lumped with long-term debt, to be labeled "non-current liabilities"
    - Includes non-current long-term debt, employee benefits, non-current deferred revenue, non-current accrued expenses, long-term capital lease obligation
      - These other non-current liabilities are like interest-free long-term debt
      - The total long-term liability is assumed to have the average maturity of AvgM years, and it generates an average LT interest (AvgLTInt) owed by the company
    - Something important to note is that the "long-term debt" in an actual balance sheet EXCLUDES the principal owed next year.

· Non-current liabilities$_t$ = Total LT liability$_t$ * (1 – 1/AvgM)

= Total LT liability$_t$ * (AvgM - 1) / AvgM

- We group all other liabilities due next year as "current liabilities."
  - Includes STDebt, current accrued expenses which include dealer and customer allowances, claims and discounts, product warranty, payrolls and benefits, and others, to be considered as "short-term loan"
    * Accrued expenses are like interest-free short-term debt.
    * Short-term loan (STLoan$_t$) generates an average ST interest (AvgSTInt$_t$) owed by the company
  - Also includes current debt which includes principal payment from short-term debt and ALSO INCLUDES the current portion of the total LT liability
    * Current liabilities$_t$ = Total ST liability$_t$ + Total LT liability$_t$ / AvgM

= Total ST liability$_t$ + Non-current liabilities$_t$ / (AvgM – 1)

- Equity
  - Represented by Stockholders Equity (SE$_t$)
    * SE$_t$ = SE$_{t-1}$ + EF$_t$ + NI$_t$ – Dividends$_{t-1}$ – BB$_t$
      · SE$_{t-1}$ = stockholder equity last year
      · EF$_t$ = equity financing to cover portion of CapEx spending
      · NI$_t$ = net income this year
      · Dividends$_{t-1}$ = Dividends earned last year, paid this year
      · BB$_t$ = stock buyback this year

So, our updated balance sheet, along with subitems for clarity, becomes:

Assets

Non-current assets (**NCA$_t$**)

- Net PPE, deferred tax, non-current account receivables, goodwill, other assets

Advance payments to suppliers for purchases (**AdvPP$_t$**)

Account receivables (**AR$_t$**)

Inventory (**Inv$_t$**)

Total liquidity (**TL$_t$**)

- Investments in market securities (**IMS$_t$**)
  - Short-term and long-term investments
- Cash equivalent (**Cash$_t$**)

Liabilities

Account payables (**AP$_t$**)

Advance payments from customers for sales (**AdvPS$_t$**)

Non-current liabilities (**NLiab$_t$**)

- Non-current employee benefits, non-current deferred revenue, non-current accrued expenses, long-term debt and capital lease obligations MINUS next year's principal, other non-current liabilities
- **NLiab$_t$** = Total LT liability$_t$ * (1 – 1/**AvgM**)

Current liabilities (**CLiab$_t$**)

- Consists of the operational short-term obligations (Effective ST Debt) plus the principal due this year on long-term loans.
- Effective ST Debt (**EffSTDebt$_t$**): ST debt (commercial paper), current accrued expenses, and other current liabilities. Two policy variants are implemented:
  - **Simple policy** (`SimpleDebtPolicy`): ST debt is deficit-driven: the model borrows short-term only to cover any remaining liquidity shortfall after operating and investing activities. EffSTDebt$_t$ = max(0, deficit$_t$).
  - **Advanced policy** (`TrendDebtPolicy`): ST debt is modeled as an affine function of sales, capturing the empirical pattern that firms like Apple scale short-term borrowing with revenue while also maintaining a fixed baseline (e.g., revolving facility minimums, notes payable): EffSTDebt$_t$ = EffSTDebt$_{\text{baseline}}$ + Sales$_t$ · %STDebt, where both EffSTDebt$_{\text{baseline}}$ and %STDebt are trainable non-negative parameters.
- Current LT Debt (**CurrLTDebt$_t$**): Current portion of the long-term debt and capital lease obligations principals.
  - CurrLTDebt$_t$ = Total LT liability$_t$ / AvgM = NLiab$_t$ / (AvgM – 1)
- Total: CLiab$_t$ = EffSTDebt$_t$ + CurrLTDebt$_t$

Equity

Stockholder equity (**SE$_t$**)

- **SE$_t$** = **SE$_{t-1}$** + **EF$_t$** + **NI$_t$** – **Dividends$_{t-1}$** – **BB$_t$**
  - SE$_{t-1}$ : total stockholder equity last year
  - EF$_t$ : equity financing to cover portion of CapEx spending
  - NI$_t$ : net income this year
  - Dividends$_{t-1}$ = Dividends earned last year, paid this year
  - BB$_t$ : stock buyback this year

## 2.2 Time Evolution of Financial Elements

We refer to Pareja(09) for constructing the mathematical equations that link all the elements on the balance sheet without circularity or plugs. First, we define a single primary driver that will predict the evolution of the balance sheet's elements.

- $\text{Sales}_t$ : the total revenue generated by the company in year t

This is a different approach from Pareja(09) where they modeled the unit price, the unit cost, and the volume sold based on market research data on volume's price sensitivity. For the scope of this problem set, we lump these variables into an overarching sales driver whose evolution is modeled independently based on historical data. Sales serves as the primary driver because it is the most fundamental measure of a firm's economic activity: all other financial statement items are either costs of generating revenue, assets deployed to support revenue, or financing decisions to fund those assets. In the corporate finance literature, revenue-driven forecasting is standard practice [Koller(20)]. The limitation of this approach is that it does not model the underlying demand drivers (price elasticity, market size, competitive dynamics) that determine revenue growth. The sales forecast itself is generated via linear extrapolation of historical trends, a pragmatic choice given the 4–8 years of available data, though macroeconomic-sensitive models (e.g., LSTM with GDP and inflation inputs) would improve robustness.

### 2.2.1 Forecast Flow Overview

Before diving into the equations for each balance sheet element, it is helpful to trace how a single period's sales forecast ripples through all three financial statements to produce next year's Balance Sheet. The flow is as follows.

**Income Statement.** Revenue equals $\text{Sales}_t$ at the top of the Income Statement. From Revenue we subtract the operating costs:

- Cost of Goods Sold (COGS)
- Operating Expenses (OpEx, e.g., administrative and sales)
- Depreciation and amortization

This yields EBIT (Earnings Before Interest and Taxes). Stated sequentially:

$$\text{EBIT} = \text{Revenue} - \text{COGS} - \text{OpEx} - \text{Depreciation}$$

Next, we account for interest payments on outstanding debt and returns on short-term investments in market securities (see (55)), producing EBT (Earnings Before Tax):

$$\text{EBT} = \text{EBIT} - \text{Interest} + \text{MSReturn}$$

Income tax is assessed as a fixed percentage of EBT, giving Net Income:

$$\text{Net Income} = \text{EBT} \cdot (1 - \%\text{IT})$$

From Net Income, a fraction is allocated as next year's dividends, and the remainder accumulates into Retained Earnings (a component of Equity).

**Cash Budget.** Net Income is an accrual concept and does not correspond directly to cash generated. To track actual liquidity, we build a Cash Budget that records cash movements when they physically occur, independent of revenue recognition timing. The Cash Budget therefore serves the same role as the Statement of Cash Flows but is built prospectively: rather than reconciling Net Income to cash after the fact, it schedules every expected cash receipt and disbursement directly.

Unlike the standard three-section Statement of Cash Flows, we decompose financing activities into two separate modules, namely External Financing (debt) and Transactions with Owners (equity and dividends), and add a fifth module for Discretionary Transactions in market securities, reflecting how these are managed as a policy allocation rather than a residual. Each module produces a Net Liquidity Balance (NLB):

- **Operating Activities**: Inflows from sales (including collections on receivables and advance payments) minus outflows for purchases, OpEx, and income tax.
- **Investing Activities**: Capital expenditure outflows (CapEx) for fixed asset investment.
- **External Financing**: Inflows from new short-term and long-term borrowing, minus outflows for principal repayments and interest expense.
- **Transactions with Owners**: Inflows from equity financing, minus outflows for dividends paid and stock buybacks.
- **Discretionary Transactions**: Returns from short-term investments in market securities. In our model, ST investments are a policy allocation of total liquidity rather than an excess-cash destination, so this module has no discretionary outflow.

The sum of the five NLBs determines the change in total liquidity (Cash + Investments in Market Securities) from one period to the next.

**Closing the loop.** With the Income Statement producing Net Income (and hence Retained Earnings) and the Cash Budget producing the change in total liquidity, every element of next year's Balance Sheet is determined:

- **Assets** evolve via working-capital ratios (AR, AP, Inv, AdvPP, AdvPS) applied to next year's Sales and Purchases, the NCA roll-forward (CapEx minus Depreciation), and the updated liquidity balance from the Cash Budget.
- **Liabilities** evolve via the debt rollover schedule (new borrowing from External Financing, principal payments, amortization of long-term debt into current portion) and working-capital ratios for AP and AdvPS.
- **Equity** evolves via the Retained Earnings accrual from Net Income, minus dividends and buybacks, plus any new equity financing.

No element is left as a residual plug. That is, no line item is set to whatever makes the balance sheet balance, which would otherwise mask errors in the model. Instead, the accounting identity Assets = Liabilities + Equity emerges automatically at $t + 1$ as a direct consequence of double-entry bookkeeping: every cash or accrual transaction affects at least two accounts by equal and opposite amounts, so the identity is structurally guaranteed, not verified after the fact.

The remainder of this section provides the detailed equations for each financial element: the individual asset line items, the Cash Budget, the liability line items, Equity, and the Income Statement.

### 2.2.2 Non-Current Assets (NCA)

The balance of non-current assets evolves sequentially as existing fixed assets depreciate over their useful lifecycle and new capital expenditures (CapEx) are capitalized. The fundamental roll-forward equation is defined as:

$$\text{NCA}_t = \text{NCA}_{t-1} - \text{Depreciation}_t + \text{CapEx}_t \tag{1}$$

Depreciation is modeled as a percentage of the beginning-of-period non-current asset base:

$$\text{Depreciation}_t = \text{NCA}_{t-1} \cdot \%\text{Depreciation} \tag{2}$$

Note: Because our model aggregates all Non-Current Assets, which may include non-depreciable assets such as land or goodwill, the trained effective depreciation rate (%Depr) will naturally be lower than the strict accounting depreciation rate applied purely to Plant, Property, and Equipment (PP&E).

To model CapEx, we diverge from a rigid steady-state assumption where maintenance perfectly offsets depreciation. Instead, we decompose CapEx into a maintenance component (scaled by a trainable parameter, %AM, to reflect partial, full, or inflationary replacement) and a growth component driven by sales expansion:

$$\text{CapEx}_t = \%\text{AM} \cdot \text{Depreciation}_t + \%\text{AG} \cdot \text{Sales}_t \tag{3}$$

By substituting (2) and (3) into (1), we can express the net change in non-current assets ($\Delta\text{NCA}_t$) purely in terms of our modeled variables. The growth component (%AG $\times$ Sales) assumes constant capital intensity of incremental revenue, a reasonable first approximation for asset-light technology firms where growth CapEx scales with business volume. For capital-intensive industries, a non-linear formulation (e.g., step functions for capacity expansion) may be more appropriate. During model training, the optimizer learns the parameters for the effective depreciation rate (%Depr), the maintenance replacement ratio (%AM), and the incremental asset growth coefficient (%AG).

### 2.2.3 Cost of Goods Sold (COGS) and Inventory

Instead of modeling purchases independently and using the Cost of Goods Sold (COGS) as a plug, we directly model COGS based on historical margin trends. We define the Cost Ratio (CR) as the proportion of sales consumed by COGS:

$$\text{CR}_t = \text{COGS}_t / \text{Sales}_t \tag{4}$$

To ensure our predicted cost ratio remains strictly bounded between 0 and 1 (a cost ratio outside this range would imply negative costs or costs exceeding revenue, neither of which is sustainable), we transform the historical cost ratio using the logit function and model it as a linear function of time (t). The logit-linear parameterization allows for gradual trend shifts. For example, a declining cost ratio may reflect economies of scale, supply chain efficiencies, or pricing power gains. The linear time trend is a parsimonious choice; in

practice, cost ratios may exhibit mean reversion as competitive pressure limits sustained margin expansion.

$$\text{logit}(\text{CR}_t) = \ln\left(\frac{\text{CR}_t}{1-\text{CR}_t}\right) = \alpha_{\text{CR}} + \beta_{\text{CR}} \cdot t \tag{5}$$

The parameters $\alpha_{\text{CR}}$ and $\beta_{\text{CR}}$ are trained on historical data by minimizing the mean squared error between the historical logit(CR) and the predicted logit(CR). Once the future Cost Ratio is predicted, COGS is calculated directly:

$$\text{COGS}_t = \text{CR}_t \cdot \text{Sales}_t \tag{6}$$

Next, inventory can be modeled as a function of last year's inventory, this year's purchases, and this year's COGS:

$$\text{Inv}_t = \text{Inv}_{t-1} + \text{Purchases}_t - \text{COGS}_t \tag{7}$$

We set the inventory target as a percentage of cost of goods sold, rather than sales. Inventory is carried at cost (under FIFO or weighted-average accounting), so it should scale with the cost to produce goods, not the selling price. This is equivalent to modeling Days Sales of Inventory: $\%\text{Inv}_{\text{COGS}} = \text{DSI}/365$.

$$\text{Inv}_t = \%\text{Inv}_{\text{COGS}} \cdot \text{COGS}_t = \%\text{Inv}_{\text{COGS}} \cdot \text{CR}_t \cdot \text{Sales}_t \tag{8}$$

Since $\text{Inv}_t$ and $\text{COGS}_t$ are both independently forecast, $\text{Purchases}_t$ is fully determined by rearranging the inventory identity (7):

$$\text{Purchases}_t = \text{Inv}_t - \text{Inv}_{t-1} + \text{COGS}_t \tag{9}$$

$$\text{Purchases}_t = \%\text{Inv} \cdot (\text{Sales}_t - \text{Sales}_{t-1}) + \text{COGS}_t \tag{10}$$

Rather than being forecast independently, $\text{Purchases}_t$ is derived directly from the structural accounting identity: it is the quantity of goods that must have been purchased to arrive at the target inventory level given the period's COGS. This is distinct from a balance-sheet plug: no residual is being forced to reconcile a constraint; the identity holds by construction, and $\text{Purchases}_t$ is simply its remaining free variable.

We add %Inventory ($\%\text{Inv}_{\text{COGS}}$), $\alpha_{\text{CR}}$ and $\beta_{\text{CR}}$ to the variables to be trained.

### 2.2.4 Advance Payments for Purchases

Advance payments to suppliers for purchases (AdvPP) can be modeled as strictly proportional to the total purchases made during the current year. This approach relies on the simplifying assumption that current purchasing volumes serve as a reliable proxy for near-term future demand. Therefore, we define:

$$\text{AdvPP}_t = \%\text{AdvPP} \cdot \text{Purchases}_t \tag{11}$$

We add %Advance payments to suppliers (%AdvPP) to the variables to learn in our training. Advance payments are computed as a percentage of the current period's purchases (not next period's), avoiding forward-looking bias in the time-series model.

### 2.2.5 Account Receivables

Next, we have account receivables. A fraction of this year's sales to customers are on credit and is receivable next year:

$$\text{AR}_t = \%\text{AR} \cdot \text{Sales}_t \tag{12}$$

We add %Account receivables (%AR) to the variables to learn in our training. Note that %AR is equivalent to DSO/365, where DSO (Days Sales Outstanding) measures the average collection period. Similarly, the inventory ratio $\%\text{Inv}_{\text{COGS}}$ corresponds to DSI/365 (Days Sales of Inventory), and the payables ratio %AP corresponds to DPO/365 (Days Payable Outstanding). Together, these three metrics define the Cash Conversion Cycle: $\text{CCC} = \text{DSO} + \text{DSI} - \text{DPO}$, a standard measure of working capital efficiency in corporate finance.

### 2.2.6 Total Liquidity

Finally, the last element in our assets is total liquidity (TL). Following Pareja(09), we model the total liquidity available as a function of sales, split between cash and investments in market securities (IMS). Two policy variants are implemented:

**Simple policy (`SimpleLiquidityPolicy`).** Total liquidity is a static percentage of sales, with a fixed cash/liquidity split:

$$\text{TL}_t = \%\text{TL} \cdot \text{Sales}_t \tag{13}$$

$$\text{Cash}_t = \%\text{Cash} \cdot \text{TL}_t \tag{14}$$

$$\text{IMS}_t = (1 - \%\text{Cash}) \cdot \text{TL}_t \tag{15}$$

The simple policy has a notable limitation: it assumes that total liquidity scales linearly with sales at a fixed ratio, which is unrealistic for cash-rich firms whose liquidity trajectory is driven by strategic capital-return programs rather than by operational needs. Apple is a clear example. Its cash balance and stock buybacks over the past decade have been governed primarily by a multi-year buyback program designed to draw down excess cash, not by any stable relationship to revenue. A pure sales-proportional model cannot capture this secular decline in the cash-to-sales ratio.

**Advanced policy (`TrendLiquidityPolicy`).** To address this, both the total liquidity ratio and the cash allocation evolve over time via logit-linear trends, with a baseline floor that is independent of sales:

$$\mathrm{TL}_t = \beta_{\text{base}} + \mathrm{Sales}_t \cdot \sigma(\alpha_{\mathrm{TL}} + \beta_{\mathrm{TL}} \cdot t) \tag{16}$$

$$\mathrm{Cash}_t = \mathrm{TL}_t \cdot \sigma(\alpha_{\mathrm{Cash}} + \beta_{\mathrm{Cash}} \cdot t) \tag{17}$$

The baseline floor $\beta_{\text{base}}$ captures the minimum liquidity a firm maintains regardless of revenue (e.g., strategic cash reserves), while the logit-linear terms allow the sales-proportional component and the cash/IMS split to shift gradually over time.

We add the liquidity policy parameters to our variables to be trained.

Before we move onto liabilities, it is helpful to generate a Cash Budget to model the flow of cash in and out of the liquidity balance (cash and short-term investments in market securities).

### 2.2.7 Cash Budget

We can break down flows into/out of liquidity balance into net liquidity balances (NLB) pertaining to the following activities: operating, capital expense, financing, external investment, and transaction with owners. The final liquidity is the sum of all NLBs.

**Operating NLB** Operating activity captures all inflows from sales (current year sales revenue, account receivables from last year, advance payments from customers for next year) and all outflows (current year purchases, account payables from last year, advance payments to suppliers for next year, operating expenses, income tax):

$$\mathrm{Inflows}_t = \mathrm{AR}_{t-1} + \mathrm{AdvPS}_t + \mathrm{Sales}_t \cdot (1 - \%\mathrm{AR} - \%\mathrm{AdvPS}) \tag{18}$$

$$\mathrm{Outflows}_t = \mathrm{AP}_{t-1} + \mathrm{AdvPP}_t + \mathrm{Purchases}_t \cdot (1 - \%\mathrm{AP} - \%\mathrm{AdvPP}) + \mathrm{OpEx}_t + \mathrm{IT}_t \tag{19}$$

$$\text{Operating NLB}_t = \mathrm{Inflows}_t - \mathrm{Outflows}_t \tag{20}$$

where $\mathrm{OpEx}_t$ is the operating expense, and $\mathrm{IT}_t$ is income tax. We can model $\mathrm{OpEx}_t$ as a function of sales and inflation:

$$\mathrm{OpEx}_t = \%\mathrm{OR} \cdot \mathrm{Sales}_t + \mathrm{OB}_{t-1} \cdot (1 + \mathrm{Inflation}_t) \tag{21}$$

where OB is the base operating expense that increases with inflation each year (rent, insurance), and %OR is the rate of increase proportional to this year's sales revenue that impacts the variable portion of the operating expense. Inflation data can be obtained from U.S. Bureau of Labor Statistics.

We can model $\text{IT}_\text{t}$ as a percentage of net income ($\text{NI}_\text{t}$):

$$\text{IT}_t = \%\text{IT} \cdot \text{NI}_t/(1 - \%\text{IT}) = \text{NI}_t/(1/\%\text{IT} - 1) \tag{22}$$

We add %IT, $\text{OB}_{\text{T\_start}}$ (base operating expense of the first element in a time series) and %OR to our variables to be trained.

**Capital Expense NLB** Investment is made into the non-current assets to:

a) keep non-current assets price the same by accounting for depreciation

b) grow non-current assets by some percentage.

We have already done the calculation in (3), which we reproduce here:

$\text{CapEx}_t = \text{Depreciation}_t + \text{Sales}_t \cdot \%\text{Asset growth from (3)}$

$$\text{CapEx NLB}_t = -\text{CapEx}_t = -\text{Depreciation}_t - \text{Sales}_t \cdot \%\text{Asset growth} \tag{23}$$

**Financing NLB** This is the net liquidity balance after obtaining more liquidity from new short-term loan ($\text{STLoan}_\text{t}$) and new long-term loan ($\text{LTLoan}_\text{t}$), and paying all debt obligations this year.

$$\text{Financing NLB}_t = \text{STLoan}_t + \text{LTLoan}_t - \text{CLiab}_{t-1} - \text{Avg. ST interest}_t - \text{Avg. LT interest}_t \tag{24}$$

where current $\text{liabilities}_{\text{t-1}}$ ($\text{CLiab}_{\text{t-1}}$) are the principals from short-term liabilities and long-term liabilities paid this year, represented by:

$$\text{CLiab}_{t-1} = \text{STLoan}_{t-1} + \text{NLiab}_{t-1}/(\text{AvgM} - 1) \tag{25}$$

The average short-term interest ($\text{AvgSTInt}_\text{t}$) and average long-term interest ($\text{AvgLTInt}_\text{t}$) owed are calculated as follows:

$$\text{AvgSTInt}_t = \%\text{AvgSTInt} \cdot \text{STLoan}_{t-1} = \%\text{AvgSTInt} \cdot (\text{CLiab}_{t-1} - \text{NLiab}_{t-1}/(\text{AvgM} - 1)) \tag{26}$$

$$\text{AvgLTInt}_t = \%\text{AvgLTInt} \cdot \text{Total LT liability}_{t-1} = \%\text{AvgLTInt} \cdot (\text{NLiab}_{t-1}/(1 - 1/\text{AvgM})) \tag{27}$$

We add AvgM, %AvgSTInt and %AvgLTInt to our variables to learn. We derive the expression for $\text{STLoan}_\text{t}$ and $\text{LTLoan}_\text{t}$ in the *Liabilities* subsection.

**External Investment NLB** We redeem external investments from last year with an effective percentage of return (%MSReturn, derived from (EBT − EBIT + InterestExpense)/IMS historically, see (55)), and make new investments this year:

$$\text{External investment NLB}_t = \text{IMS}_{t-1} \cdot (1 + \%\text{MSReturn}) - \text{IMS}_t \tag{28}$$

Instead of only allocating the excess cash in market securities, we invest a proportion of total liquidity balance. Using (15):

$$\text{External investment NLB}_t = (1 - \%\text{Cash}) \cdot (\text{TL}_{t-1} \cdot (1 + \%\text{MSReturn}) - \text{TL}_t) \tag{29}$$

We add %MSReturn to our training variables to be learned.

**Transaction with Owners NLB** In Pareja(09)'s model, owners can make additional investments to finance a portion of long-term debt, which counts as an inflow to liquidity. The company pays dividends to owners and can also engage in stock buybacks:

$$\text{Owners transaction NLB}_t = \text{Equity financing}_t - \text{Dividends}_{t-1} - \text{Stock buybacks}_t \tag{30}$$

where

$$\text{Equity financing}_t = \text{LTLoan}_t \cdot \%\text{EF}/(1 - \%\text{EF}) \tag{31}$$

The equity financing proportion is trained from historical data, implicitly capturing the firm's revealed preference between debt and equity. This is consistent with the pecking order hypothesis [Myers(84)], which predicts firms prefer internal funds, then debt, then equity, resulting in a low and relatively stable %EF for profitable firms. The trained %EF thus reflects the firm's historical position on the pecking order rather than an arbitrary split.

Dividends are modeled with two policy variants:

**Simple policy (`SimpleDividendPolicy`).** Dividends are a fixed percentage of prior-year net income, with no memory of past payouts:

$$\text{Dividends}_{t-1} = \%\text{Payout ratio} \cdot \text{Net income}_{t-1} \tag{32}$$

**Advanced policy (`LintnerDividendPolicy`).** Rather than calculating dividends strictly as a fixed percentage of current net income, we apply the Lintner partial-adjustment model to account for dividend smoothing. Corporate management typically prefers stable dividend payouts to avoid signaling financial distress to investors. Therefore, the modeled dividend is represented as a weighted average between a target payout (driven by a baseline %Payout_Ratio) and the actual dividends paid in the prior period, regulated by an optimized Adjustment_Speed parameter:

$$\text{Dividends}_{t-1} = \text{Adj. Speed} \cdot (\%\text{PR} \cdot \text{NI}_{t-1}) + (1 - \text{Adj. Speed}) \cdot \text{Dividends}_{t-2} \tag{33}$$

The Lintner(56) partial-adjustment model remains empirically robust for mature dividend-paying firms [Brav(05)]. The key economic insight is that managers treat dividends as a commitment: cutting dividends signals financial distress, so firms smooth payouts to avoid involuntary future cuts. For mega-cap technology firms, the adjustment speed tends to be low (conservative smoothing), as these firms have ample retained earnings and prefer share buybacks for flexible capital return. The trained Adjustment_Speed parameter captures this firm-specific conservatism.

Stock buybacks are also modeled with two policy variants:

**Simple policy (`SimpleBuybackPolicy`).** Buybacks are proportional to depreciation, using the asset base as a scaling proxy:

$$\text{Stock buybacks}_t = \%\text{BB} \cdot \text{Depreciation}_t \tag{34}$$

**Advanced policy (`BaselineBuybackPolicy`).** An affine model that adds a fixed baseline component, capturing the observation that large firms often maintain a minimum buyback program independent of asset turnover:

$$\text{Stock buybacks}_t = \beta_{\text{base}} + \beta_{\text{ratio}} \cdot \text{Depreciation}_t \tag{35}$$

In both variants, any excess cash remaining after meeting the liquidity target is also distributed as additional buybacks.

For companies like Apple, their revenue generates tremendous amount of cash that they sometimes use the excess cash to buy back shares to spread the earnings to shareholders instead of paying a higher dividend which has tax implications. We add %Equity financing (%EF), %Payout ratio (%PR) and %Buyback ratio (%BB) to our variables to learn. We will define the equations for Net income$_t$ in the *Income Statement* subsection.

**Final NLB** We obtain the final net liquidity balance at time $t$ by summing all the different activities' NLBs:

$$\begin{aligned}\text{Final NLB}_t = \text{Operating NLB}_t + \text{CapEx NLB}_t + \text{Financing NLB}_t \\ + \text{External investment NLB}_t + \text{Owners transaction NLB}_t\end{aligned} \tag{36}$$

By construction, the Final NLB is the period's net cash inflow (the change in total liquidity from year $t-1$ to year $t$), so it must equal the difference between this year's and last year's total liquidity balances:

$$\text{Final NLB}_t = \text{TL}_t - \text{TL}_{t-1} \tag{37}$$

This identity is what closes the loop and makes training possible. Because the liquidity policy (Section 2.2.6) gives us an independent target for $\mathrm{TL}_t$ via (16), namely $\mathrm{TL}_t = \mathrm{TL}_{\text{baseline}} + \mathrm{Sales}_t \cdot \sigma(\alpha_{\mathrm{TL}} + \beta_{\mathrm{TL}} \cdot t)$. Substituting this into (37) yields a single scalar constraint that ties together every policy ratio appearing in the five NLBs. Specifically, the only free variable on the left-hand side is $\mathrm{LTLoan}_t$ (buried inside Financing $\mathrm{NLB}_t$), which can therefore be solved for endogenously as the residual financing needed to bring total liquidity to its policy target. Once $\mathrm{LTLoan}_t$ is determined, every remaining quantity is fully specified and the rest of the balance sheet assembles mechanically. No plug is required; the "balancing" is done by a financing decision that has real economic meaning (how much long-term debt to raise this period), not by absorbing arbitrary residuals into cash.

### 2.2.8 Account Payables

Account payables are modeled as a percentage of total purchases made this year, payable next year to the suppliers:

$$\mathrm{AP}_t = \%\mathrm{AP} \cdot \mathrm{Purchases}_t \tag{38}$$

We add %AP to our variables to be trained.

### 2.2.9 Advance Payments for Future Sales

Advance payments received from customers for future sales ($\mathrm{AdvPS}_t$) are modeled as a direct proportion of the current year's total sales. This assumes that the volume of upfront customer payments scales linearly with overall revenue generation. Therefore, we define:

$$\mathrm{AdvPS}_t = \%\mathrm{AdvPS} \cdot \mathrm{Sales}_t \tag{39}$$

We add %AdvPS to our variables to be trained.

### 2.2.10 Current Liabilities

To model the current liabilities, we first model the effective short-term loan ($\mathrm{STLoan}_t$) issued this year. Short-term loan covers any excess cash needed to cover operational expenses and last year's short-term loan obligation (principal + interest) due this year and bring the total liquidity to this year's liquidity target. We also account for the return from investment in market securities here because the size of investment is directly proportional to cash, **instead of** investing excess cash after ST and LT loan calculations which is done in Pareja(09):

$$\begin{aligned}\text{Short-term cash deficit}_t = \mathrm{TL}_t - \mathrm{TL}_{t-1} - \text{Operating NLB}_t \\ + \mathrm{STLoan}_{t-1} \cdot (1 + \%\mathrm{AvgSTInt}) - \%\mathrm{MSReturn} \cdot \mathrm{IMS}_{t-1}\end{aligned} \tag{40}$$

$$\mathrm{STLoan}_t = \max(0,\ \text{Short-term cash deficit}_t) \tag{41}$$

Then, we calculate the effective long-term loan ($\text{LTLoan}_t$) issued this year to finance $\text{CapEx}_t$, last year's current portion of the long-term liability, interest on long-term liability, and dividends to bring the liquidity to this year's target after accounting for the equity financing:

$$\begin{aligned}\text{LTLoan}_t = \max(0,\ &\text{Short-term cash deficit}_t - \text{STLoan}_t - \text{CapEx NLB}_t \\ &+ \text{NLiab}_{t-1}/(\text{AvgM} - 1) + \text{AvgLTInt}_t + \text{Dividends}_{t-1} + \text{Stock buybacks}_t) \cdot (1 - \%\text{EF})\end{aligned} \tag{42}$$

Current liabilities are summation of the effective short-term loan and the current portion of non-current liabilities, and are represented by:

$$\text{CLiab}_t = \text{STLoan}_t + \text{Total LT liability}_t/\text{AvgM} \tag{43}$$

where

$$\text{Total LT liability}_t = \text{LTLoan}_t + \text{NLiab}_{t-1} \tag{44}$$

### 2.2.11 Modeling Effective Short-Term Debt

The simple deficit-driven formulation of short-term borrowing does not fit the historical behavior of cash-rich firms like Apple. The following subsubsection motivates the advanced (logit-linear) policy variant and derives the metric used throughout the forecasting model.

**The Problem** A strictly deficit-driven approach calculates new short-term loans as a reaction to cash shortfalls. This forces short-term borrowing to zero when a company has surplus liquidity. However, in reality, large corporations (e.g., Apple) strategically carry significant commercial paper and revolving credit lines ($45B–$88B) despite holding massive cash reserves.

This means the short-term cash deficit is less than zero, i.e., cash surplus, which would result in no short-term borrowing for any year.

**Defining the Metric** Effective short-term debt isolates the operational and short-term financing liabilities that scale directly with the size of the business, stripping out strict trade payables and the current portion of long-term debt.

The balance sheet identity is defined as:

$$\begin{aligned}\text{Effective ST Debt}_t = \text{Current Liabilities}_t - \text{Accounts Payable}_t \\ - \text{Advance Payments Sales}_t - \text{Current LT Debt}_t\end{aligned} \tag{45}$$

Which equates to the core components we are modeling:

$$\text{Effective ST Debt}_t = \text{ST Debt}_t + \text{Accrued Expenses}_t + \text{Other Current Liabilities}_t \tag{46}$$

**The Solution & Implementation** To reflect actual corporate financing behavior, effective short-term debt is now modeled as a policy-driven metric rather than a deficit-driven reaction. It is projected as an affine function of sales, consisting of a fixed baseline and a sales-proportional component:

$$\text{Effective ST Debt}_t = \text{EffSTDebt}_{\text{baseline}} + \text{Sales}_t \cdot \%\text{STDebt} \tag{47}$$

Both $\text{EffSTDebt}_{\text{baseline}}$ and %STDebt are trainable parameters constrained to be non-negative via a softplus bijector. The baseline component captures fixed short-term obligations that do not scale from zero with sales (e.g., revolving facility minimums, notes payable commitments), while the sales-proportional component captures working-capital-driven borrowing. This affine form was chosen over a pure sales ratio because the latter would asymptote to zero at zero sales (unrealistic for firms that carry a standing level of short-term paper) and over a logit-linear form because the latter saturates as sales grow, which is inappropriate for firms whose borrowing scales proportionally with revenue.

**Liquidity Cascade** Because short-term debt is now an established structural policy, it acts as an *input* to the long-term liquidity deficit. The calculated effective short-term debt funds a portion of the required capital, shifting the role of the purely deficit-driven "plug" entirely to the long-term financing variable.

**Non-current Liabilities** Non-current liabilities include the new effective $\text{LTLoan}_t$ issued this year, and last year's non-current liabilities minus the current portion of this year's non-current liabilities.

$$\text{NLiab}_t = \text{Total LT liability}_t \cdot (1 - 1/\text{AvgM}) \tag{48}$$

### 2.2.12 Equity

This year's equity accounts for the additional equity financing this year, net income from this year, dividends paid out this year, and stock buyback this year:

$$\text{SE}_t = \text{SE}_{t-1} + \text{EF}_t + \text{NI}_t - \text{Dividends}_{t-1} - \text{BB}_t \tag{49}$$

where

$$\text{EF}_t = \text{LTLoan}_t/(1 - \%\text{EF}) \cdot \%\text{EF} \tag{50}$$

$$\text{Dividends}_{t-1} = \%\text{PR} \cdot \text{NI}_{t-1} \tag{51}$$

$$\text{BB}_t = \%\text{BB} \cdot \%\text{Depreciation} \cdot \text{NCA}_{t-1} + \text{any excess cash after meeting liquidity target} \tag{52}$$

We have most of what we need to learn the training variables except dividends and income tax, which will come from modeling the net income in the income statement.

### 2.2.13 Income Statement: EBITDA and EBT

We have all the components needed from the previous sections to construct the elements related to net income.

$\text{EBITDA}_t$ (earnings before interest, taxes, depreciation, and amortization) can be calculated by:

$$\text{EBITDA}_t = \text{Sales}_t - \text{COGS}_t - \text{OpEx}_t \tag{53}$$

Earnings before tax is calculated by subtracting interest payments, depreciation (and amortization):

$$\text{EBT}_t = \text{EBITDA}_t - \text{Depreciation}_t - \text{AvgSTInt}_t - \text{AvgLTInt}_t + \%\text{MSReturn} \cdot \text{IMS}_{t-1} \tag{54}$$

### 2.2.14 Derivation of Historical %MSReturn

The term $\%\text{MSReturn} \cdot \text{IMS}_{t-1}$ represents income from short-term investments in market securities. In practice, not all companies separately disclose this line item in their income statement; many lump it with other non-operating items (foreign exchange gains/losses, miscellaneous items, etc.) into a single "Other income/(expense), net" (OI&E) line. We therefore derive the historical values from the income statement identity:

$$\text{MSReturn}_t = \text{EBT}_t - \text{EBIT}_t + \text{InterestExpense}_t \tag{55}$$

This isolates all non-operating income/expense *except* interest expense (which the model already handles via %AvgSTInt and %AvgLTInt). For companies like Apple, where investment income dominates OI&E, this residual is a close proxy for the true return on the securities portfolio. For companies with significant foreign exchange exposure, it may also absorb FX gains/losses.

When the company's 10-K filing does not provide Interest Expense separately (as is the case for some fiscal years), we fall back to $\text{MSReturn}_t = \text{EBT}_t - \text{EBIT}_t$ and set interest expense to zero for those years, which lumps all non-operating items into MSReturn but preserves the EBT identity.

**Forward projection.** During forecasting, the model projects $\text{MSReturn}_t$ as a fixed percentage return on the prior period's IMS balance: $\%\text{MSReturn} \cdot \text{IMS}_{t-1}$. The effective rate %MSReturn is learned during training as the historical average of $\text{MSReturn}_t/\text{IMS}_{t-1}$, which absorbs the blended contribution of investment income and smaller non-operating items into a single rate parameter. This fixed-percentage return assumption is a simplification: for firms with large treasury portfolios, investment returns depend on duration, credit quality, and market conditions. The trained parameter captures the historical average net return after FX effects and mark-to-market adjustments.

### 2.2.15 Net Income and Dividends

Net income ($\text{NI}_t$) is determined by subtracting the income tax:

$$\text{NI}_t = \text{EBT} \cdot (1 - \%\text{IT}) \tag{56}$$

Next year's dividends were already modeled in the dividends section (p. 17).

We now have everything we need to represent every element in the balance sheet as a time series.

## 2.3 Time Series Representation of the Balance Sheet

We re-write every element in the balance sheet as a time series that have been simplified by combining the equations above. The primary drivers are $\text{Sales}_t$ and $\text{Purchases}_t$, which drive every element given the variables to be trained are learned.

**Variables to be trained**

- %AG : Asset growth rate as a percentage of non-current assets
- %AM : Asset maintenance ratio as a percentage of depreciation
- %Depr : Depreciation rate as a percentage of non-current assets
- %AdvPS : Advance payments from customers as a percentage of next year's sales
- %AdvPP : Advance payments to suppliers as a percentage of next year's purchases
- %AR : Account receivables as a percentage of this year's sales
- %AP : Account payables as a percentage of this year's purchases
- %Inv : Inventory target as a percentage of this year's sales
- $\alpha_{\text{CR}}$ and $\beta_{\text{CR}}$ : Variables that make up the Cost Ratio via logit model
  - $\text{logit}(\text{CR}_t) = \ln\left(\text{CR}_t \,/\, (1 - \text{CR}_t)\right) = \alpha_{\text{CR}} + \beta_{\text{CR}} * t$
- $\text{TL}_{\text{baseline}}$, $\alpha_{\text{TL}}$, $\beta_{\text{TL}}$ : Baseline floor and logit-linear trend parameters for total liquidity with a sales-proportional component: $\text{TL}_t = \text{TL}_{\text{baseline}} + \text{Sales}_t \cdot \sigma(\alpha_{\text{TL}} + \beta_{\text{TL}} \cdot t)$
- $\alpha_{\text{Cash}}$, $\beta_{\text{Cash}}$ : Logit-linear trend parameters for the cash share of total liquidity: $\text{Cash}_t = \text{TL}_t \cdot \sigma(\alpha_{\text{Cash}} + \beta_{\text{Cash}} \cdot t)$
- %IT : Income tax as a percentage of net income this year
- %OR : Variable portion of OpEx that increases in proportion to this year's sales
- $\text{OB}_{\text{T\_start}}$ : Baseline OpEx that increases with this year's average inflation
- $\text{EffSTDebt}_{\text{baseline}}$ and %STDebt : Baseline and sales-proportional parameters for calculating effective short-term debt ($\text{Effective ST Debt}_t = \text{EffSTDebt}_{\text{baseline}} + \text{Sales}_t \cdot \%\text{STDebt}$)
- %AvgSTInt : Average short-term interest on the effective short-term loan

- %AvgLTInt : Average interest on total long-term liabilities
- AvgM : Average maturity of total long-term liability
- %MSReturn : Effective percentage return on the investment in market securities, derived from historical (EBT − EBIT + InterestExpense)/IMS
- $\alpha_{\text{EF}}$, $\beta_{\text{EF}}$ : Logit-linear parameters for the equity financing mix, $\%\text{EF}_t = \sigma(\alpha_{\text{EF}} + \beta_{\text{EF}} \cdot t)$, where %EF is the fraction of long-term financing raised as equity rather than debt
- %PR : Target dividend payout ratio (fraction of net income) in the Lintner partial-adjustment model
- Adj. Speed : Lintner partial-adjustment speed controlling how quickly actual dividends converge to the target payout $\%\text{PR} \cdot \text{NI}_{t-1}$
- $\text{BB}_{\text{baseline}}$ and $\text{BB}_{\text{ratio}}$ : Fixed baseline and depreciation-proportional parameters for stock buyback in the affine buyback policy ($\text{BB}_t = \text{BB}_{\text{baseline}} + \text{BB}_{\text{ratio}} \cdot \text{Depreciation}_t$)

**Parameter Constraints and Bounds** To ensure the model generates economically logical and mathematically stable financial statements, strict constraints are enforced on both the trainable variables and the underlying equations. These constraints are implemented natively within the optimizer using TensorFlow Probability Bijectors.

### 2.3.1 Domain Constraints on Trainable Variables

- **Strict Positivity ($x > 0$):** Variables that cannot logically be negative, such as the Depreciation Rate (%Depr), Asset Growth (%AG), and Interest Rates (%AvgSTInt, %AvgLTInt), are constrained using a Softplus activation function. This prevents the optimizer from testing negative rates during gradient descent.
- **Percentage Bounds ($0 < x < 1$):** Variables representing fractions of a whole, such as the Income Tax Percentage (%IT), Dividend Payout Ratio (%PR), and the Lintner Dividend Adjustment Speed ($\alpha$), are constrained using a Sigmoid function.
- **Maturity Boundary (AvgM $> 1.001$):** Because the calculation for the current portion of long-term debt divides by (AvgM − 1), the average maturity must strictly exceed 1 year to prevent mathematical singularity and infinite liability generation. This is enforced using a chained bijector: a Softplus function shifted by a constant of 1.001.
- **Trended Ratios:** Time-varying policies, such as the Cost Ratio (CR) and Effective Short-Term Debt percentage, are modeled as logit-linear trends. By wrapping the linear trend ($\alpha + \beta t$) in a sigmoid function, the forecasted ratios are strictly bounded between 0 and 1, even if extrapolated decades into the future.

### 2.3.2 Logical Constraints on Equations

- **Non-Negative Borrowing:** A company cannot issue "negative" debt to reduce cash. In the Cash Budget equations, the issuance of new short-term and long-term loans is constrained using a maximum function: $\text{New\_Loan}_t =$

$\max(0, \text{Liquidity\_Deficit}_t)$. If there is a cash surplus, borrowing falls to exactly zero, and the excess cash cascades into stock buybacks.

- **Structural Identity Check:** The fundamental accounting equation (Assets = Liabilities + Equity) is strictly evaluated at every time step $t$. Rather than enforcing this via a loss function penalty (which would allow minor deviations), it is enforced structurally by the double-entry routing of the Cash Budget. The simulation calculates a delta to verify that the mismatch is exactly zero.

**Variables from External Environment** Since we assume a flat short-term and long-term interest for loans and return on market securities, the only external variable is inflation. This flat-rate assumption is a first-order simplification. In practice, borrowing costs depend on the firm's leverage and credit quality [Merton(74)]. As the forecast shows rising debt levels, a more complete model would incorporate credit spread widening, a natural extension that would link the cost of debt to the evolving debt-to-equity ratio. The current approach is conservative: it understates the cost of increased leverage, producing optimistic net income projections for high-debt scenarios.

- $\text{Inflation}_t$ : Average inflation at year t

**Assets**

- Non-current assets
  - $\text{NCA}_t = \text{NCA}_{t-1} - \text{Depreciation}_t + \text{CapEx}_t$
    - Where $\text{CapEx}_t = \text{Depreciation}_t + \text{Sales}_t * \%\text{AG}$
- Advance payments to suppliers for purchases
  - $\text{AdvPP}_t = \%\text{AdvPP} * \text{Purchases}_t$
- Account receivables
  - $\text{AR}_t = \%\text{AR} * \text{Sales}_t$
- Inventory
  - $\text{Inv}_t = \%\text{Inv} * \text{Sales}_t$
- Total liquidity ($\mathbf{TL_t}$)
  - Target $\text{TL}_t = \%\text{TL} * \text{Sales}_t$
    - Investments in market securities
      - $\text{IMS}_t = (1 - \%\text{Cash}) * \text{TL}_t$
    - Cash equivalent
      - $\text{Cash}_t = \%\text{Cash} * \text{TL}_t$
  - Should be equivalent to: Final $\text{TL}_t$ = Operating $\text{NLB}_t$ + CapEx $\text{NLB}_t$ + Financing $\text{NLB}_t$ + External investment $\text{NLB}_t$ + Owners transactions $\text{NLB}_t$

**Liabilities**

- Account payables
  - $AP_t$ = %AP * $Purchases_t$
- Advance payments from customers for sales
  - $AdvPS_t$ = %AdvPS * $Sales_t$
- Current liabilities
  - $CLiab_t$

  = $STLoan_t$ + $NLiab_t$ / (AvgM – 1)

  where
- $STLoan_t$ = max(0, $STCashDeficit_t$)
- $STCashDeficit_t$ = $TL_t$ – $TL_{t-1}$ – Operating $NLB_t$ + $STLoan_{t-1}$ * (1 + %AvgSTInt) – %MSReturn * $IMS_{t-1}$
- $STLoan_{t-1}$ = $CLiab_{t-1}$ - $NLiab_{t-1}$ / (AvgM – 1)
- Operating $NLB_t$ = $AR_{t-1}$ + $AdvPS_t$ + $Sales_t$ * (1 - %AR - %AdvPS) – ($AP_{t-1}$ + $AdvPP_t$ + $Purchases_t$ * (1 - %AP - %AdvPP) + $OpEx_t$ + $IT_t$)
- $AP_{t-1}$ = %AP * $Purchases_{t-1}$
- $AdvPP_t$ = %AdvPP * $Purchases_t$
- $OpEx_t$ = %OR * $Sales_t$ + $OB_{t-1}$ * (1 + $Inflation_t$)
- $IT_t$ = $NI_t$ / (1/%IT - 1)
- $AR_{t-1}$ = %AR * $Sales_{t-1}$
- $AdvPS_t$ = %AdvPS * $Sales_t$
- Non-current liabilities
  - $NLiab_t$

  = ($LTLoan_t$ + $NLiab_{t-1}$) * (AvgM – 1)/AvgM

  where
- $LTLoan_t$ = (max(0, $STCashDeficit_t$ – $STLoan_t$ – CapEx $NLB_t$ + $NLiab_t$ / (AvgM – 1) + $AvgLTInt_t$ + $Dividends_{t-1}$ + Stock $buybacks_t$) * (1 - %EF)
- CapEx $NLB_t$ = $NCA_{t-1}$ * %Depr + %AG * $Sales_t$
- $Dividends_{t-1}$ = %PR * $NI_{t-1}$
- $NI_{t-1}$ : net income from last year

  = (1 - %IT) * ($Sales_{t-1}$ – $COGS_{t-1}$ – $OpEx_{t-1}$ – $Depreciation_{t-1}$ – $AvgSTInt_{t-1}$ – $AvgLTInt_{t-1}$ + %MSReturn * (1 - %Cash) * $TL_{t-1}$)
- $TL_{t-1}$ = Total liquidity last year.
- $COGS_{t-1}$ = %Inv * ($Sales_{t-2}$ – $Sales_{t-1}$) + $Purchases_{t-1}$

- $\text{OpEx}_{t\text{-}1}$ = %OR * $\text{Sales}_{t\text{-}1}$ + $\text{OB}_{t\text{-}2}$ * (1 + $\text{Inflation}_{t\text{-}1}$)
- $\text{Depreciation}_t$ = $\text{NCA}_{t\text{-}1}$ * %Depr
- $\text{AvgSTInt}_t$ = %AvgSTInt * ($\text{CLiab}_{t\text{-}t}$ – $\text{NLiab}_{t\text{-}t}$ / (AvgM – 1))
- $\text{AvgLTInt}_t$ = %AvgLTInt * ($\text{NLiab}_{t\text{-}1}$ / (1 – 1/AvgM))
- Stock $\text{buybacks}_t$ = %BB * $\text{Depreciation}_t$ + excess cash after meeting liquidity target

**Equity**

- Stockholder equity

  – $\text{SE}_t$

  = $\text{SE}_{t\text{-}1}$ + $\text{LTLoan}_t$ / (1 - %EF) * %EF + $\text{NI}_t$ – %PR * $\text{NI}_{t\text{-}1}$ – %BB * %Depr * $\text{NCA}_{t\text{-}1}$ – excess cash after meeting liquidity target

We see in all elements' equations that they are driven by Sales and Purchases and elements from previous years, indicating no circularity or plugs. The net liquidity balance calculation and the new short-term and long-term borrowing equations ensure that the assets and liabilities + equity are increased/decreased the same. Thus, assets = liabilities + equity will always be satisfied. We can check with test values in our simple TensorFlow model.

This forward model has been implemented in Python in `run_simple_model_forecast.py`, which composes the modular policy classes from the `financial_forecast/` package using data from Apple's financial statements. The financial statements were first obtained from Yahoo finance as suggested by Question 4 (we wrote the script in `yahoofinance.py`), but we could only access past 4 years of data, which is not sufficient for training in the following section. We managed to obtain older financial data by directly pulling from Apple's 10-K filings. The script that we wrote and used for the data extraction is found in `run_build_financial_statements_sec.py`.

The following is the output of the code when run with `python run_simple_model_forecast_demo.py`, which uses Apple's FY2024 balance sheet as the initial state, approximates every policy variable from the most recent year's ratios, holds them constant, and simply iterates four annual forecast steps:

| Line Item | FY2025 | FY2026 | FY2027 | FY2028 |
|---|---|---|---|---|
| **Assets** | | | | |
| Non-Current Assets | 211.2 | 211.0 | 210.9 | 210.8 |
| Adv Payments (Purch) | 17.5 | 18.4 | 19.3 | 20.2 |
| Accounts Receivable | 67.4 | 70.9 | 74.4 | 77.9 |
| Inventory | 6.5 | 6.9 | 7.2 | 7.5 |
| Cash | 42.7 | 44.9 | 47.1 | 49.3 |
| Invest in Mkt Sec | 0.0 | 0.0 | 0.0 | 0.0 |
| **Total Assets** | **345.3** | **352.1** | **358.9** | **365.7** |
| **Liabilities** | | | | |
| Accounts Payable | 71.2 | 74.7 | 78.4 | 82.1 |
| Adv Payments (Sales) | 9.0 | 9.4 | 9.9 | 10.4 |
| Effective ST Debt | 0.0 | 0.0 | 0.0 | 0.0 |
| Current LT Debt | 13.5 | 14.5 | 15.4 | 16.2 |
| Non-Current Liabilities | 193.4 | 207.7 | 220.3 | 230.9 |
| **Total Liabilities** | **287.1** | **306.4** | **324.1** | **339.5** |
| **Equity** | **58.2** | **45.7** | **34.8** | **26.2** |
| **Total Liab + Equity** | **345.3** | **352.1** | **358.9** | **365.7** |
| **Income Statement** | | | | |
| Net Income | 74.6 | 80.8 | 83.7 | 86.7 |
| CHECK: Assets−(L+E) | 0.0 | 0.0 | 0.0 | 0.0 |

All figures in $ billions.

Even without using any plugs, we see that the assets = liabilities + equity identity is satisfied because our model follows the double-entry principle with any inflows and outflows on the balance sheet as done by Pareja(09).

## 2.4 Training the Model

The model's trainable parameters are split into two groups and trained in two sequential phases. **Policy-phase** parameters directly govern individual working-capital, liquidity, OpEx, dividend, and buyback line items, and can each be anchored to a corresponding observable series in the historical financial statements; they are trained independently by minimizing mean-squared error between predicted and observed values. **Structural-phase** parameters (interest rates, debt maturity, market securities return, equity financing mix) affect Net Income, Equity, and the liability side of the Balance Sheet only through the full multi-year forecast rollout via `forecast_step` transitions; they are therefore trained jointly after the policy phase, with the policy parameters held fixed.

The primary forecast driver for both phases is:

- $\text{Sales}_t$ : Total revenue (Income Statement)

**Policy-phase variables.** Grouped by the policy module that owns them:

- **Non-current assets (`CapexPolicy`).** %Depr (effective depreciation rate as a fraction of beginning NCA), %AM (maintenance CapEx multiplier on Depreciation),

and %AG (growth CapEx as a fraction of Sales). Learned from historical NCA, Depreciation, and Sales.

- **Cost ratio (`TrendCostRatioPolicy`).** $\alpha_{\text{CR}}$, $\beta_{\text{CR}}$ — logit-linear cost ratio parameters such that $\text{CR}_t = \sigma(\alpha_{\text{CR}} + \beta_{\text{CR}} \cdot t)$. Learned from historical COGS and Sales.
- **Working capital (`WorkingCapitalPolicy`).** %AR (AR / Sales), %AP (AP / Purchases), %AdvPS (advance payments from customers / Sales), %AdvPP (advance payments to suppliers / Purchases), and %Inv (Inventory / COGS). Each learned from its respective historical ratio.
- **Total liquidity (`TrendLiquidityPolicy`).** $\text{TL}_{\text{baseline}}$, $\alpha_{\text{TL}}$, $\beta_{\text{TL}}$ for total liquidity with a baseline floor and a logit-linear sales-proportional component: $\text{TL}_t = \text{TL}_{\text{baseline}} + \text{Sales}_t \cdot \sigma(\alpha_{\text{TL}} + \beta_{\text{TL}} \cdot t)$. Plus $\alpha_{\text{Cash}}$, $\beta_{\text{Cash}}$ for the cash share of liquidity: $\text{Cash}_t = \text{TL}_t \cdot \sigma(\alpha_{\text{Cash}} + \beta_{\text{Cash}} \cdot t)$. Learned from historical Cash, short-term investments, and Sales.
- **Effective short-term debt (`TrendDebtPolicy`).** $\text{EffSTDebt}_{\text{baseline}}$ and %STDebt for the affine ST debt policy: $\text{EffSTDebt}_t = \text{EffSTDebt}_{\text{baseline}} + \text{Sales}_t \cdot$ %STDebt. Learned from historical Effective ST Debt and Sales.
- **Operating expense (`SimpleOpEx`).** %OR (variable OpEx rate as a slope on sales deviation from the historical mean) and $\text{OB}_{T\_\text{start}}$ (baseline OpEx, accumulated with cumulative inflation). Learned from historical OpEx, Sales, and inflation.
- **Income tax (`SimpleTax`).** %IT — effective income tax rate. Learned from historical Net Income and income tax provision.
- **Dividends (`LintnerDividendPolicy`).** %PR (target payout ratio as a fraction of Net Income) and Adj. Speed (Lintner partial-adjustment speed: how quickly actual dividends converge to the target). Learned from historical dividend payouts and Net Income.
- **Stock buyback (`BaselineBuybackPolicy`).** $\text{BB}_{\text{baseline}}$ (fixed baseline buyback amount) and $\text{BB}_{\text{ratio}}$ (multiplier on Depreciation for the variable component), giving an affine buyback model $\text{BB}_t = \text{BB}_{\text{baseline}} + \text{BB}_{\text{ratio}} \cdot \text{Depreciation}_t$. Learned from historical repurchases and Depreciation.

Each variable in this group can be trained independently against its corresponding observable series using mean-squared error minimization under Adam via TensorFlow's `GradientTape`.

**Structural-phase variables.** Trained jointly after the policy phase converges:

- %AvgSTInt : Average interest rate on Effective Short-Term Debt.
- %AvgLTInt : Average interest rate on total long-term liabilities.
- AvgM : Average maturity of the long-term debt portfolio.
- %MSReturn : Effective return on Investments in Market Securities, derived from historical (EBT − EBIT + InterestExpense)/IMS.
- $\alpha_{\text{EF}}$, $\beta_{\text{EF}}$ : Logit-linear equity financing mix such that $\%\text{EF}_t = \sigma(\alpha_{\text{EF}} + \beta_{\text{EF}} \cdot t)$, where %EF is the fraction of long-term financing raised as equity rather than debt.

These parameters affect Net Income, Equity, and the liability side of the Balance Sheet only through the forward rollout, so they are trained concurrently by rolling out the full forecast and minimizing MSE between predicted and observed Net Income, Stockholders' Equity, Current Liabilities, and Non-current Liabilities.

The code has been implemented in `run_trainable_model_forecast_simple_policies.py`, which composes the `TrainableFinancialModel` with simple policy modules and runs the `PolicyTrainer` and `StructuralTrainer`. For this simple training model, %AM = 100% and Purchases are also used together with Sales as dual driver.

In order to test the model's prediction capability, we train the parameters from Apple's 2018~2024 data and leave 2025 for backtesting, and forecast 9 more years to observe the trend. The following is the output after running `run_trainable_model_forecast_adv_policies.py`:

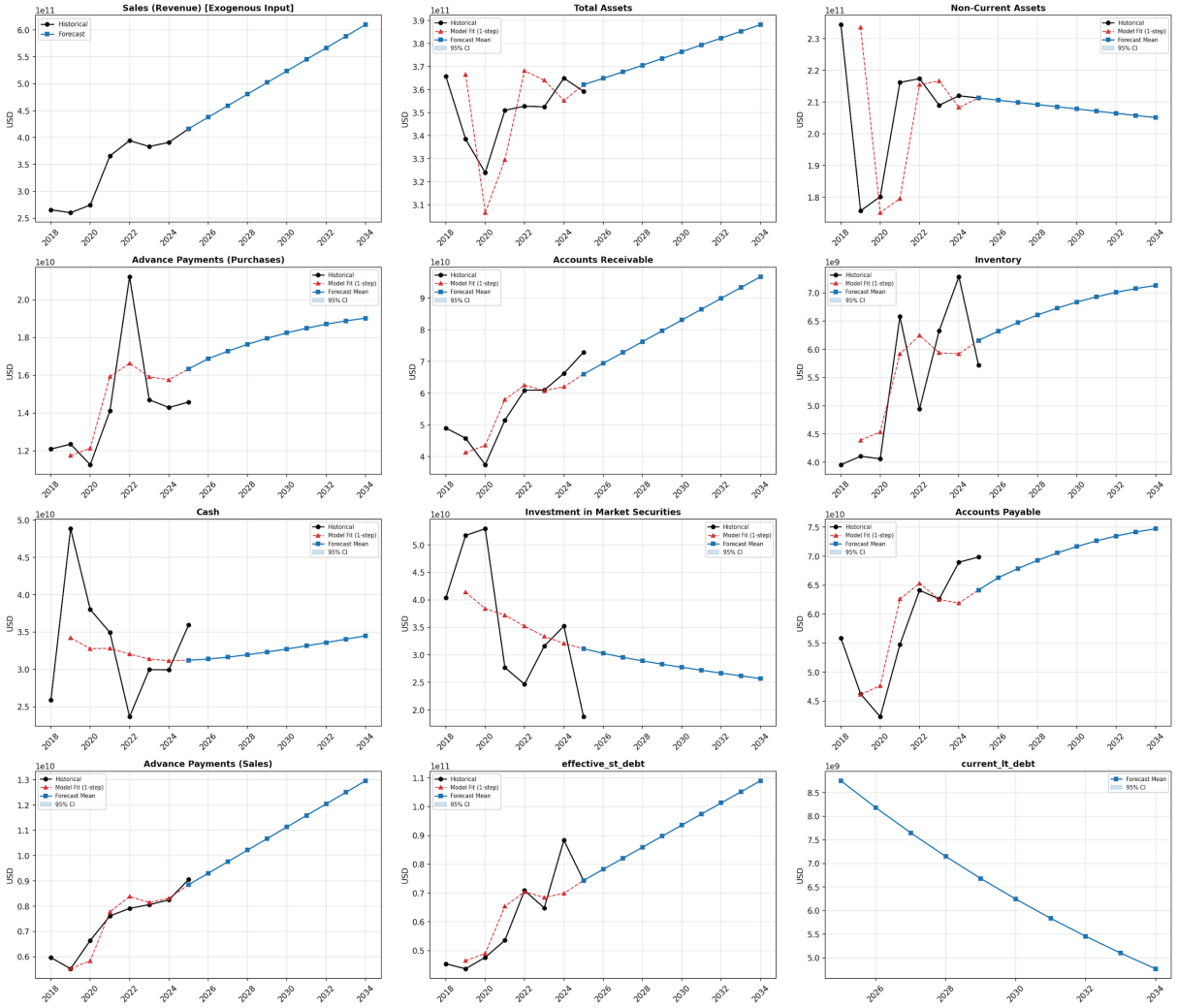
```
Simple parameters:
Final %AG: 0.00000
Final %AM: 0.94058
Final %Depr: 0.05552
Final %AdvPS: 0.02126
Final %AdvPP: 0.07840
Final %AR: 0.15866
Final %AP: 0.30796
Final %Inv (COGS): 0.02939
Total Liquidity (baseline + logit-linear): baseline=0.6012, alpha=-2.3463,
    beta=-0.411515
  => %TL at t=0: 0.0874, %TL at t=6: 0.0080
Cash % of Liquidity (logit-linear): alpha=-0.2235, beta=0.032375
  => %Cash at t=0: 0.4444, %Cash at t=6: 0.4927
Final %IT: 0.17023
Final %PR: 0.99973
Final DivAdjSpeed: 0.00370
Stock Buyback (baseline + ratio*depr): baseline=1.5612, ratio=-6.674733
Effective ST Debt (baseline + % of sales): baseline=0.0000, pct=0.17869
Cost Ratio (logit-linear): alpha=0.3459, beta=-0.047524
  => CR at t=0: 0.5856, CR at t=6: 0.5152
OpEx Variable %: 0.0939
OpEx Baseline (USD): 4.02e+10
```

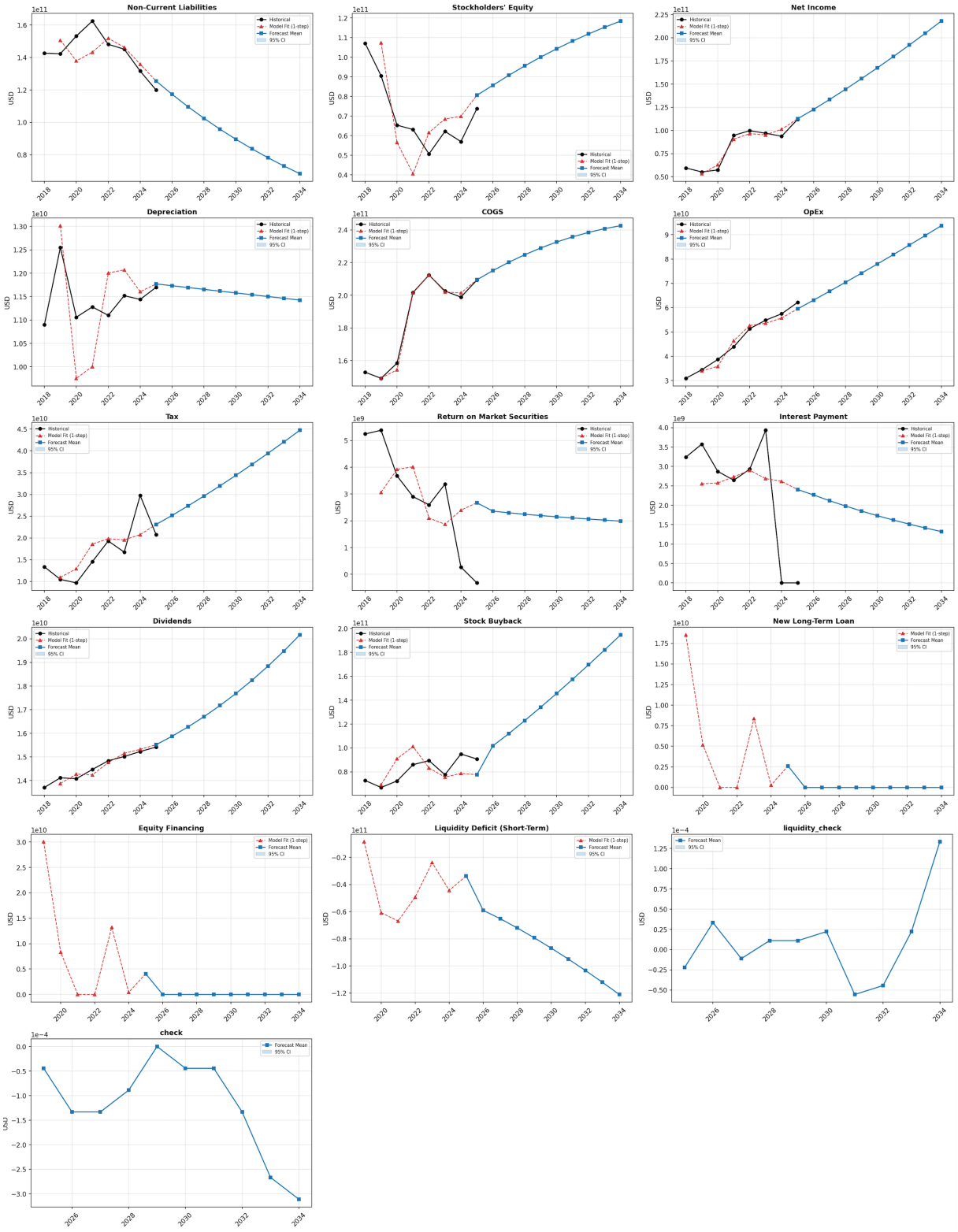
```
Structural parameters:

Final %AvgSTInt: 0.00003
Final %AvgLTInt: 0.01687
Final %MSReturn: 0.07588
Equity Financing (logit-linear): alpha=0.4894, beta=-0.006361
Avg Maturity Years: 15.3410
```

**BACKTEST — FY2025 (Out-of-Sample)**

| Line Item | Actual | Predicted | Error % |
|---|---|---|---|
| **Assets** | | | |
| Non-Current Assets | 211.3 | 211.3 | +0.0% |
| Adv Payments (Purchases) | 14.6 | 16.3 | +12.0% |
| Accounts Receivable | 73.0 | 66.0 | −9.5% |
| Inventory | 5.7 | 6.2 | +7.7% |
| Cash | 35.9 | 31.2 | −13.1% |
| Invest in Mkt Securities | 18.8 | 31.1 | +65.9% |
| **Total Assets** | **359.2** | **362.2** | +0.8% |
| **Liabilities** | | | |
| Accounts Payable | 69.9 | 64.2 | −8.2% |
| Adv Payments (Sales) | 9.1 | 8.8 | −2.3% |
| Effective ST Debt | 74.4 | 74.4 | +0.0% |
| Non-Current Liabilities | 119.9 | 125.5 | +4.6% |
| **Equity** | **73.7** | **80.6** | +9.3% |
| **Income Statement** | | | |
| COGS | 209.3 | 209.5 | +0.1% |
| OpEx | 62.2 | 59.5 | −4.3% |
| Depreciation | 11.7 | 11.8 | +0.6% |
| Interest Payments | 0.0 | 2.4 | N/A |
| ST Investment Returns | −0.3 | 2.7 | +932.8% |
| Tax | 20.7 | 23.1 | +11.5% |
| **Net Income** | **112.0** | **112.6** | +0.6% |
| **Cash Flow** | | | |
| Dividends | 15.4 | 15.5 | +0.6% |
| Stock Buyback | 90.7 | 77.6 | −14.5% |

All figures in $ billions.


Sales (Revenue) [Exogenous Input]
Historical
Forecast
Total Assets
Historical
Model Fit (1-step)
Forecast Mean
95% CI
Non-Current Assets
Historical
Model Fit (1-step)
Forecast Mean
95% CI
Advance Payments (Purchases)
Historical
Model Fit (1-step)
Forecast Mean
95% CI
Accounts Receivable
Historical
Model Fit (1-step)
Forecast Mean
95% CI
Inventory
Historical
Model Fit (1-step)
Forecast Mean
95% CI
Cash
Historical
Model Fit (1-step)
Forecast Mean
95% CI
Investment in Market Securities
Historical
Model Fit (1-step)
Forecast Mean
95% CI
Accounts Payable
Historical
Model Fit (1-step)
Forecast Mean
95% CI
Advance Payments (Sales)
Historical
Model Fit (1-step)
Forecast Mean
95% CI
effective_st_debt
Historical
Model Fit (1-step)
Forecast Mean
95% CI
current_lt_debt
Forecast Mean
95% CI
USD




The backtest shows mixed but informative results. On the income statement side, the model predicts Net Income within 0.6% of the actual figure, and COGS and Depreciation are nearly exact (+0.1% and +0.6%), suggesting that the trained cost ratio and CapEx parameters generalize well out of sample. Balance sheet aggregates are also close: Total Assets err by only +0.8%.

The larger item-level errors reflect known modelling simplifications. We have lumped various balance sheet elements together to be more consistent with the models described

in Pareja(07) and Pareja(09), as explained in the *Balance Sheet Elements* subsection, so individual line items such as Accounts Receivable (−9.5%) and Accounts Payable (−8.2%) absorb the aggregation mismatch even when the totals are accurate.

The Interest Payments actual value is reported as zero because Apple's FY2025 10-K does not disclose a single comparable interest expense figure in the format our extraction pipeline expects, so the error percentage is not meaningful. Similarly, the ST Investment Returns actual value is −$0.3B (a small net loss on the portfolio), while the model predicts $2.7B, yielding a +932.8% error. This is driven by the same liquidity-split limitation discussed below.

The largest outlier is Investment in Market Securities (+65.9%). This is a direct consequence of the logit-linear liquidity policy: the trend model captures the smooth, secular decline in Apple's total liquidity ratio but cannot reproduce the jagged year-to-year reallocation between cash and market securities, which is driven by discrete treasury decisions (e.g., shifting maturities in the bond portfolio) rather than a smooth function of sales or time. Cash itself is underestimated by 13.1% for the same reason. Because the model overestimates the IMS balance, it also overestimates the return generated on those securities, explaining the ST Investment Returns miss. A richer liquidity model that distinguishes active rebalancing from trend-driven allocation would improve these individual estimates, though the total liquidity error remains modest.

### 2.5 Applying Machine Learning

As our model is rather a simple model based on Pareja(09), there is a lot of improvements that could make the model more performant. A short list of potential improvements is as follows:

- Predicting purchases and sales of future years using Long Short-Term Memory and macroeconomic indicators (e.g., inflation, interest rate)
- Implementing Bayesian model to account for probability distribution of accounting variables (e.g., company policy for capital expenditure, operating expense prediction, cash balance targets) to account for the noise term, n(t).
- Implementing neural networks for each accounting variables that take into consideration macroeconomic signals (e.g., incorporate consumer price index, GDP growth, and interest rates to predict advance purchases percentage)
- Incorporating the company's stock buyback target if it has been disclosed publicly into training the buyback ratio with a neural network
- Endogenizing the Weighted Average Cost of Capital (WACC) as a function of the forecasted capital structure. Since WACC depends on the debt-to-equity ratio and cost of debt (both of which change across the forecast horizon), this would link borrowing costs to leverage and enable DCF-based valuation directly from the model output.
- Propagating uncertainty across all key drivers (sales growth, cost ratios, interest rates, CapEx intensity) via full Monte Carlo simulation, not just OpEx. Adding scenario analysis (base case, downside with revenue contraction, upside with margin expansion) would strengthen the forecast's decision-support value.

- Computing projected financial ratios (debt-to-equity, interest coverage, current ratio) from the forecast output and flagging years where they breach investment-grade thresholds, enabling proactive capital structure management.

For our model, we choose to implement Bayesian learning with Variational Inference method [Dürr(20)] to train the operational expense because

a. OpEx is influential in the net income value

b. OpEx, in our model, is heavily influenced by sales revenue, which is one of the main drivers in our model, up to 23%

Applying Variational Inference allows us to model the aleatoric and epistemic uncertainties of OpEx by calculating the posterior distribution of OpEx's linear regression parameters given the past OpEx, sales, and inflation data instead of a singular set of parameters (i.e., slope and offset) that a simple linear regression gives you. It allows us to predict the OpEx, and therefore the earnings and other balance sheet elements, with a confidence interval.

Most of the code from the simple trainable model remains unchanged except how we model the OpEx during training. The OpEx module is swapped from `SimpleOpEx` to `BayesianOpEx`, and the trajectory simulator from `DeterministicSimulator` to `MonteCarloSimulator`. The following updates are made:

1. Bayesian Conversion: Replaced the deterministic %OR and $\text{OB}_{\text{T\_start}}$ variables with Variational Posteriors.

- We learn a Mean ($\mu$) and a Standard Deviation($\sigma$) for each of these variables.
- During training/inference, we sample their values from their distributions:
  - Value = $\mu + \sigma * \epsilon$, where $\epsilon \sim \mathcal{N}(0, 1)$

2. Loss Function: Replaced the simple Mean Squared Error (MSE) for OpEx with Negative Log Likelihood + KL Divergence.

- This balances fitting the data (Likelihood) with keeping the distribution close to our prior beliefs (KL).

3. Monte Carlo Simulation: Added a run_monte_carlo_forecast function. Instead of predicting one single Net Income line, we run the forecast 1,000 times, sampling different OpEx values each time, to generate Confidence Intervals.

So, to answer the 'Hint', we are improving the model such that when we predict the earnings (i.e. net income), we can now model n(t) with the confidence intervals given by the Monte Carlo simulation. After we have trained all the variables, they become the very function that take in this year's y(t) and input x(t) to predict y(t+1). The input x(t) includes the primary drivers, which are sales and purchases time series, and inflation, which is the external environment variable that we include in the model for predicting the operating expense.

We implemented the updated Bayesian model in `run_trainable_model_forecast_adv_policies_w_bayesianopex.py`, which composes the `TrainableFinancialModel` with `BayesianOpEx`, `MonteCarloSimulator`, and all advanced trend-based policies

(`TrendLiquidityPolicy`, `LintnerDividendPolicy`, etc.) but with a `SimpleTax` module (no tax anomalies). The output is reproduced below:

```
Simple parameters:

Final %AG: 0.00000
Final %AM: 0.94058
Final %Depr: 0.05552
Final %AdvPS: 0.02126
Final %AdvPP: 0.07840
Final %AR: 0.15866
Final %AP: 0.30796
Final %Inv (COGS): 0.02939
Total Liquidity (baseline + logit-linear): baseline=0.6012, alpha=-2.3463,
    beta=-0.411515
  => %TL at t=0: 0.0874, %TL at t=6: 0.0080
Cash % of Liquidity (logit-linear): alpha=-0.2235, beta=0.032375
  => %Cash at t=0: 0.4444, %Cash at t=6: 0.4927
Final %IT: 0.17023
Final %PR: 0.99973
Final DivAdjSpeed: 0.00370
Stock Buyback (baseline + ratio*depr): baseline=1.5612, ratio=-6.674733
Effective ST Debt (baseline + % of sales): baseline=0.0000, pct=0.17869
Cost Ratio (logit-linear): alpha=0.3459, beta=-0.047524
  => CR at t=0: 0.5856, CR at t=6: 0.5152
Bayesian OpEx Variable %: Mean=0.0930, Std=0.0161
Bayesian OpEx Baseline (USD):   Mean=3.93e+10, Std=8.27e+08
OpEx aleatoric uncertainty (USD): 2.45e+09
```

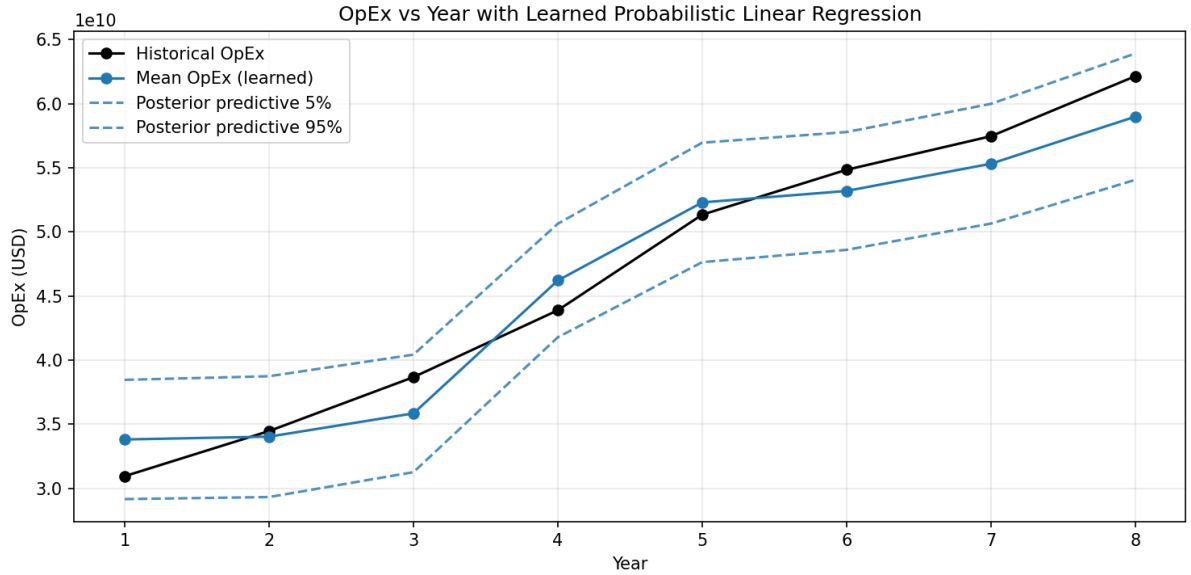
```
Structural parameters:

Final %AvgSTInt: 0.00003
Final %AvgLTInt: 0.01687
Final %MSReturn: 0.07587
Equity Financing (logit-linear): alpha=0.4917, beta=-0.005617
Avg Maturity Years: 15.3390
```

**BACKTEST — FY2025 (Out-of-Sample)**

| Line Item | Actual | Pred. (Mean) | Lower 2.5% | Upper 97.5% | Error % |
|---|---|---|---|---|---|
| **Assets** | | | | | |
| Non-Current Assets | 211.3 | 211.3 | 211.3 | 211.3 | +0.0% |
| Adv Payments (Purch.) | 14.6 | 16.3 | 16.3 | 16.3 | +12.0% |
| Accounts Receivable | 73.0 | 66.0 | 66.0 | 66.0 | −9.5% |
| Inventory | 5.7 | 6.2 | 6.2 | 6.2 | +7.7% |
| Cash | 35.9 | 31.2 | 31.2 | 31.2 | −13.1% |
| Invest in Mkt Sec | 18.8 | 31.1 | 31.1 | 31.1 | +65.9% |
| **Total Assets** | **359.2** | **362.2** | **362.2** | **362.2** | +0.8% |
| **Liabilities** | | | | | |
| Accounts Payable | 69.9 | 64.2 | 64.2 | 64.2 | −8.2% |
| Adv Payments (Sales) | 9.1 | 8.8 | 8.8 | 8.8 | −2.3% |
| Effective ST Debt | 74.4 | 74.4 | 74.4 | 74.4 | +0.0% |
| Non-Current Liabilities | 119.9 | 125.4 | 123.6 | 127.2 | +4.6% |
| **Equity** | **73.7** | **80.7** | **78.7** | **82.6** | +9.4% |
| **Income Statement** | | | | | |
| COGS | 209.3 | 209.5 | 209.5 | 209.5 | +0.1% |
| OpEx | 62.2 | 59.3 | 53.2 | 65.2 | −4.6% |
| Depreciation | 11.7 | 11.8 | 11.8 | 11.8 | +0.6% |
| Interest Payments | 0.0 | 2.4 | 2.4 | 2.4 | N/A |
| ST Investment Returns | −0.3 | 2.7 | 2.7 | 2.7 | +932.7% |
| Tax | 20.7 | 23.1 | 22.1 | 24.2 | +11.7% |
| **Net Income** | **112.0** | **112.8** | **107.9** | **117.8** | +0.7% |
| **Cash Flow** | | | | | |
| Dividends | 15.4 | 15.5 | 15.5 | 15.5 | +0.6% |
| Stock Buyback | 90.7 | 77.6 | 77.6 | 77.6 | −14.5% |

All figures in $ billions. Confidence intervals reflect Bayesian OpEx uncertainty only; deterministic line items show identical lower/upper bounds.

The Operation Expense learned with Bayesian Variational Inference is as follows:



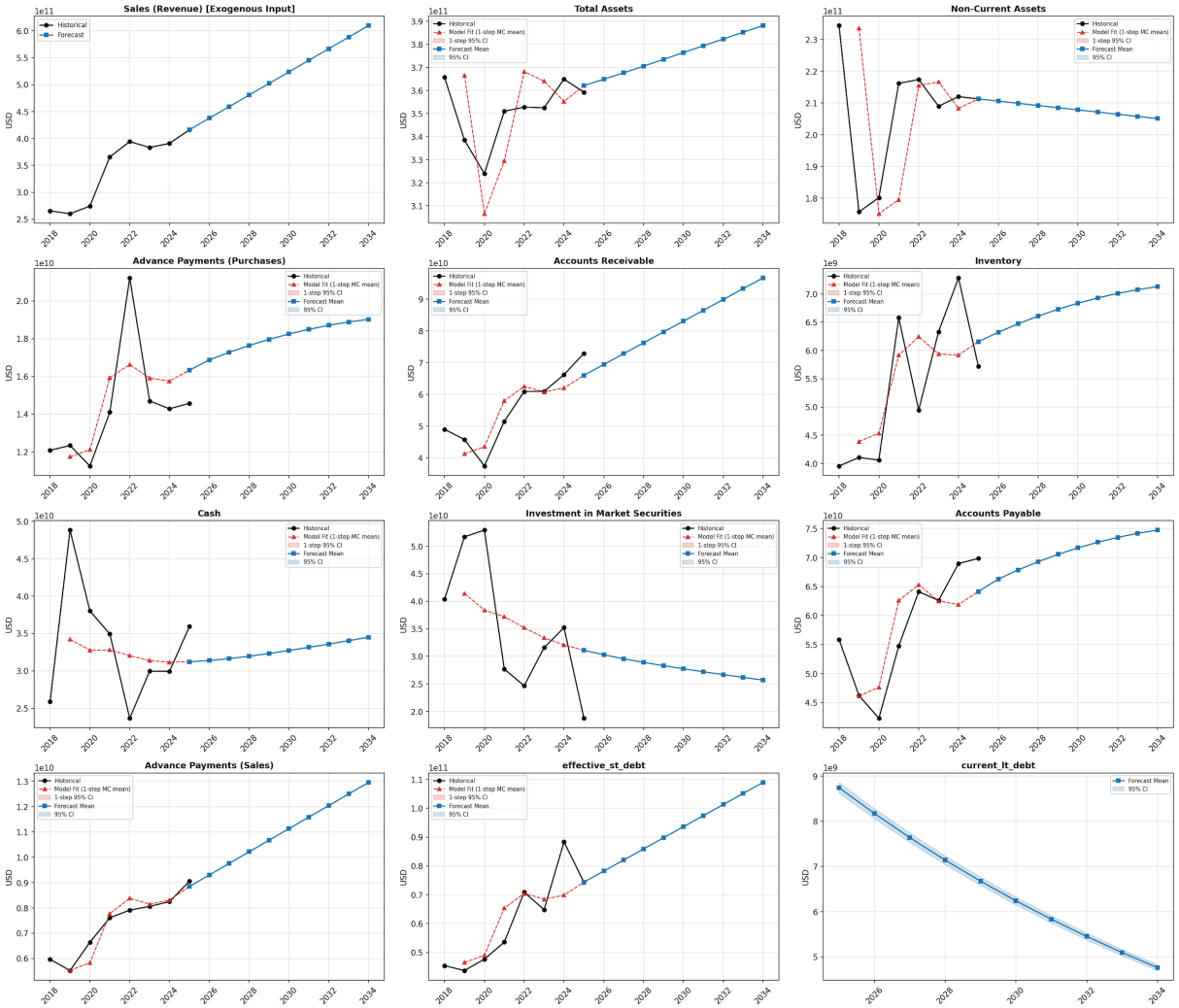

And the net income now has a confidence level due to the noise term, and we see that the uncertainty increases the further the forecast year as expected for Variational Inference

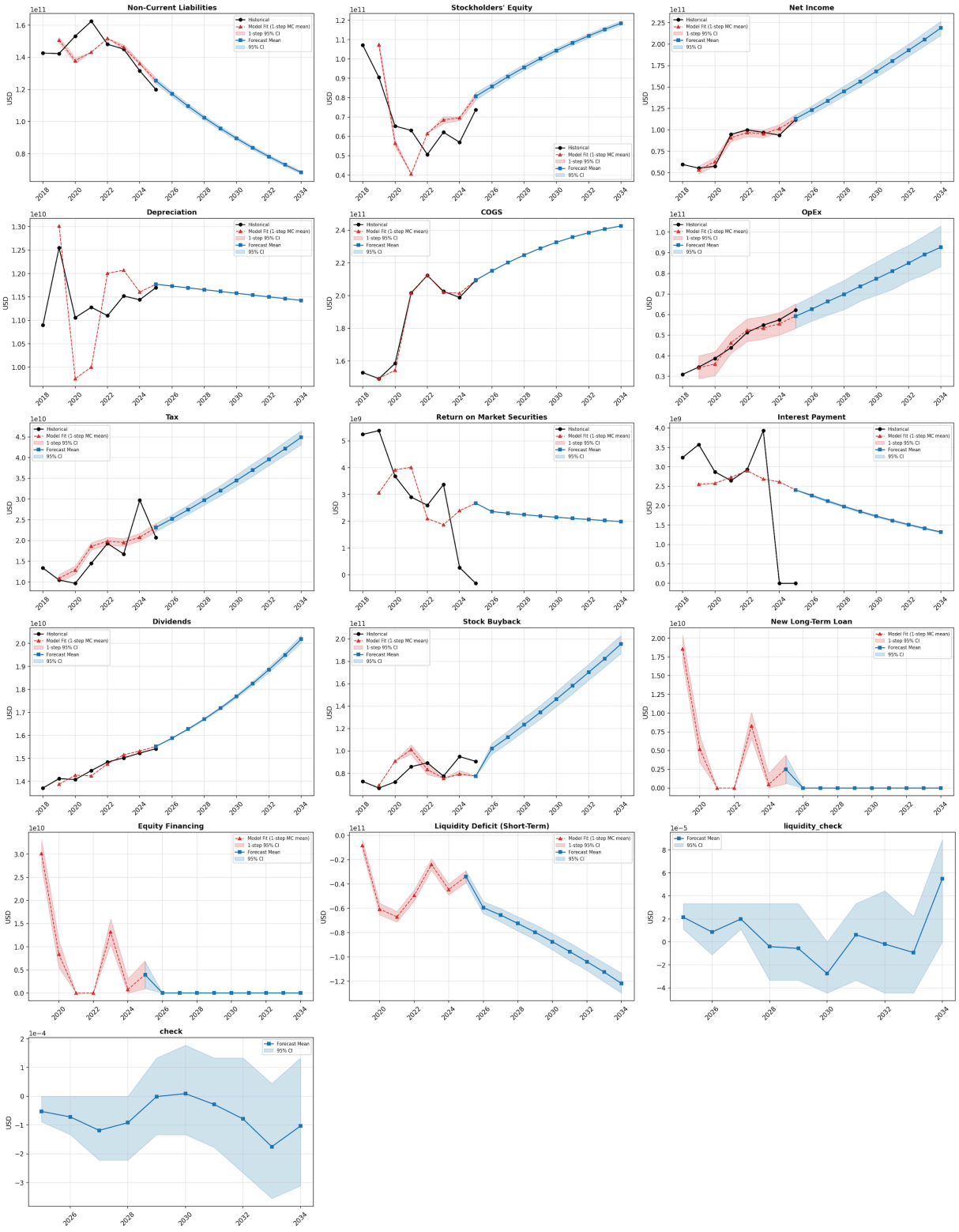
model's increasing epistemic uncertainty outside of the training range:

**Net Income**

| Year | Mean | 2.5% CI | 97.5% CI |
|---|---|---|---|
| 2025 | 1.13e+11 | 1.08e+11 | 1.18e+11 |
| 2026 | 1.23e+11 | 1.18e+11 | 1.28e+11 |
| 2027 | 1.34e+11 | 1.28e+11 | 1.39e+11 |
| 2028 | 1.45e+11 | 1.39e+11 | 1.51e+11 |
| 2029 | 1.56e+11 | 1.50e+11 | 1.62e+11 |
| 2030 | 1.68e+11 | 1.61e+11 | 1.75e+11 |
| 2031 | 1.80e+11 | 1.73e+11 | 1.88e+11 |
| 2032 | 1.93e+11 | 1.86e+11 | 2.00e+11 |
| 2033 | 2.05e+11 | 1.98e+11 | 2.13e+11 |
| 2034 | 2.19e+11 | 2.10e+11 | 2.27e+11 |

All figures in USD.






## 2.6 Forecast Results

The forecasted 10 years (with inflation = 3%, sales growing linearly at the historical rate of training data) of Balance Sheet, Income Statement, and Cash Budget is attached below:

FORECAST BALANCE SHEET — Mean across Monte Carlo samples (USD)

| Line Item | FY2025 | FY2026 | FY2027 | FY2028 | FY2029 | FY2030 | FY2031 | FY2032 | FY2033 | FY2034 |
|---|---|---|---|---|---|---|---|---|---|---|
| **ASSETS** | | | | | | | | | | |
| Non-Current Assets | $211,294,134,686 | $210,597,581,775 | $209,903,333,641 | $209,211,382,686 | $208,521,721,334 | $207,834,342,035 | $207,149,237,266 | $206,466,399,525 | $205,785,821,338 | $205,107,495,254 |
| Advance Payments (Purchases) | $16,374,224,975 | $16,890,984,352 | $17,292,784,103 | $17,655,445,055 | $17,979,604,046 | $18,265,980,509 | $18,515,372,659 | $18,728,653,189 | $18,906,764,530 | $19,050,713,718 |
| Accounts Receivable | $66,026,078,355 | $69,438,659,583 | $72,851,240,810 | $76,263,822,038 | $79,676,403,265 | $83,088,984,493 | $86,501,565,720 | $89,914,146,948 | $93,326,728,175 | $96,739,309,403 |
| Inventory | $6,670,655,799 | $7,015,431,005 | $7,360,206,211 | $7,704,981,417 | $8,049,756,622 | $8,394,531,828 | $8,739,307,034 | $9,084,082,240 | $9,428,857,446 | $9,773,632,652 |
| Cash | $31,223,669,354 | $31,387,036,118 | $31,642,257,505 | $31,963,551,974 | $32,331,912,553 | $32,733,440,586 | $33,158,028,588 | $33,598,350,511 | $34,049,103,448 | $34,506,446,402 |
| Investment in Market Securities | $31,123,361,807 | $30,289,228,865 | $29,562,470,016 | $28,911,034,982 | $28,312,312,782 | $27,750,509,142 | $27,214,687,305 | $26,697,337,455 | $26,193,348,602 | $25,699,278,206 |
| TOTAL ASSETS | $362,712,124,977 | $365,618,921,698 | $368,612,292,286 | $371,710,218,151 | $374,871,710,602 | $378,067,788,593 | $381,278,198,571 | $384,488,969,867 | $387,690,623,539 | $390,876,875,635 |
| **** | | | | | | | | | | |
| **LIABILITIES** | | | | | | | | | | |
| Accounts Payable | $64,316,968,943 | $66,346,768,633 | $67,925,013,842 | $69,349,524,208 | $70,622,801,190 | $71,747,670,678 | $72,727,268,012 | $73,565,021,081 | $74,264,631,697 | $74,830,055,432 |
| Advance Payments (Sales) | $8,846,925,987 | $9,304,182,488 | $9,761,438,989 | $10,218,695,489 | $10,675,951,990 | $11,133,208,491 | $11,590,464,992 | $12,047,721,492 | $12,504,977,993 | $12,962,234,494 |
| Effective ST Debt | $87,204,056,424 | $96,117,022,855 | $105,620,475,218 | $115,735,721,110 | $126,483,234,187 | $137,882,470,325 | $149,951,679,605 | $162,707,716,402 | $176,165,850,080 | $190,339,579,048 |
| Current LT Debt | $8,547,079,717 | $7,992,201,779 | $7,473,346,616 | $6,988,175,623 | $6,534,502,017 | $6,110,280,982 | $5,713,600,453 | $5,342,672,494 | $4,995,825,244 | $4,671,495,379 |
| Non-Current Liabilities | $123,108,130,835 | $115,115,929,056 | $107,642,582,440 | $100,654,406,817 | $94,119,904,800 | $88,009,623,818 | $82,296,023,365 | $76,953,350,871 | $71,957,525,628 | $67,286,030,248 |
| TOTAL LIABILITIES | $292,023,161,905 | $294,876,104,811 | $298,422,857,104 | $302,946,523,246 | $308,436,394,184 | $314,883,254,294 | $322,279,036,426 | $330,616,482,341 | $339,888,810,642 | $350,089,394,601 |
| **** | | | | | | | | | | |
| **EQUITY** | | | | | | | | | | |
| Equity | $70,688,963,072 | $70,742,816,887 | $70,189,435,182 | $68,763,694,904 | $66,435,316,418 | $63,184,534,300 | $58,999,162,146 | $53,872,487,527 | $47,801,812,898 | $40,787,481,034 |
| **** | | | | | | | | | | |
| TOTAL LIAB + EQUITY | $362,712,124,977 | $365,618,921,698 | $368,612,292,286 | $371,710,218,151 | $374,871,710,602 | $378,067,788,593 | $381,278,198,571 | $384,488,969,867 | $387,690,623,539 | $390,876,875,635 |
| CHECK: Assets-(L+E) | $-0 | $-0 | $-0 | $0 | $-0 | $-0 | $-0 | $0 | $-0 | $-0 |

FORECAST INCOME STATEMENT — Mean across Monte Carlo samples (USD)

| Line Item | FY2025 | FY2026 | FY2027 | FY2028 | FY2029 | FY2030 | FY2031 | FY2032 | FY2033 | FY2034 |
|---|---|---|---|---|---|---|---|---|---|---|
| Revenue (Sales_t) | $416,161,000,000 | $437,670,428,571 | $459,179,857,143 | $480,689,285,714 | $502,198,714,286 | $523,708,142,857 | $545,217,571,429 | $566,727,000,000 | $588,236,428,571 | $609,745,857,143 |
| COGS | $209,479,943,818 | $215,111,448,777 | $220,236,683,361 | $224,862,675,415 | $228,997,548,241 | $232,650,478,706 | $235,831,648,594 | $238,552,189,724 | $240,824,123,455 | $242,660,295,213 |
| OpEx | $59,189,357,476 | $62,743,853,543 | $66,268,928,527 | $69,970,096,920 | $73,867,187,499 | $77,530,912,397 | $81,309,330,879 | $85,303,190,906 | $89,343,258,569 | $93,310,113,709 |
| Depreciation & Amortization | $11,772,820,547 | $11,734,009,756 | $11,695,327,383 | $11,656,773,003 | $11,618,346,194 | $11,580,046,536 | $11,541,873,610 | $11,503,826,998 | $11,465,906,283 | $11,428,111,049 |
| EBIT = Revenue - COGS - OpEx - Depreciation | $135,718,878,159 | $148,081,116,495 | $160,978,917,872 | $174,199,740,376 | $187,715,632,351 | $201,946,705,217 | $216,534,718,346 | $231,367,792,372 | $246,603,140,266 | $262,347,337,172 |
| Interest Payments | $4,494,432,488 | $4,376,763,861 | $4,672,141,362 | $4,994,830,605 | $5,345,430,056 | $5,724,521,177 | $6,132,660,059 | $6,570,368,970 | $7,038,127,887 | $7,536,366,110 |
| ST Investment Returns | $247,941,789 | $219,052,515 | $213,181,719 | $208,066,643 | $203,481,711 | $199,267,783 | $195,313,696 | $191,542,473 | $187,901,260 | $184,354,085 |
| EBT = EBIT - Interest + ST Returns | $131,472,387,460 | $143,923,405,149 | $156,519,958,229 | $169,412,976,414 | $182,573,684,006 | $196,421,451,823 | $210,597,371,983 | $224,988,965,875 | $239,752,913,638 | $254,995,325,146 |
| Income Taxes = EBT * %EffTax (15.51%) | $20,396,751,105 | $22,328,413,819 | $24,282,655,032 | $26,282,890,123 | $28,324,654,803 | $30,473,010,660 | $32,672,276,382 | $34,905,001,932 | $37,195,494,816 | $39,560,217,019 |
| Net Income = EBT - Income Taxes | $111,075,636,355 | $121,594,991,330 | $132,237,303,196 | $143,130,086,291 | $154,249,029,203 | $165,948,441,164 | $177,925,095,601 | $190,083,963,943 | $202,557,418,823 | $215,435,108,127 |
| Dividends | $15,525,131,801 | $15,879,492,946 | $16,271,553,727 | $16,701,630,303 | $17,170,510,589 | $17,678,889,409 | $18,228,773,075 | $18,821,035,846 | $19,456,196,296 | $20,135,262,019 |

FORECAST CASH BUDGET — Mean across Monte Carlo samples (USD)

| Line Item | FY2025 | FY2026 | FY2027 | FY2028 | FY2029 | FY2030 | FY2031 | FY2032 | FY2033 | FY2034 |
|---|---|---|---|---|---|---|---|---|---|---|
| **MODULE 1: Operating Activities** | | | | | | | | | | |
| Inflows from Sales | $416,161,000,000 | $437,670,428,571 | $459,179,857,143 | $480,689,285,714 | $502,198,714,286 | $523,708,142,857 | $545,217,571,429 | $566,727,000,000 | $588,236,428,571 | $609,745,857,143 |
| Total Inflows (Operating) | $416,161,000,000 | $437,670,428,571 | $459,179,857,143 | $480,689,285,714 | $502,198,714,286 | $523,708,142,857 | $545,217,571,429 | $566,727,000,000 | $588,236,428,571 | $609,745,857,143 |
| Payments for Purchases (COGS) | $209,479,943,818 | $215,111,448,777 | $220,236,683,361 | $224,862,675,415 | $228,997,548,241 | $232,650,478,706 | $235,831,648,594 | $238,552,189,724 | $240,824,123,455 | $242,660,295,213 |
| Operational Expenses (OpEx) | $59,189,357,476 | $62,743,853,543 | $66,268,928,527 | $69,970,096,920 | $73,867,187,499 | $77,530,912,397 | $81,309,330,879 | $85,303,190,906 | $89,343,258,569 | $93,310,113,709 |
| Income Tax | $20,396,751,105 | $22,328,413,819 | $24,282,655,032 | $26,282,890,123 | $28,324,654,803 | $30,473,010,660 | $32,672,276,382 | $34,905,001,932 | $37,195,494,816 | $39,560,217,019 |
| Total Outflows (Operating) | $289,066,052,399 | $300,183,716,139 | $310,788,266,920 | $321,115,662,459 | $331,189,390,544 | $340,654,401,763 | $349,813,255,854 | $358,760,382,562 | $367,362,876,839 | $375,530,625,941 |
| Operating Net Cash Balance | $127,094,947,601 | $137,486,712,432 | $148,391,590,223 | $159,573,623,255 | $171,009,323,742 | $183,053,741,094 | $195,404,315,574 | $207,966,617,438 | $220,873,551,732 | $234,215,231,202 |
| **** | | | | | | | | | | |
| **MODULE 2: Investing Activities** | | | | | | | | | | |
| Total Inflows (Investing) | $0 | $0 | $0 | $0 | $0 | $0 | $0 | $0 | $0 | $0 |
| Investment in Fixed Assets (CapEx) | $11,073,955,233 | $11,037,456,845 | $11,001,079,249 | $10,964,822,047 | $10,928,684,842 | $10,892,667,238 | $10,856,768,841 | $10,820,989,257 | $10,785,328,096 | $10,749,784,966 |
| Total Outflows (Investing) | $11,073,955,233 | $11,037,456,845 | $11,001,079,249 | $10,964,822,047 | $10,928,684,842 | $10,892,667,238 | $10,856,768,841 | $10,820,989,257 | $10,785,328,096 | $10,749,784,966 |
| Investing Net Cash Balance | $-11,073,955,233 | $-11,037,456,845 | $-11,001,079,249 | $-10,964,822,047 | $-10,928,684,842 | $-10,892,667,238 | $-10,856,768,841 | $-10,820,989,257 | $-10,785,328,096 | $-10,749,784,966 |
| **** | | | | | | | | | | |
| **MODULE 3: External Financing** | | | | | | | | | | |
| ST Loan | $87,204,056,424 | $96,117,022,855 | $105,620,475,218 | $115,735,721,110 | $126,483,234,187 | $137,882,470,325 | $149,951,679,605 | $162,707,716,402 | $176,165,850,080 | $190,339,579,048 |
| LT Loan | $17,210,551 | $0 | $0 | $0 | $0 | $0 | $0 | $0 | $0 | $0 |
| Total Inflows (Financing) | $87,221,266,975 | $96,117,022,855 | $105,620,475,218 | $115,735,721,110 | $126,483,234,187 | $137,882,470,325 | $149,951,679,605 | $162,707,716,402 | $176,165,850,080 | $190,339,579,048 |
| ST Principal Payment | $88,271,000,000 | $87,204,056,424 | $96,117,022,855 | $105,620,475,218 | $115,735,721,110 | $126,483,234,187 | $137,882,470,325 | $149,951,679,605 | $162,707,716,402 | $176,165,850,080 |
| ST Interest | $3,509,926,558 | $3,467,501,599 | $3,821,908,568 | $4,199,795,075 | $4,602,008,375 | $5,029,362,563 | $5,482,631,266 | $5,962,540,163 | $6,469,759,434 | $7,004,896,238 |
| LT Principal Payment | $10,912,000,000 | $8,547,079,717 | $7,992,201,779 | $7,473,346,616 | $6,988,175,623 | $6,534,502,017 | $6,110,280,982 | $5,713,600,453 | $5,342,672,494 | $4,995,825,244 |
| LT Interest | $984,505,930 | $909,262,262 | $850,232,794 | $795,035,530 | $743,421,682 | $695,158,614 | $650,028,793 | $607,828,808 | $568,368,453 | $531,469,872 |
| Total Outflows (Financing) | $103,677,432,488 | $100,127,900,001 | $108,781,365,996 | $118,088,652,439 | $128,069,326,789 | $138,742,257,381 | $150,125,411,366 | $162,235,649,028 | $175,088,516,783 | $188,698,041,434 |
| Financing Net Cash Balance | $-16,456,165,513 | $-4,010,877,147 | $-3,160,890,779 | $-2,352,931,329 | $-1,586,092,602 | $-859,787,057 | $-173,731,761 | $472,067,375 | $1,077,333,297 | $1,641,537,614 |
| **** | | | | | | | | | | |
| **MODULE 4: Transactions with Owners** | | | | | | | | | | |
| Invested Equity (Equity Financing) | $17,790,069 | $0 | $0 | $0 | $0 | $0 | $0 | $0 | $0 | $0 |
| Total Inflows (Owners) | $17,790,069 | $0 | $0 | $0 | $0 | $0 | $0 | $0 | $0 | $0 |
| Dividends from Last Year | $15,525,131,801 | $15,879,492,946 | $16,271,553,727 | $16,701,630,303 | $17,170,510,589 | $17,678,889,409 | $18,228,773,075 | $18,821,035,846 | $19,456,196,296 | $20,135,262,019 |
| Stock Buyback | $81,829,331,551 | $105,661,644,567 | $116,519,131,174 | $127,854,196,266 | $139,406,897,100 | $151,520,333,873 | $163,881,694,680 | $176,389,602,716 | $189,171,897,156 | $202,314,177,972 |
| Total Outflows (Owners) | $97,354,463,352 | $121,541,137,514 | $132,790,684,901 | $144,555,826,569 | $156,577,407,689 | $169,199,223,282 | $182,110,467,755 | $195,210,638,562 | $208,628,093,452 | $222,449,439,991 |
| Owners' Transaction Net Cash Balance | $-97,336,673,283 | $-121,541,137,514 | $-132,790,684,901 | $-144,555,826,569 | $-156,577,407,689 | $-169,199,223,282 | $-182,110,467,755 | $-195,210,638,562 | $-208,628,093,452 | $-222,449,439,991 |
| **** | | | | | | | | | | |
| **MODULE 5: Discretionary Transactions** | | | | | | | | | | |
| Return from ST Investments | $247,941,789 | $219,052,515 | $213,181,719 | $208,066,643 | $203,481,711 | $199,267,783 | $195,313,696 | $191,542,473 | $187,901,260 | $184,354,085 |
| Total Inflows (Discretionary) | $247,941,789 | $219,052,515 | $213,181,719 | $208,066,643 | $203,481,711 | $199,267,783 | $195,313,696 | $191,542,473 | $187,901,260 | $184,354,085 |
| Total Outflows (Discretionary) | $0 | $0 | $0 | $0 | $0 | $0 | $0 | $0 | $0 | $0 |
| Discretionary Transaction Net Cash Balance | $247,941,789 | $219,052,515 | $213,181,719 | $208,066,643 | $203,481,711 | $199,267,783 | $195,313,696 | $191,542,473 | $187,901,260 | $184,354,085 |

# 3 Part 2

## 3.1 a) LLM Selection and Infrastructure Setup

My favorite LLM is DeepSeek V3.2 for reasoning which has 128k context window and gpt-5-nano for non-reasoning task which has 400k context window. From my daily usage, DeepSeekV3.2 follows instructions well with less hallucinations than other models from competitors. Gpt-5-nano, from my experience, is an inexpensive model that performs well for pretty complicated tasks but hallucinates more. I use Microsoft's Azure Foundry to access these models via Azure API. We have set up an AWS Lambda function that makes the API calls so that we can hide the API keys for our Azure API. The AWS Lambda address is saved as an environment variable in a .env file, with its content being:

AZURE_DEEPSEEK_ENDPOINT=https://dru54knwxa7sb4rlwrgws3kaze0jdvtj.lambda-url.us-west-1.on.aws/

We have included the .env file in the zipped file attached with our report submission.

## 3.2 b) LLM-Only Balance Sheet Forecasting

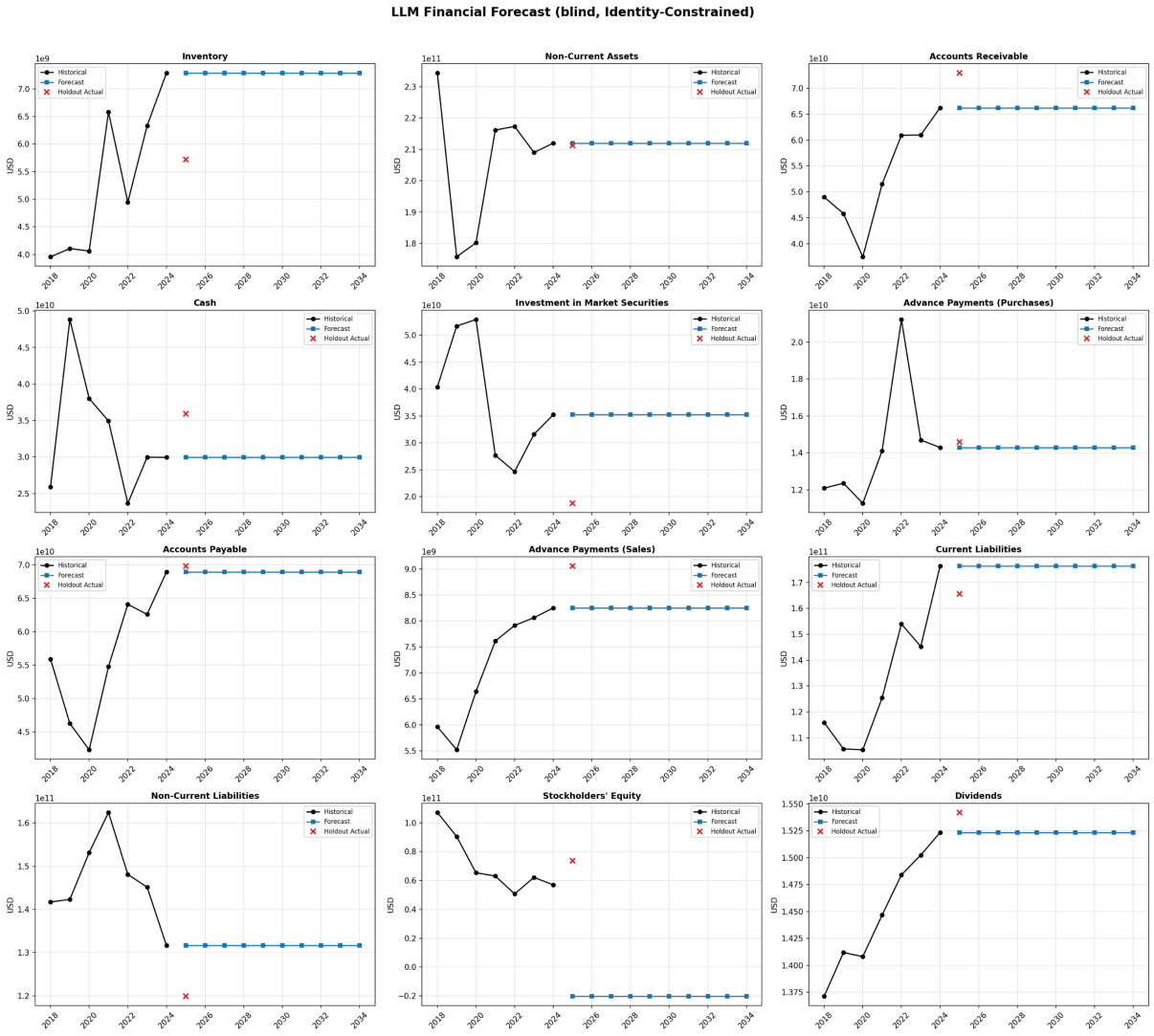
We can use the same input data from part 1 and feed it to DeepSeekV3.2 to forecast the balance sheet, as implemented in `financial_forecast/models/llm_forecaster.py`.

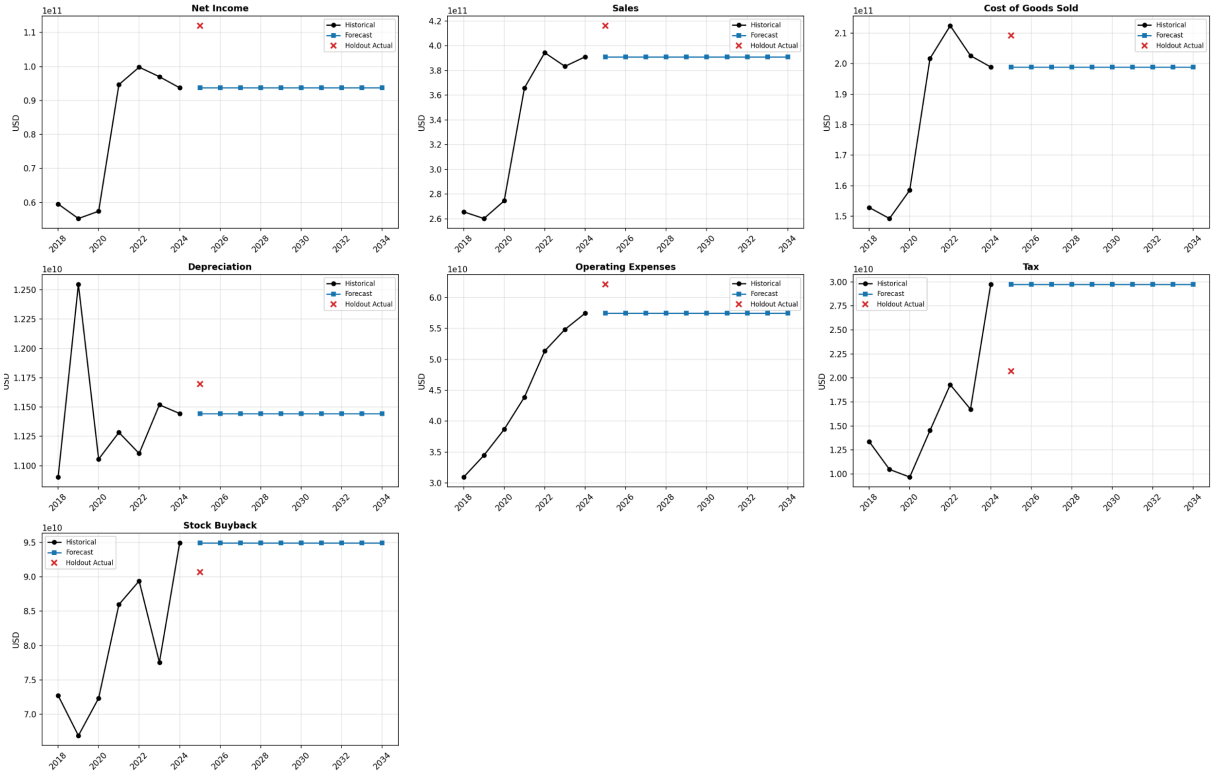
> We use the following prompt with the identity check that assets = liabilities + equity:

```
prompt: Dict[str, Any] = {
    "task": "Forecast all required financial elements for future years. "
            "Return ONLY valid JSON (no markdown).",
    "historical_facts": historical_rows,
    "value_units": "trillions_of_usd",
    "value_scale_to_usd": LLM_AMOUNT_SCALE,
    "required_elements": ELEMENT_KEYS,
    "accounting_identity": (
        "inventory + nca + accounts_receivable + cash + "
        "investment_in_market_securities + advance_payments_purchases = "
        "accounts_payable + advance_payments_sales + current_liabilities + "
        "non_current_liabilities + equity"
    ),
    "output_schema": {
        "forecast": [
            {
                "inventory": "float",
                "nca": "float",
                "accounts_receivable": "float",
                "cash": "float",
                "investment_in_market_securities": "float",
                "advance_payments_purchases": "float",
                "accounts_payable": "float",
                "advance_payments_sales": "float",
                "current_liabilities": "float",
                "non_current_liabilities": "float",
                "equity": "float",
                "dividends": "float",
                "net_income": "float",
```

```
                "sales": "float",
                "cogs": "float",
                "depreciation": "float",
                "opex": "float",
                "tax": "float",
                "stock_buyback": "float",
            }
        ]
    },
}
```

The returned fields are further processed within `financial_forecast/models/llm_forecaster.py` to ensure the balance sheet identity holds. The forecasted values are plotted below:



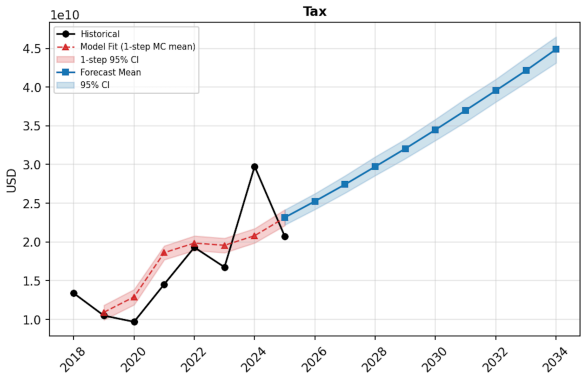


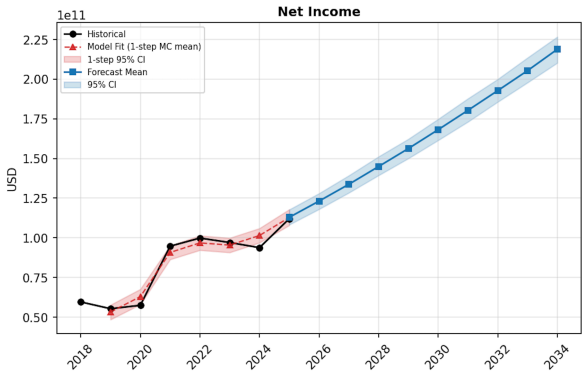

We see in "Stockholders' Equity" graph that it drops to negative. This is because the LLM actually failed to return the projected balance sheet fields that abide by the accounting identity of assets = liabilities + equity, so the script's procedure that ensures the identity results in negative equity value. It is clear that LLM-only forecasting of balance sheet fields is severely inadequate.

### 3.3 c) Combining LLM with the Balance Sheet Forecasting Model (Tax Anomalies)

Is it possible to combine LLM with the balance sheet forecasting model? Yes. When we look at the forecasted model, we see that one of the major contributors for predicted income discrepancy from data is the calculated tax (income underestimated by ~5 billion, and tax overestimated by ~2 billion). Net income (~100 billion dollars) is sales – cogs – opex – depreciation – debt interest + market securities return – tax, and we actually see good predicted values for COGS, opex, and depreciation (market securities return is derived as EBT − EBIT + InterestExpense per (55), and their combined amount is an order of magnitude smaller than tax) all show discrepancy from data of less than 1 billion, while tax, which takes ~30% of net income, is off from the data by ~2 billion, almost 10% higher than predicted which lowers the predicted net income.

In the case of Apple, there was a spike in effective tax rate in 2024 due to a one-time income tax charge resulting from the final resolution of the European Commission State Aid Decision. This skewed the learned effective tax rate upwards during training, as evidenced by the consistent over-estimation of tax for one-step forecast model fit.




To avoid this issue, we can ask LLM to notify us of any one-time tax payments like this so that we can appropriately handle it during training. When calculating the MSE between predicted and actual tax owed, the ground truth data can be offsetted by the one-time amount to better learn the baseline effective tax rate.

We can feed the 10-K to LLM and ask it to identify pages that contain the one-off tax information, and extract this data in JSON format for model consumption:

Script: `run_tax_anomaly_extraction.py` (using `TaxAnomalyExtractor` from `financial_forecast/extraction/`)

```
DEFAULT_SELECTION_PROMPT = (
    "You are given pages from a Form 10-K annual report.\n"
    "Identify all pages that contain any of the following:\n"
    "1) Item 7: Management's Discussion and Analysis (MD&A)\n"
    "2) Item 8: Financial Statements and Supplementary Data notes "
    "for:\n"
    " - Income Taxes\n"
    "Include pages with headings, substantive discussion, tables, "
    "and continuations of "
    "these sections.\n"
    'Return ONLY valid JSON with this exact shape: '
    '{{"pages": [<page_number>, ...]}}.\n'
    'If none match, return {{"pages": []}}.\n\n'
    "Pages:\n{pages}"
)
```

```
DEFAULT_EXTRACTION_PROMPT = (
    "You are an expert financial analyst. Your task is to analyze the
    provided "
    "excerpts from a company's Form 10-K (MD&A and Financial Footnotes) and
     "
    "extract data regarding one-time tax anomalies.\n\n"
    "Carefully evaluate the text for the following:\n\n"
    "Current Year Anomalies: Identify any massive, non-recurring, discrete
    tax "
    "charges or benefits ASSESSED THIS YEAR (not last year or next year)
    that skewed the current year's net income (e.g., "
    "finalized state aid decisions, sudden impacts from new tax legislation
    ). Positive for tax paid by company, negative for tax reduction or
    benefit for the company.\n\n"
    "You must respond ONLY with a valid JSON object using the exact schema
    below. "
    "Do not include any markdown formatting, preamble, or postscript. If a
    specific "
```

```
    "data point is not explicitly mentioned or cannot be reliably
    quantified from "
    "the text, output null for that field.\n\n"
    "JSON Schema:\n"
    "{{\n"
    '  "current_tax_year": <number>,\n'
    '  "tax_onetime_amount": <number in billions or null>,\n'
    '  "tax_onetime_note": "<string explaining the anomaly or null>",\n'
    "}}\n\n"
    "Pages:\n{pages}"
)
```

LLM was able to extract the one-time tax payment from this portion of 10-K:

2024 10-K page 24:

**Provision for Income Taxes**

Provision for income taxes, effective tax rate and statutory federal income tax rate for 2024, 2023 and 2022 were as follows (dollars in millions):

| | 2024 | 2023 | 2022 |
|---|---|---|---|
| Provision for income taxes | $ 29,749 | $ 16,741 | $ 19,300 |
| Effective tax rate | 24.1% | 14.7% | 16.2% |
| Statutory federal income tax rate | 21% | 21% | 21% |

The Company's effective tax rate for 2024 was higher than the statutory federal income tax rate due primarily to a one-time income tax charge of $10.2 billion, net, related to the State Aid Decision (refer to Note 7, "Income Taxes" in the Notes to Consolidated Financial Statements in Part II, Item 8 of this Form 10-K) and state income taxes, partially offset by a lower effective tax rate on foreign earnings, the impact of the U.S. federal R&D credit, and tax benefits from share-based compensation.

The Company's effective tax rate for 2024 was higher compared to 2023 due primarily to a one-time income tax charge of $10.2 billion, net, related to the State Aid Decision, a higher effective tax rate on foreign earnings and lower tax benefits from share-based compensation.

And page 39:

**Note 7 – Income Taxes**

**European Commission State Aid Decision**

On August 30, 2016, the Commission announced its decision that Ireland granted state aid to the Company by providing tax opinions in 1991 and 2007 concerning the tax allocation of profits of the Irish branches of two subsidiaries of the Company (the "State Aid Decision"). The State Aid Decision ordered Ireland to calculate and recover additional taxes from the Company for the period June 2003 through December 2014. Irish legislative changes, effective as of January 2015, eliminated the application of the tax opinions from that date forward. The recovery amount was calculated to be €13.1 billion, plus interest of €1.2 billion.

From time to time, the Company requested approval from the Irish Minister for Finance to reduce the recovery amount for certain taxes paid to other countries. As of September 28, 2024, the adjusted recovery amount of €12.7 billion plus interest of €1.2 billion was held in escrow and restricted from general use. The total balance of the escrow, including net unrealized investment gains, was €14.2 billion or $15.8 billion as of September 28, 2024, of which $2.6 billion was classified as cash and cash equivalents and $13.2 billion was classified as current marketable securities in the Consolidated Balance Sheet. Refer to the Cash, Cash Equivalents and Marketable Securities section of Note 4, "Financial Instruments" for more information.



The Company and Ireland appealed the State Aid Decision to the General Court of the Court of Justice of the European Union (the "General Court"). On July 15, 2020, the General Court annulled the State Aid Decision. On September 25, 2020, the Commission appealed the General Court's decision to the European Court of Justice (the "ECJ") and a hearing was held on May 23, 2023. On September 10, 2024, the ECJ announced that it had set aside the 2020 judgment of the General Court and confirmed the Commission's 2016 State Aid Decision. As a result, during the fourth quarter of 2024 the Company recorded a one-time income tax charge of $10.2 billion, net, which represents $15.8 billion payable to Ireland via release of the escrow, partially offset by a U.S. foreign tax credit of $4.8 billion and a decrease in unrecognized tax benefits of $823 million.

JSON output:

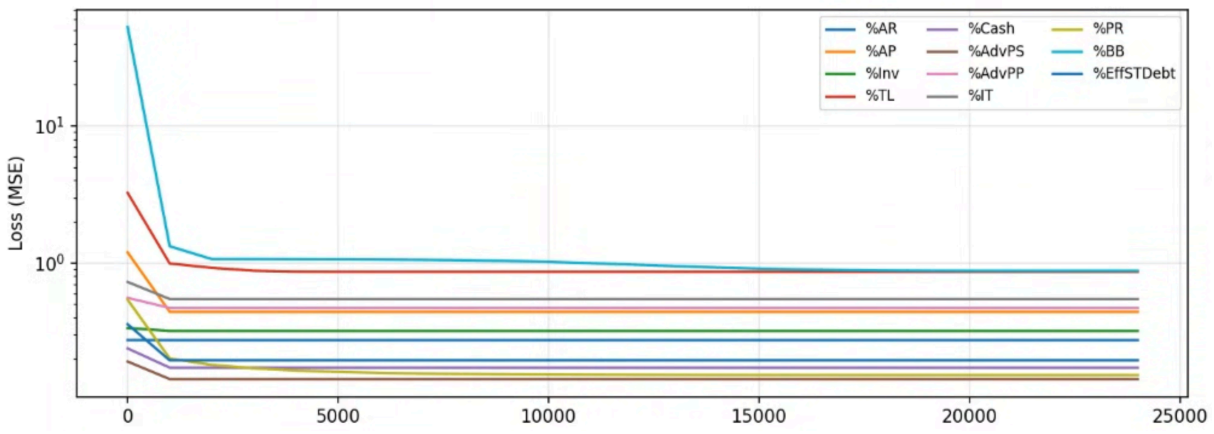
```
{
    "selected_pages": [
        23, 24, 25, 26, 27, 41, 42, 43, 45, 46, 47
    ],
    "extraction": {
        "tax_onetime_amount": 10.2,
        "tax_onetime_note": "One-time income tax charge of $10.2 billion
   net related to the European Commission State Aid Decision."
    }
}
```

We update our balance sheet forecasting script to include the tax_onetime_amount for better learning of the baseline tax rate. We run `run_trainable_model_forecast_adv_policies_w_bayesianopex_taxanomaly.py`, which injects a `TaxWithAnomalies(data.tax_onetime_payments)` module into the model instead of `SimpleTax()`.

Effective tax rate drops from 17% to 15.5%! And the learning loss is smaller for the effective tax rate %IT:

```
--------------------------------------------------
Simple parameters:
Final %AG: 0.00000
Final %AM: 0.94062
Final %Depr: 0.05553
Final %AdvPS: 0.02126
Final %AdvPP: 0.07840
Final %AR: 0.15866
Final %AP: 0.30794
Final %Inv: 0.01603
Total Liquidity (baseline + logit-linear): baseline=0.6013, alpha=-2.3463, beta=-0.411579
  => %TL at t=0: 0.0874, %TL at t=7: 0.0053
Cash % of Liquidity (logit-linear): alpha=-0.2235, beta=0.032385
  => %Cash at t=0: 0.4444, %Cash at t=7: 0.5008
Final %IT: 0.17021
Final %PR: 0.99969
Final DivAdjSpeed (α): 0.00371
Stock Buyback (baseline + ratio*depr): baseline=1.5540, ratio=-6.610795
Effective ST Debt % of Sales (logit-linear): alpha=-1.7458, beta=0.059738
  => %EffSTDebt at t=0: 0.1486, %EffSTDebt at t=7: 0.2095
Cost Ratio (logit-linear): alpha=0.3459, beta=-0.0475
  => CR at t=0: 0.5856, CR at t=7: 0.5034
Bayesian OpEx Variable %: Mean=0.0923, Std=0.0155
Bayesian OpEx Baseline (USD):   Mean=3.91e+10, Std=8.09e+08
OpEx aleatoric uncertainty (USD): 2.49e+09
--------------------------------------------------
Structural parameters:
Final %AvgSTInt: 0.03970
Final %AvgLTInt: 0.00692
Final AvgM: 15.36485
Final %MSReturn: 0.01119
Equity Financing % (logit-linear): alpha=0.5389, beta=0.018432
  => %EF at t=0: 0.6316, %EF at t=7: 0.6610
--------------------------------------------------
```

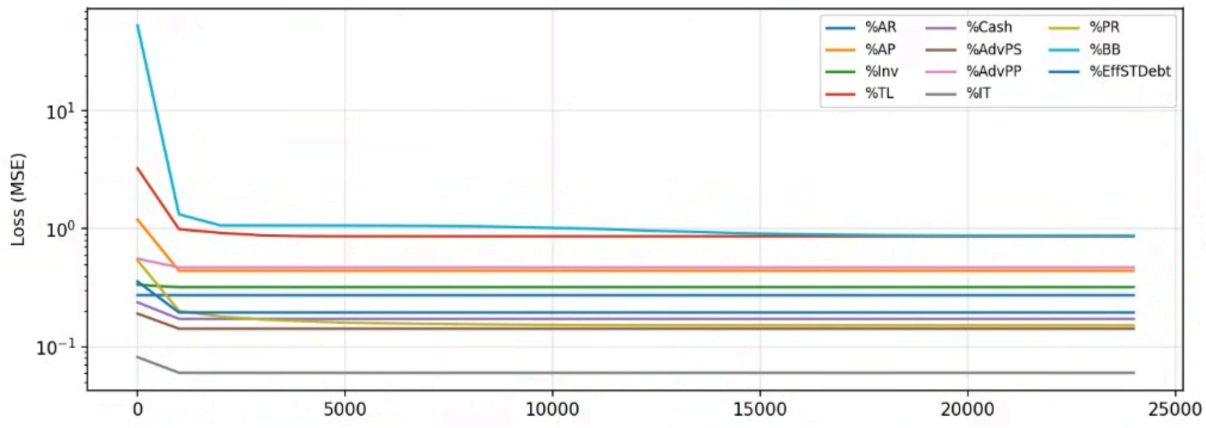
**Before**

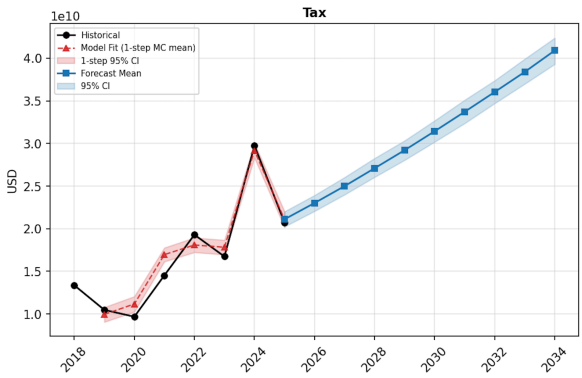


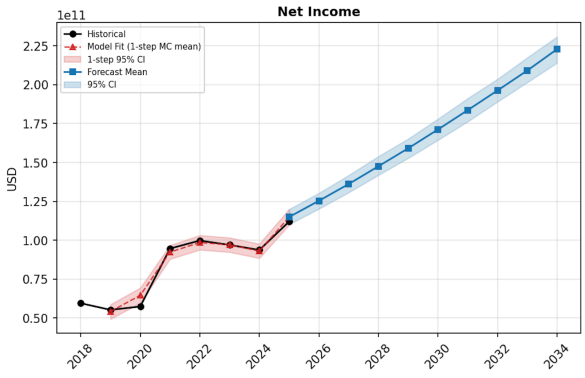

Before accounting for tax anomalies, the training losses are: tax loss = 5.45e−01, net income loss = 3.37e−01.

```
--------------------------------------------------
Simple parameters:
Final %AG: 0.00000
Final %AM: 0.94062
Final %Depr: 0.05553
Final %AdvPS: 0.02126
Final %AdvPP: 0.07840
Final %AR: 0.15866
Final %AP: 0.30794
Final %Inv: 0.01603
Total Liquidity (baseline + logit-linear): baseline=0.6013, alpha=-2.3463, beta=-0.411579
  => %TL at t=0: 0.0874, %TL at t=7: 0.0053
Cash % of Liquidity (logit-linear): alpha=-0.2235, beta=0.032385
  => %Cash at t=0: 0.4444, %Cash at t=7: 0.5008
Final %IT: 0.15514
Final %PR: 0.99969
Final DivAdjSpeed (α): 0.00371
Stock Buyback (baseline + ratio*depr): baseline=1.5540, ratio=-6.610795
Effective ST Debt % of Sales (logit-linear): alpha=-1.7458, beta=0.059738
  => %EffSTDebt at t=0: 0.1486, %EffSTDebt at t=7: 0.2095
Cost Ratio (logit-linear): alpha=0.3459, beta=-0.0475
  => CR at t=0: 0.5856, CR at t=7: 0.5034
Bayesian OpEx Variable %: Mean=0.0957, Std=0.0160
Bayesian OpEx Baseline (USD):   Mean=3.90e+10, Std=8.05e+08
OpEx aleatoric uncertainty (USD): 2.45e+09
--------------------------------------------------
Structural parameters:
Final %AvgSTInt: 0.03976
Final %AvgLTInt: 0.00691
Final AvgM: 15.40353
Final %MSReturn: 0.00704
Equity Financing % (logit-linear): alpha=0.6014, beta=-0.081179
  => %EF at t=0: 0.6460, %EF at t=7: 0.5083
--------------------------------------------------
```

**After**






After accounting for tax anomalies, the training losses drop to: tax loss = 5.99e−02, net income loss = 1.69e−01. The tax loss falls by nearly an order of magnitude (from 0.545 to 0.060), because the model no longer needs to distort its baseline tax rate to accommodate one-time charges. The net income loss roughly halves (from 0.337 to 0.169), since the cleaner tax estimates flow directly into the income statement.

### 3.4 d) Strategic Recommendations for CEO/CFO

For recommendation to CEO/CFO given our result, we feed the following prompt, and attach the historical and 10-year forecast of balance sheet and income statement:

```
CEO_ADVISOR_PROMPT = (
    "You are an elite Corporate Finance Advisor and Strategic Capital "
    "Expert at a top-tier investment bank. You specialize in advising "
    "the CEOs and CFOs of mature, cash-generative mega-cap technology "
    "firms on capital structure optimization, liquidity management, "
    "and capital allocation.\n\n"

    "Context & Task:\n"
    "I will provide you with two sets of financial data for a target "
    "company:\n"
    "  Historical Baseline: Actual financial performance from recent "
    "years.\n"
    "  ML Forecast: A multi-year financial forecast (Balance Sheet, "
    "Income Statement, and Cash Budget) generated by an advanced "
    "Monte Carlo machine learning ensemble.\n"
    "Your task is to analyze the historical data against the forecast "
    "to identify the three most critical financial leverage points "
    "(risks or opportunities) over the forecasted period.\n\n"

    "STRICT GUARDRAILS (CRITICAL):\n"
    "  NO OPERATIONAL ADVICE: Do not recommend operational, marketing,"
    " or business strategy changes.\n"
    "  STRICTLY FINANCIAL DOMAIN: Recommendations must be confined to "
    "Capital Structure (debt/equity mix, WACC), Liquidity & Treasury "
    "(working capital, commercial paper), and Capital Allocation "
    "(dividends, share repurchases, CapEx).\n"
    "  MANDATORY EVIDENCE: You cannot make a claim without citing "
    "specific numbers from the provided tables.\n\n"

    "Analytical Heuristics (The Mega-Cap Tech Lens):\n"
    "  Historical Benchmarking: Always contextualize the forecast by "
    "comparing it to the historical baseline. Identify structural "
    "breaks or trend accelerations.\n"
    "  Strategic Leverage vs. Distress: Do not penalize shrinking "
```

```
    "Equity from aggressive buybacks -- treat as WACC optimization, "
    "not distress, if FCF remains strong.\n"
    "  Net Debt Focus: Evaluate Cash + Marketable Securities against "
    "Liabilities; assess if debt is easily serviced.\n"
    "  Operating Leverage: Note if Revenue/NI grows faster or slower "
    "than historical OpEx.\n\n"

    "Output Format:\n"
    "Provide exactly 3 strategic priorities. For each:\n"
    "  Observation: [Quantitative trend, forecast vs. historical]\n"
    "  Implication: [Financial risk or WACC opportunity]\n"
    "  Recommendation: [Precise corporate finance action]\n\n"

    "Data Input:\n"
    "{historical_tables}\n\n"
    "{forecast_tables}"
)
```

The prompt, together with the historical and forecast markdown tables from the JSON report artifact exported by `ForecastPipeline`, is sent to our reasoning model via `run_recommendation_to_ceo.py`. This script loads the self-contained `forecast_report.json`, constructs the prompt using the `DeepseekCEOAdvisor` class (which extends the `Advisor` ABC from `financial_forecast/reporting/advisor.py`), and calls the Azure DeepSeek endpoint with greedy decoding (`temperature=0`, `top_k=1`) for minimal hallucination

And here is the recommendation that our reasoning model generated:

**LLM-Generated Recommendation: Analysis of Financial Leverage Points**

**Priority 1: Capital Return Sustainability**
*Observation.* The forecast projects a significant and accelerating decline in Stockholders' Equity, from $70.7B in FY2025 to $40.8B in FY2034, while Net Income grows from $111.1B to $215.4B over the same period. This is a structural break from the historical baseline (2017–2023), where Equity averaged $78.5B and showed no consistent downward trend, even as Net Income grew.
*Implication.* This trajectory indicates an extremely aggressive capital return policy is embedded in the forecast, far exceeding historical levels. While this optimizes WACC by shrinking the equity base, it creates a critical dependency on continuous, high levels of free cash flow. The balance sheet is being deliberately levered, shifting from an equity-heavy to a more debt-dependent structure.
*Recommendation.* The CFO must formally establish and communicate a **Sustainable Capital Return Framework**. This framework should define the maximum allowable annual capital return (dividends + buybacks) as a percentage of forward-looking Free Cash Flow (e.g., 80–90%), ensuring it does not jeopardize the credit rating or the ability to service the forecasted rising debt load. This moves the policy from implicit to explicit, managing investor expectations and credit agency concerns.

---

**Priority 2: Refinancing and Liquidity Risk**
*Observation.* The forecast shows a dramatic increase in **Effective Short-Term**

**Debt**, which balloons from $87.2B in FY2025 to $190.3B by FY2034. Concurrently, highly liquid assets (Cash + Marketable Securities) are projected to decline from a combined $62.3B to $60.2B. This creates a rising **Net Debt** position. Historically (2020–2023), the company maintained a robust liquidity buffer, with Cash + Securities averaging $55.8B against much lower debt levels.
*Implication.* The company is forecasted to become significantly more reliant on short-term wholesale funding. This introduces substantial refinancing and interest rate risk, especially as interest payments are projected to rise from $4.5B to $7.5B. The shrinking liquidity coverage ratio could pressure credit spreads and commercial paper rates during market stress.
*Recommendation.* Execute a **Strategic Liability Management Exercise**. The Treasury must term out a significant portion of the forecasted short-term debt into longer-dated fixed-rate bonds. This locks in historically low(er) long-term rates for a mega-cap tech issuer, reduces refinancing risk, and better matches the duration of the company's long-lived assets. Concurrently, establish a minimum liquidity covenant (e.g., Cash + Securities $\geq$ 1.0x ST Debt) for internal risk management.

---

**Priority 3: Operating Leverage and Capital Allocation**
*Observation.* The forecast predicts a powerful **positive operating leverage** trend. From FY2025 to FY2034, Revenue grows by 46.5% ($416B to $610B), while the combined growth of COGS and OpEx is only 39.5%. This divergence accelerates EBIT margin expansion. This is a continuation but acceleration of a historical trend; from 2017–2023, Revenue grew 47.2% while COGS+OpEx grew 45.8%.
*Implication.* This trajectory will generate super-linear growth in Free Cash Flow, the fundamental driver for all capital allocation decisions (debt service, buybacks, dividends). It provides immense balance sheet flexibility and debt capacity. However, it also raises the strategic opportunity cost of holding excess, low-yielding cash on the balance sheet.
*Recommendation.* Initiate a **Dynamic Share Repurchase Program** tied to Free Cash Flow Surplus. Instead of a fixed buyback amount, the program should automatically allocate a percentage of quarterly FCF in excess of a pre-determined threshold (e.g., FCF above levels needed for debt service and a baseline dividend) to repurchases. This mechanism directly links the benefit of operating leverage to per-share earnings accretion and WACC optimization, while maintaining a cash safety net.

### 3.5 e, f, g) Financial Ratios and Automated Extraction Pipeline

We need to calculate cost-to-income, quick ratio, debt-to-equity ratio, debt-to-assets ratio, debt-to-capital ratio, debt-to-EBITDA ratio, and interest coverage ratio:

1. Cost-to-Income ratio = Operating Expense / Revenue

   = Describes company's operating efficiency. Operating Expense includes cost of revenue.

   - LLM to extract **Revenue** and **Operating Expense** from **Income Statement**. Operating Expense may not be a simple line item, may need to combine a few lines, e.g., GM has line for "Automotive and other

cost of sales", "GM Financial interest, operating and other expenses" and "Automotive and other selling, general and administrative expense"

2. Quick Ratio = (Cash & Cash Equivalents + Short-Term Market Securities + Accounts Receivable) / Current Liabilities

   = Ability to meet short-term liabilities with the most liquid assets

   - LLM to extract **Cash & Cash Equivalents**, **Short-Term Market Securities**, **Accounts Receivable**, and **Current Liabilities** from **Balance Sheet**

3. Debt-to-Equity ratio = (Short-Term Debt + Long-Term Debt) / Total Equity

   - LLM to extract **Total Debt** (including short-term and long-term debts) and **Total Equity** from **Balance Sheet**

4. Debt-to-Assets ratio = (Short-Term Debt + Long-Term Debt) / Total Assets

   - Total Debt already extracted. LLM to extract **Total Assets** from **Balance Sheet**

5. Debt-to-Capital ratio = (Short-Term Debt + Long-Term Debt) / (Short-Term Debt + Long-Term Debt + Total Equity)

   - Total Debt and Total Equity already extracted.

6. Debt-to-EBITDA ratio = (Short-Term Debt + Long-Term Debt) / EBITDA

   = How many years it would take for a company to pay back its debt using its EBITDA

   - Total Debt already extracted. LLM to extract **Net Income**, **Taxes**, and **Interest Expenses** from **Income Statement**; and **Depreciation and Amortization** from **Cash Flows** to calculate EBITDA = Net Income + Interest Expenses + Taxes + Depreciation + Amortization

7. Interest Coverage ratio = EBIT / Interest Expense

   = How easily a company can pay interest on its outstanding debt

   - Interest Expense already extracted. Calculate EBIT = Net Income + Interest Expenses + Taxes

### 3.5.1 List of Fields to Be Extracted

**Balance Sheet**

- Cash & Cash Equivalents
- Short-Term Market Securities
- Accounts Receivables
- Total Current Liabilities
- Total Debt (Short-Term and Long-Term)
- Total Equity

- Total Assets

**Income Statement**

- Revenue
- Operating Expenses
- Net Income
- Taxes
- Interest Expenses

**Cash Flows**

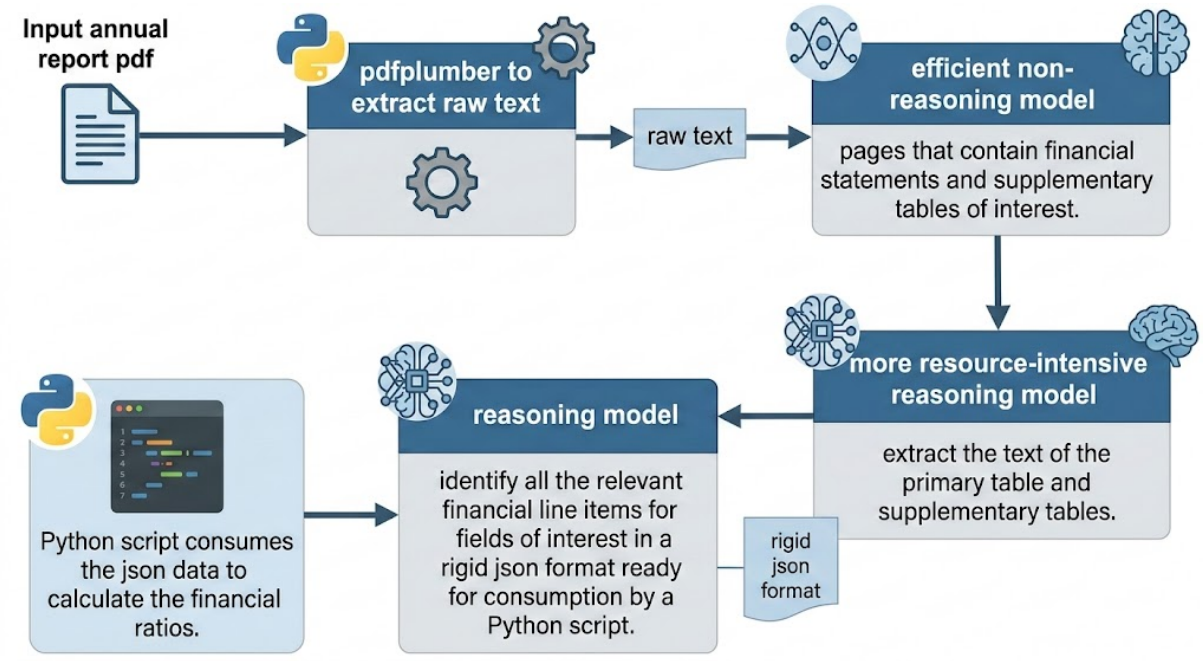
- Depreciation and Amortization

When extracting these values, also give LLM context in which we are obtaining these values, such that in case there isn't a 1:1 mapping of these variables to line items in these statements, LLM can assign the relevant fields and/or combine fields to return the field of interest (e.g., operating expenses in ExxonMobile include:

- Crude oil and product purchases
- Production and manufacturing expenses
- Selling, general and administrative expenses
- Depreciation and depletion
- Exploration expenses, including dry holes
- Non-service pension and postretirement benefit expense
- Other taxes and duties

When extracting the financial statements, also we had to make sure that we extract any supplementary tables. For example, Google's Income Statement had a line for 'Other Income' which combined interest income, interest expenses, and others, without listing out individual components. Only in a separate 'Other Income (Expenses), Net' table do we see the line item 'Interest expense' which we need for calculating Interest Coverage ratio and Debt-to-EBITDA ratio. So, we had to update the LLM query to also extract supplementary tables.

The extraction step is broken down into three LLM agents for extraction and a Python script for actual calculation:




Input annual report pdf -> pdfplumber to extract raw text -> Feed raw text to efficient non-reasoning model to identify pages that contain financial statements and supplementary tables of interest -> Feed the relevant pages to the more resource-intensive reasoning model to extract the text of the primary table and supplementary tables -> Feed the extracted text data to the reasoning model to identify all the relevant financial line items for fields of interest in a rigid json format ready for consumption by a Python script -> Python script consumes the json data to calculate the financial ratios

We have tried using embeddings as done in [Zhang(25)], but we noticed that it is not effective in 2 ways:

1. Vectorization of each page fails to encapsulate all the semantic content. For example, if a page contains a supplementary table of interest along with other supplementary tables and texts, the vectorization of that page may not align closely with our query vector for that supplementary table of interest, and we cannot retrieve that information.

2. Embeddings are more sensitive than LLM to use of different synonyms or languages. We have observed from our attempt to use embedding models on Alibaba's annual report that our query for 'Consolidated Income Statement' failed to retrieve the page with "合併利潤表"

LLMs excel in this area because they are incredibly good at semantic understanding and most of them are trained on multi-lingual data, giving them multi-lingual ability as an emergent property. However, LLMs are susceptible to hallucination. If you try to get it to do too many things at once, or perform a highly quantitative task, it will hallucinate and produce wrong answers. It is essential to break the task down into a step-by-step process that reduces hallucination and response quality, similar to how chain-of-thought prompting improves output quality [Wei(22)]. Also, by first extracting the relevant text before feeding it to the LLM to extract the financial fields, we prevent context dilution which is known to degrade output quality [Liu(23)].

### 3.5.2 run_statement_extraction.py

Using `FinancialStatementExtractor` from `financial_forecast/extraction/`.

#### Primary financial statement extraction

**Step 1: Page number identifying step**

```
prompt = (
    "You are given multiple pages from an annual report. "
    f"Identify which pages contain the table "
    "for the requested query. "
    'Return ONLY JSON in the shape: {"pages": '
    '[<page_number>, ...]}. '
    f"Include all pages that contain the table or line items. "
    'If none, return {"pages": []}.\n\n'
    f"Query: {query}\n\n"
    f"Pages:\n{pages_text}"
)
```

Where:

- query is one of [“Consolidated Balance Sheet”, “Consolidated Income Statement”, “Consolidated Cash Flow Statement”]
- pages_text is the entire text from annual report directly extracted with pdfplumber

**Step 2: Table extraction step**

```
prompt = (
    f"Find the table that corresponds to {query} and output it "
    f"in a nice tabular form from the following pages data.\n"
    f"Pages:\n{pages_text}"
)
```

Where:

- query is one of [“Consolidated Balance Sheet”, “Consolidated Income Statement”, “Consolidated Cash Flow Statement”]
- pages_text is the portion of the pdfplumber-extracted text from only the pages identified in Step 1

Output from Step 2:

**Alphabet Inc. Consolidated Statements of Income**
*(in millions, except per share amounts)*

| Line Item | 2022 | 2023 | 2024 |
|---|---|---|---|
| **Revenues** | $282,836 | $307,394 | $350,018 |
| **Costs and expenses:** | | | |
| Cost of revenues | 126,203 | 133,332 | 146,306 |
| Research and development | 39,500 | 45,427 | 49,326 |
| Sales and marketing | 26,567 | 27,917 | 27,808 |
| General and administrative | 15,724 | 16,425 | 14,188 |
| **Total costs and expenses** | 207,994 | 223,101 | 237,628 |
| **Income from operations** | 74,842 | 84,293 | 112,390 |
| Other income (expense), net | (3,514) | 1,424 | 7,425 |
| **Income before income taxes** | 71,328 | 85,717 | 119,815 |
| Provision for income taxes | 11,356 | 11,922 | 19,697 |
| **Net income** | $59,972 | $73,795 | $100,118 |
| Basic net income per share (Note 12) | $4.59 | $5.84 | $8.13 |
| Diluted net income per share (Note 12) | $4.56 | $5.80 | $8.04 |

*Source: Alphabet 2024 Annual Report, page 57*

## Supplementary tables extraction

### Step 3: Page number identifying step

```
supplementary_query = (
    f"Supplementary disclosures, breakdowns, expansions, "
    f"and note tables for {query}. "
    f"Use this extracted primary statement as context to "
    f"infer relevant line items and "
    f"find related supplementary pages: {primary_context}"
)

prompt = (
    "You are given multiple pages from an annual report. "
    f"Identify which pages contain the table "
    "for the requested query. "
    'Return ONLY JSON in the shape: {"pages": '
    '[<page_number>, ...]}. '
    f"Include all pages that contain the table or line items. "
    'If none, return {"pages": []}.\n\n'
    f"Query: {supplementary_query}\n\n"
    f"Pages:\n{pages_text}"
)
```

Where:

- query is one of [“Consolidated Balance Sheet”, “Consolidated Income Statement”, “Consolidated Cash Flow Statement”]
- primary_context is the extracted table from Step 2
- pages_text is the entire text from annual report directly extracted with pdfplumber

**Step 4: Table extraction step**

```
prompt = (
    "You are extracting supplementary financial disclosures "
    "from annual report pages.\n"
    f"Primary statement: {query}\n"
    "Primary statement extraction context:\n"
    f"{primary_context}\n\n"
    "Infer the line items from the primary statement context "
    "above. "
    "Extract supplementary tables related to those inferred "
    "line items "
    "(for example, breakdowns/expansions/schedules/notes such "
    "as an expanded "
    "'Other Income' table). "
    "You can be generous with the supplementary tables you "
    "extract since it is better to have more than not have "
    "necessary information.\n"
    "Return in a nice tabular format.\n"
    'If none are found, return "No supplementary tables '
    'found".\n\n'
    f"Pages:\n{pages_text}"
)
```

Where:

- query is one of [“Consolidated Balance Sheet”, “Consolidated Income Statement”, “Consolidated Cash Flow Statement”]
- primary_context is the extracted table from Step 2
- pages_text is the portion of the pdfplumber-extracted text from only the pages identified in Step 3

Output from Step 4:

Based on the primary Consolidated Income Statement context, the inferred line items are: Revenues, Cost of revenues, Research and development, Sales and marketing, General and administrative, Income from operations, Other income (expense), net, Income before income taxes, Provision for income taxes, and Net income.

The provided notes to the financial statements contain several supplementary tables that directly expand upon these line items, particularly **Revenues**, **Other income (expense), net**, and **Provision for income taxes**.

Here are the extracted supplementary tables:

### 1. Disaggregated Revenues (Note 2, Page 69)

This table provides a detailed breakdown of the "Revenues" line item.

**By Type**

| Revenue Type | 2022 | 2023 | 2024 |
|---|---|---|---|
| Google Search & other | $162,450 | $175,033 | $198,084 |
| YouTube ads | 29,243 | 31,510 | 36,147 |
| Google Network | 32,780 | 31,312 | 30,359 |
| **Google advertising** | **224,473** | **237,855** | **264,590** |
| Google subscriptions, platforms, and devices | 29,055 | 34,688 | 40,340 |
| **Google Services total** | **253,528** | **272,543** | **304,930** |
| Google Cloud | 26,280 | 33,088 | 43,229 |
| Other Bets | 1,068 | 1,527 | 1,648 |
| Hedging gains (losses) | 1,960 | 236 | 211 |
| **Total revenues** | **$282,836** | **$307,394** | **$350,018** |

**By Geography**

| Geography | 2022 | 2023 | 2024 |
|---|---|---|---|
| United States | $134,814 (48%) | $146,286 (47%) | $170,447 (49%) |
| EMEA | 82,062 (29%) | 91,038 (30%) | 102,127 (29%) |
| APAC | 47,024 (16%) | 51,514 (17%) | 56,815 (16%) |
| Other Americas | 16,976 (6%) | 18,320 (6%) | 20,418 (6%) |
| Hedging gains (losses) | 1,960 (1%) | 236 (0%) | 211 (0%) |
| **Total revenues** | **$282,836 (100%)** | **$307,394 (100%)** | **$350,018 (100%)** |

### 2. Gains and Losses on Debt Securities (Note 3, Page 72)

This table details a component of "Other income (expense), net".

| Description | 2022 | 2023 | 2024 |
|---|---|---|---|
| Unrealized gain (loss) on fair value option debt securities | $(557) | $386 | $30 |
| Gross realized gain on debt securities | 103 | 182 | 482 |
| Gross realized loss on debt securities | (1,588) | (1,833) | (1,553) |
| (Increase) decrease in allowance for credit losses | (22) | 50 | (2) |
| **Total gain (loss) on debt securities recognized in other income (expense), net** | **$(2,064)** | **$(1,215)** | **$(1,043)** |

### 3. Gains and Losses on Equity Securities (Note 3, Page 73)

This table details a component of "Other income (expense), net".

| Description | 2022 | 2023 | 2024 |
|---|---|---|---|
| Realized net gain (loss) on equity securities sold during the period | $(442) | $690 | $186 |
| Unrealized net gain (loss) on marketable equity securities | (3,242) | 790 | 156 |
| Unrealized net gain (loss) on non-marketable equity securities | 229 | (1,088) | 3,372 |
| **Total gain (loss) on equity securities in other income (expense), net** | **$(3,455)** | **$392** | **$3,714** |

### 4. Derivative Gains/(Losses) in Income Statement (Note 3, Page 76)

This table details the impact of derivatives on "Revenues" and "Other income (expense), net".

| Description | 2022 Revenues | 2022 OI&E | 2023 Revenues | 2023 OI&E | 2024 Revenues | 2024 OI&E |
|---|---|---|---|---|---|---|
| Effect of cash flow hedges:<br>Foreign exchange contracts<br>Amount reclassified from AOCI to income<br>Amount excluded from assessment of effectiveness (amortized) | $2,046<br>(85) | $0<br>0 | $213<br>24 | $0<br>0 | $174<br>37 | $0<br>0 |
| Effect of fair value hedges:<br>Foreign exchange contracts<br>Hedged items<br>Derivatives designated as hedging instruments<br>Amount excluded from assessment of effectiveness | 0 | (162)<br>163<br>16 | 0 | 59<br>(59)<br>15 | 0 | (59)<br>58<br>13 |
| Effect of net investment hedges:<br>Foreign exchange contracts<br>Amount excluded from assessment of effectiveness | 0 | 171 | 0 | 187 | 0 | 137 |
| Effect of non designated hedges:<br>Foreign exchange contracts<br>Other contracts | 0 | (395)<br>144 | 0 | 7<br>53 | 0 | 335<br>174 |
| **Total gains (losses)** | **$1,961** | **$(63)** | **$237** | **$262** | **$211** | **$658** |

### 5. Components of Other Income (Expense), Net (Note 7, Page 83)

This table provides a full breakdown of the "Other income (expense), net" line item.

| Component | 2022 | 2023 | 2024 |
|---|---|---|---|
| Interest income | $2,174 | $3,865 | $4,482 |
| Interest expense | (357) | (308) | (268) |
| Foreign currency exchange gain (loss), net | (654) | (1,238) | (409) |
| Gain (loss) on debt securities, net | (2,064) | (1,215) | (1,043) |
| Gain (loss) on equity securities, net | (3,455) | 392 | 3,714 |
| Performance fees | 798 | 257 | 218 |
| Income (loss) and impairment from equity method investments, net | (337) | (628) | (188) |
| Other | 381 | 299 | 919 |
| **Other income (expense), net** | **$(3,514)** | **$1,424** | **$7,425** |

### 6. Income Tax Reconciliation and Deferred Taxes (Note 14, Pages 85-86)

These tables provide details related to "Provision for income taxes".

**Effective Tax Rate Reconciliation**

| Rate Component | 2022 | 2023 | 2024 |
|---|---|---|---|
| U.S. federal statutory tax rate | 21.0% | 21.0% | 21.0% |
| Foreign income taxed at different rates | 3.0 | 0.3 | 0.5 |
| Foreign-derived intangible income deduction | (5.4) | (4.6) | (3.8) |
| Stock-based compensation expense | (1.2) | (0.8) | (1.5) |
| Federal research credit | (2.2) | (1.8) | (1.5) |
| Deferred tax asset valuation allowance | 0.9 | 0.6 | 0.4 |
| State and local income taxes | 0.8 | 1.0 | 1.1 |
| Effect of tax law change | 0.0 | (1.4) | 0.0 |
| Other | (1.0) | (0.4) | 0.2 |
| **Effective tax rate** | **15.9%** | **13.9%** | **16.4%** |

**Significant Components of Deferred Tax Assets and Liabilities (as of December 31)**

| Component | 2023 | 2024 |
|---|---|---|
| **Deferred tax assets:** | | |
| Accrued employee benefits | $1,855 | $1,834 |
| Accruals and reserves not currently deductible | 2,481 | 2,552 |
| Tax credits | 6,609 | 6,384 |
| Net operating losses | 2,965 | 3,472 |
| Operating leases | 3,526 | 3,336 |
| Capitalized research and development | 17,757 | 25,903 |
| Other | 1,951 | 1,376 |
| **Total deferred tax assets** | **37,144** | **44,857** |
| Valuation allowance | (10,999) | (11,493) |
| **Total deferred tax assets net of valuation allowance** | **26,145** | **33,364** |
| **Deferred tax liabilities:** | | |
| Property and equipment, net | (8,189) | (9,932) |
| Net investment gains | (2,405) | (2,978) |
| Operating leases | (2,965) | (2,986) |
| Other | (902) | (1,008) |
| **Total deferred tax liabilities** | **(14,461)** | **(16,904)** |
| **Net deferred tax assets (liabilities)** | **$11,684** | **$16,460** |

### 7. Segment Revenue and Operating Income (Note 15, Page 96)

This table provides a breakdown of "Revenues" and "Income from operations" by reportable segment.

| Segment | 2022 Revenues | 2023 Revenues | 2024 Revenues | 2022 Operating Income (Loss) | 2023 Operating Income (Loss) | 2024 Operating Income (Loss) |
|---|---|---|---|---|---|---|
| Google Services | $253,528 | $272,543 | $304,930 | $82,699 | $95,858 | $121,263 |
| Google Cloud | 26,280 | 33,088 | 43,229 | (1,922) | 1,716 | 6,112 |
| Other Bets | 1,068 | 1,527 | 1,648 | (4,636) | (4,095) | (4,444) |
| Alphabet-level activities | — | — | — | (1,299) | (9,186) | (10,541) |
| Hedging gains (losses) | 1,960 | 236 | 211 | — | — | — |
| **Total** | **$282,836** | **$307,394** | **$350,018** | **$74,842** | **$84,293** | **$112,390** |

**Step 5: Extracting relevant fields as JSON for ratio calculation** Then, from this final extracted information containing primary and supplementary tables, we extract the relevant elements from each financial statement (Balance Sheet, Income Statement, or Cash Flows). When crafting the prompt, it is important to be careful so that it prevents LLM from hallucinating as much as possible. LLM is quite bad at doing math on the spot, but is much better at copying numbers. So, for retrieving fields that often requires summation of multiple elements, we ask LLM to copy the elements into an array instead of asking it to sum them up and return a single value. We use the following prompt:

```
STATEMENT_CONFIGS: Dict[str, Dict[str, Any]] = {
    "consolidated-balance-sheet": {
        "suffix": ".consolidated-balance-sheet.llm.json",
        "output_suffix": ".consolidated-balance-sheet.normalized.json",
        "fields": [
            "cash_and_cash_equivalents",
            "marketable_securities",
            "accounts_receivable",
            "total_current_liabilities",
            "total_debt_short_term_and_long_term",
            "total_equity",
            "total_assets",
        ],
    },
    "consolidated-income-statement": {
        "suffix": ".consolidated-income-statement.llm.json",
        "output_suffix": ".consolidated-income-statement.normalized.json",
        "fields": [
            "revenue",
            "total_operating_cost",
            "net_income",
            "income_tax_expense",
            "interest_expenses",
        ],
    },
    "consolidated-cash-flow-statement": {
        "suffix": ".consolidated-cash-flow-statement.llm.json",
        "output_suffix": ".consolidated-cash-flow-statement.normalized.json
    ",
        "fields": [
            "depreciation_and_amortization",
        ],
    },
}

fields_schema = ",\n".join(
    [f'  "{field}": number[]'
     for field in required_fields]
)

prompt = (
    "You are a financial statement extraction engine.\n"
    f"Extract normalized {statement_type} values from the "
    f"statement and supplementary tables.\n\n"
    "Return ONLY valid JSON with this exact schema:\n"
    "{\n"
    '  "company_id": "string",\n'
    '  "periods": [\n'
    "    {\n"
    '      "year": "string",\n'
    '      "currency": "string",\n'
    '      "values": {\n'
    f"        {fields_schema}\n"
    "      }\n"
    "    }\n"
    "  ],\n"
    '  "notes": string\n'
```

```
    "}\n\n"
    "Rules:\n"
    "1) Use every period/column available in the statement.\n"
    "2) Return numeric array values only (no commas, no currency "
    "symbols, no percent signs).\n"
    "3) If a value cannot be directly mapped to a field, try to "
    "map number(s) to the corresponding field by taking into "
    "account the industry the company is in, and note what you "
    "did in notes field. If you cannot find a match, return an "
    "empty array.\n"
    "4) If multiple numbers make up a field, return them in an "
    "array without adding them up.\n"
    "5) If there are multiple close synonyms, use best "
    "accounting match.\n"
    "6) For parentheses negatives, return negative numbers.\n"
    "7) For year, report the year of the period only.\n"
    "8) For currency, report its formal 3-letter acronym.\n"
    "9) For total_operating_cost, list all elements that should "
    "be included in standard practice for the industry the "
    "company is in. Elements that should be subtracted should "
    "be negative. The sign convention should be such that if an "
    "element is a cost, it should be negative, and if it is an "
    "income, it should be positive.\n"
    "10) Return JSON only.\n\n"
    f"company_id: {company_id}\n\n"
    "statement and supplementary tables:\n"
    f"{statement_and_supplementary_tables}\n"
)
```

Where

- required_fields are selected from STATEMENT_CONFIGS and are dependent on the type of financial statement we are feeding the LLM
- statement_type is one of [“Consolidated Balance Sheet”, “Consolidated Income Statement”, “Consolidated Cash Flow Statement”]
- statement_text is the extracted primary and supplementary tables from Step 4

The LLM struggled to identify the correct elements for marketable securities. So, we add the definition of marketable securities according to definition:

- short_term_market_securities: these are liquid, unrestricted debt or equity investments intended to be sold in the short term, e.g., U.S. Treasuries, commercial paper, money market funds, publicly traded equities, short-term investments.

The LLM kept wanting to add up the components before reporting them in a single field, e.g., it kept calculating the total debt by adding up the short-term and long-term borrowings often with hallucination and incorrect summed results.

So, we had to explicitly tell LLM to return the original source label and its value as an array of maps:

```
fields_schema = ",\n".join(
    [
        f'  "{field}": [{{"source_label": number}}]'
        for field in required_fields
    ]
```

```
)

prompt = (
    "You are a financial statement extraction engine.\n"
    f"Extract normalized {statement_type} values from the "
    f"statement and supplementary tables.\n\n"
    "Return ONLY valid JSON with this exact schema:\n"
    "{\n"
    '  "company_id": "string",\n'
    '  "periods": [\n'
    "    {\n"
    '      "year": "string",\n'
    '      "currency": "string",\n'
    '      "scale": number,\n'
    '      "values": {\n'
    f"        {fields_schema}\n"
    "      }\n"
    "    }\n"
    "  ],\n"
    '  "notes": string\n'
    "}\n\n"
    "Rules:\n"
    "1) Use every period/column available in the statement.\n"
    "2) Return an array of maps that make up each field where "
    "each map is {source_label: number} (source_label = actual "
    "label from the statement; no commas, no currency symbols, "
    "no percent signs in numbers).\n"
    "3) Ensure that the elements in the array correctly make up "
    "the total value of the field.\n"
    "4) If multiple elements need to be combined to make up a "
    "field, return all of them individually in an array.\n"
    "5) If a value(s) cannot be directly mapped to a field, "
    "try to map element(s) to the corresponding field by "
    "taking into account the industry the company is in, and "
    "note your reasoning in the notes field. If you cannot "
    "find a match, return an empty array.\n"
    "6) If there are multiple close synonyms, use best "
    "accounting match.\n"
    "7) For year, report the year of the period only.\n"
    "8) For currency, report its formal 3-letter acronym.\n"
    "9) For scale, report the scale of the values in the "
    "statement. For example, if the values are in millions, "
    "the scale should be 1E6.\n"
    "10) For total_operating_cost, list all elements that "
    "reflect the total operational costs incurred in "
    "generating income and are included in standard practice "
    "for the industry the company is in.\n"
    "11) Generally, numbers in parentheses are negative.\n"
    "12) For income_tax_expense, interest_expenses, and "
    "total_operating_cost, the sign convention is the "
    "opposite: if an element reduces income, it should be "
    "positive; and if it increases income, it should be "
    "negative.\n"
    "13) For short_term_market_securities, these are liquid, "
    "unrestricted debt or equity investments intended to be "
    "sold in the short term, e.g., U.S. Treasuries, "
    "commercial paper, money market funds, publicly traded "
    "equities, short-term investments.\n"
```

```
    "14) Return JSON only.\n\n"
    f"company_id: {company_id}\n\n"
    "statement and supplementary tables:\n"
    f"{statement_and_supplementary_tables}\n"
)
```

Output example for GM's 2023 annual report:

**Before: LLM attempts to add the total debt by itself**

```
{
    "year": "2023",
    "currency": "USD",
    "values": {
        "cash_and_cash_equivalents": [18853.0],
        "short_term_market_securities": [7613.0],
        "accounts_receivable": [12378.0],
        "total_current_liabilities": [94445.0],
        "total_debt_short_term_and_long_term": [16413.0, 105327.0],
        "total_equity": [68189.0],
        "total_assets": [273064.0]
    }
},
```

**After: LLM lists out all the original elements without summation**

```
{
    "year": "2023",
    "currency": "USD",
    "scale": 1000000,
    "values": {
        "cash_and_cash_equivalents": [
            {"Cash and cash equivalents": 18853.0}
        ],
        "short_term_market_securities": [
            {"Marketable debt securities": 7613.0}
        ],
        "accounts_receivable": [
            {"Accounts and notes receivable, net": 12378.0},
            {"GM Financial receivables, net (current)": 39076.0}
        ],
        "total_current_liabilities": [
            {"Total current liabilities": 94445.0}
        ],
        "total_debt_short_term_and_long_term": [
            {"Short-term debt and current portion of long-term debt - Automotive": 428.0},
            {"Short-term debt and current portion of long-term debt - GM Financial": 38540.0},
            {"Long-term debt - Automotive": 15985.0},
            {"Long-term debt - GM Financial": 66788.0}
        ],
        "total_equity": [
            {"Total Equity": 68189.0}
        ],
        "total_assets": [
            {"Total Assets": 273064.0}
        ]
    }
},
```

```
...
```

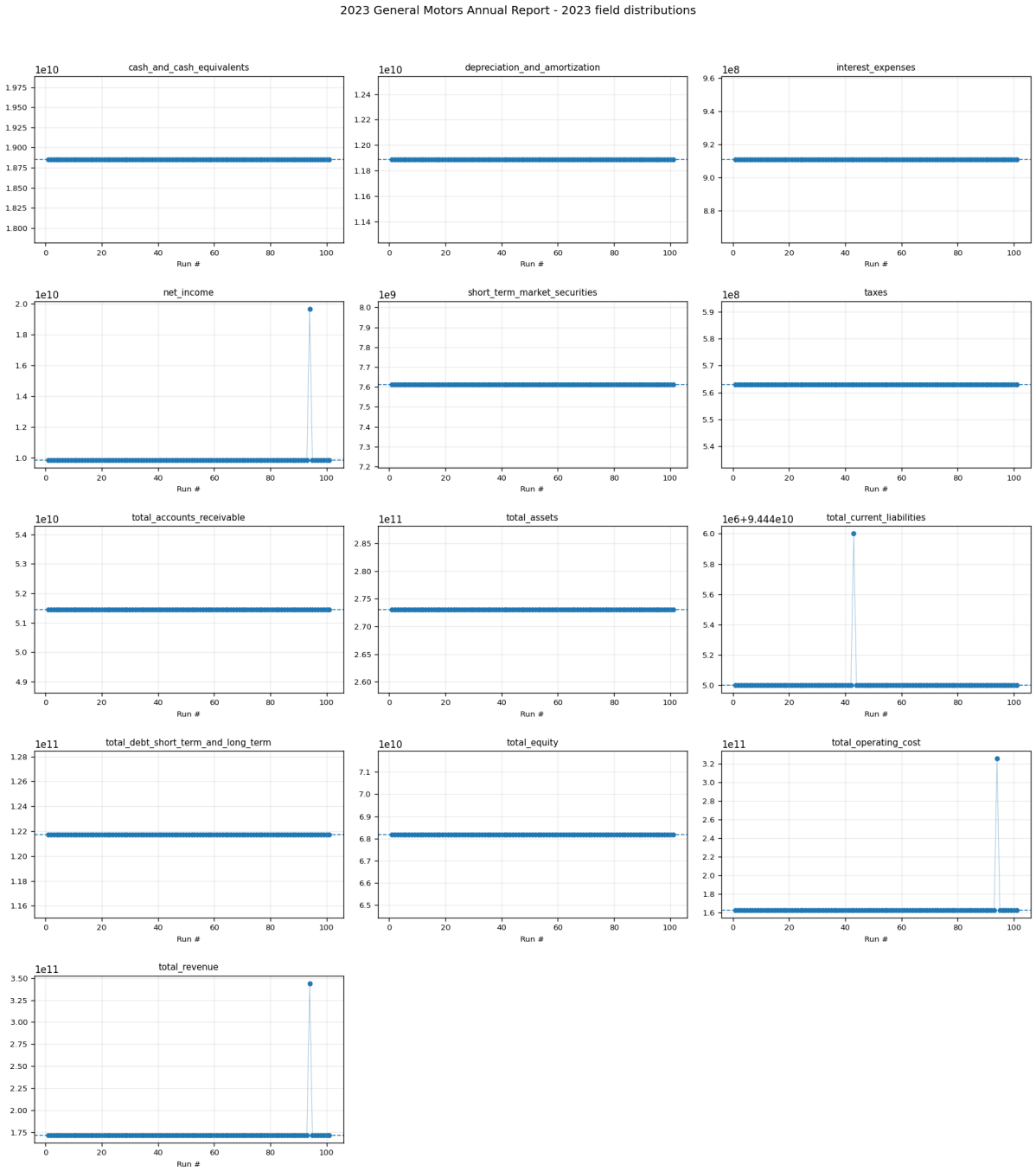
Even with clever prompt engineering, LLM still hallucinates. To measure its hallucination rate, we repeated Step 5 (i.e., extracting json from financial statements for financial ratio calculations) for 101 runs and plotted each field value vs run using the following scripts:

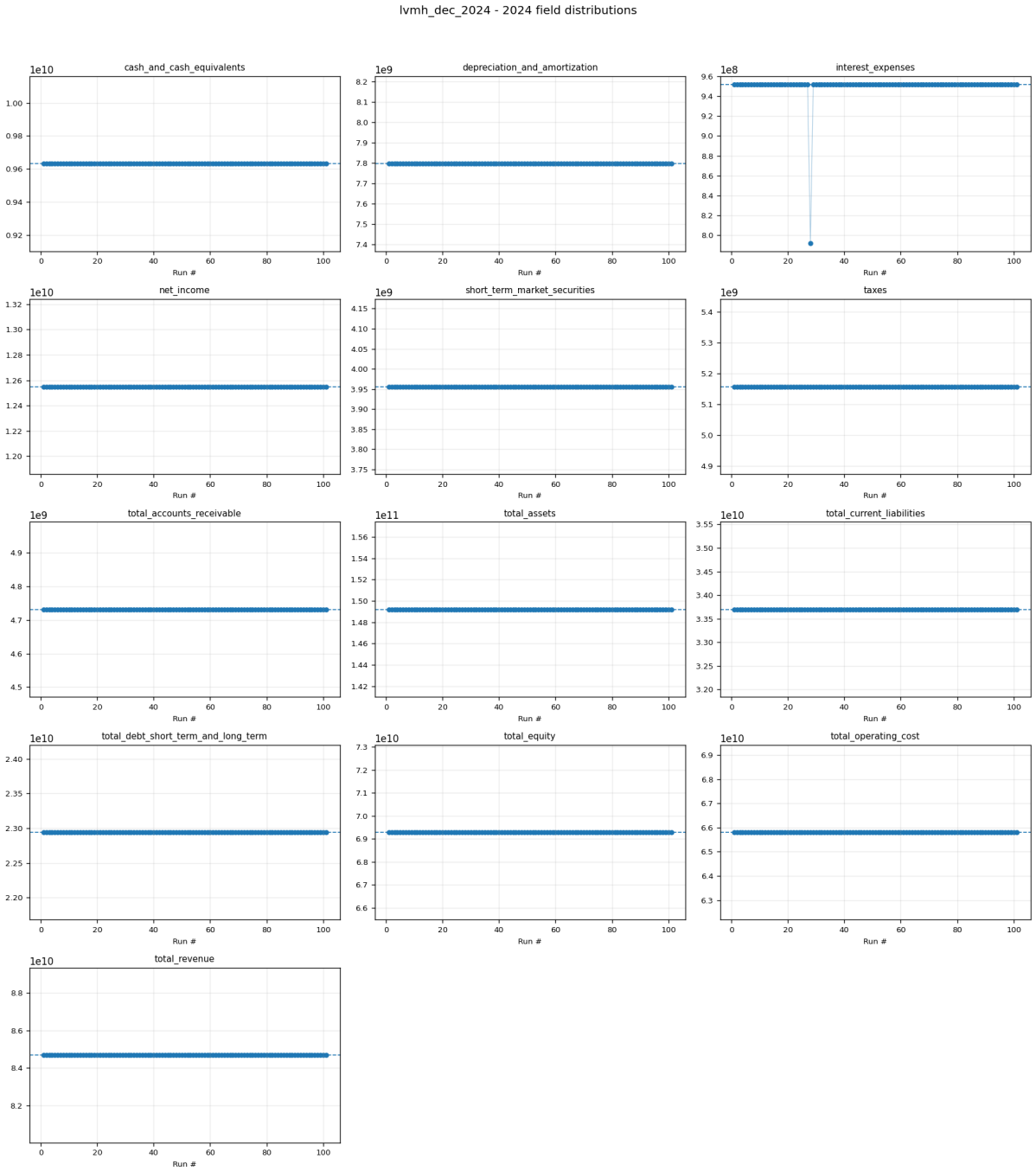
Run multi-run normalization via `run_statement_normalization_multi.py`, which invokes the `StatementNormalizer` and `MedianRatioCalculator` from `financial_forecast/extraction`

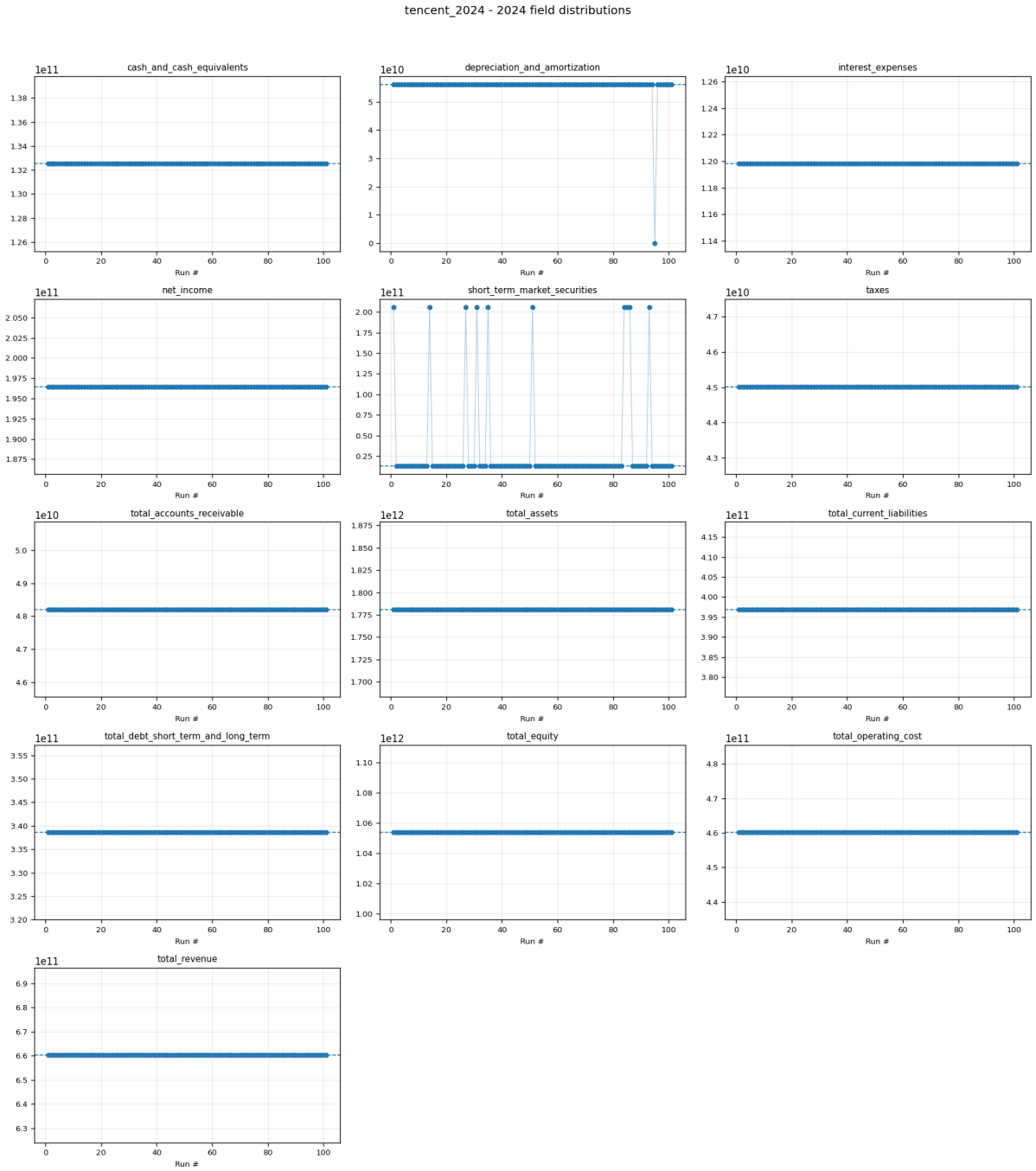
```
python run_statement_normalization_multi.py
```

This performs multiple LLM normalization passes, computes median-aggregated ratios, and generates hallucination rate analysis via the `statement_hallucination` module:

The results are reproduced below for GM, LVMH, Tencent, Alibaba, JPMorgan Chase, Exxon Mobil, Volkswagen, Microsoft, and Google:


2023 General Motors Annual Report - 2023 field distributions
cash_and_cash_equivalents
depreciation_and_amortization
interest_expenses
net_income
short_term_market_securities
taxes
total_accounts_receivable
total_assets
total_current_liabilities
total_debt_short_term_and_long_term
total_equity
total_operating_cost
total_revenue
Run #


lvmh_dec_2024 - 2024 field distributions
cash_and_cash_equivalents
depreciation_and_amortization
interest_expenses
net_income
short_term_market_securities
taxes
total_accounts_receivable
total_assets
total_current_liabilities
total_debt_short_term_and_long_term
total_equity
total_operating_cost
total_revenue
Run #


tencent_2024 - 2024 field distributions
cash_and_cash_equivalents
depreciation_and_amortization
interest_expenses
net_income
short_term_market_securities
taxes
total_accounts_receivable
total_assets
total_current_liabilities
total_debt_short_term_and_long_term
total_equity
total_operating_cost
total_revenue
Run #

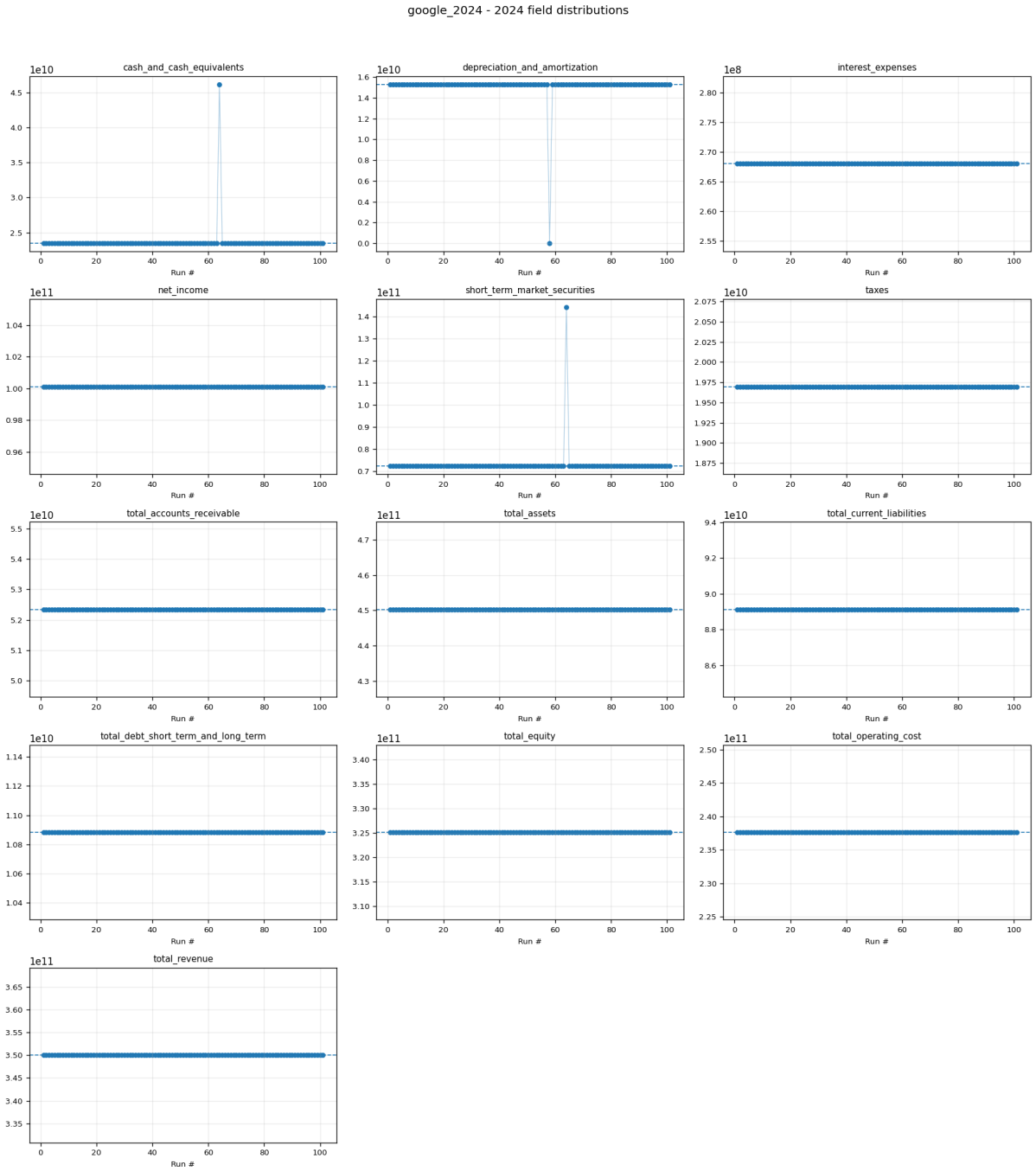

alibaba_2025 - 2025 field distributions
cash_and_cash_equivalents
depreciation_and_amortization
interest_expenses
net_income
short_term_market_securities
taxes
total_accounts_receivable
total_assets
total_current_liabilities
total_debt_short_term_and_long_term
total_equity
total_operating_cost
total_revenue
Run #


jpmorgan_2024 - 2024 field distributions
cash_and_cash_equivalents
depreciation_and_amortization
interest_expenses
net_income
short_term_market_securities
taxes
total_accounts_receivable
total_assets
total_current_liabilities
total_debt_short_term_and_long_term
total_equity
total_operating_cost
total_revenue
Run #


exxon_2024 - 2024 field distributions
cash_and_cash_equivalents
depreciation_and_amortization
interest_expenses
net_income
short_term_market_securities
taxes
total_accounts_receivable
total_assets
total_current_liabilities
total_debt_short_term_and_long_term
total_equity
total_operating_cost
total_revenue
Run #


volkswagen_2024 - 2024 field distributions
cash_and_cash_equivalents
depreciation_and_amortization
interest_expenses
net_income
short_term_market_securities
taxes
total_accounts_receivable
total_assets
total_current_liabilities
total_debt_short_term_and_long_term
total_equity
total_operating_cost
total_revenue
Run #


microsoft_2025 - 2025 field distributions
cash_and_cash_equivalents
depreciation_and_amortization
interest_expenses
net_income
short_term_market_securities
taxes
total_accounts_receivable
total_assets
total_current_liabilities
total_debt_short_term_and_long_term
total_equity
total_operating_cost
total_revenue
Run #




We also calculate the top 15 hallucinated fields as below:

```
- exxon_2024 | total_operating_cost | rate_non_null=0.2970 (90/303)
- microsoft_2025 | depreciation_and_amortization | rate_non_null=0.1980 (60/303)
- volkswagen_2024 | net_accounts_receivable | rate_non_null=0.1881 (38/202)
- jpmorgan_2024 | depreciation_and_amortization | rate_non_null=0.1683 (51/303)
- volkswagen_2024 | depreciation_and_amortization | rate_non_null=0.1683 (34/202)
- volkswagen_2024 | total_operating_cost | rate_non_null=0.1287 (26/202)
- alibaba_2025 | cash_and_cash_equivalents | rate_non_null=0.0990 (20/202)
- alibaba_2025 | short_term_market_securities | rate_non_null=0.0990 (20/202)
```

```
- alibaba_2025 | net_accounts_receivable | rate_non_null=0.0990 (20/202)
- alibaba_2025 | total_assets | rate_non_null=0.0990 (20/202)
- alibaba_2025 | total_current_liabilities | rate_non_null=0.0990 (20/202)
- alibaba_2025 | total_debt_short_term_and_long_term | rate_non_null=0.0990
    (20/202)
- alibaba_2025 | total_equity | rate_non_null=0.0990 (20/202)
- tencent_2024 | short_term_market_securities | rate_non_null=0.0990
   (20/202)
- jpmorgan_2024 | total_operating_cost | rate_non_null=0.0792 (24/303)
```

As you can see, the hallucination is much better than a coin-toss; LLM retrieves the correct values more than 70% of the time. Thus, it is important to repeat the extraction step multiple times to retrieve the correct values to do any further quantitative analysis.

The most hallucinated value was ExxonMobile's operating cost because LLM was not sure whether to include "Other taxes and duties" as part of operating cost. It reasoned more often than not that it is part of the operating cost. There are several ways to reduce this hallucination rate. One is to give direct examples of what we want for this specific item for this company. Another is to reword the name or the definition of "Operating Cost" in a way that there are less ambiguities for LLM in determining the elements that belong to this field.

As an example of this second point, we were able to reduce the hallucination rate in extracting the Short-Term Market Securities from JPMorgan Chase's annual report. Previously, we used the following name and definition in the LLM prompt:

For marketable_securities, these are most liquid, unrestricted debt or equity investments intended to be sold in the near term (e.g., U.S. Treasuries, commercial paper, money market funds, publicly traded equities)

And we observed high hallucination rate for this field because LLM outputted different array of elements that made up marketable securities, as evidenced by the following 3 extraction runs:

Run 1:

```
"marketable_securities": [
  {"Federal funds sold and securities purchased under resale agreements":
   295001.0},
  {"Securities borrowed": 219546.0},
  {"Available-for-sale securities": 406852.0}
],
```

Run 2:

```
"marketable_securities": [
  {"Federal funds sold and securities purchased under resale agreements":
   295001.0},
  {"Securities borrowed": 219546.0},
  {"Trading assets": 637784.0},
  {"Available-for-sale securities": 406852.0}
],
```

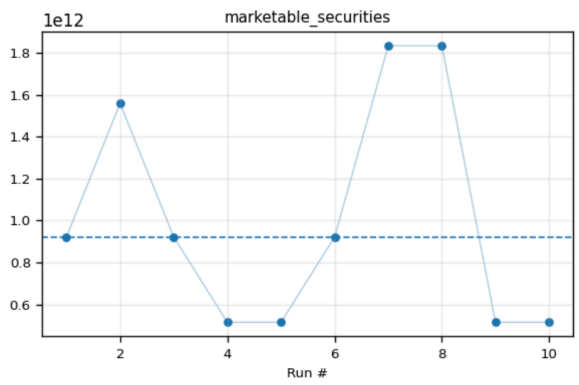
Run 5:

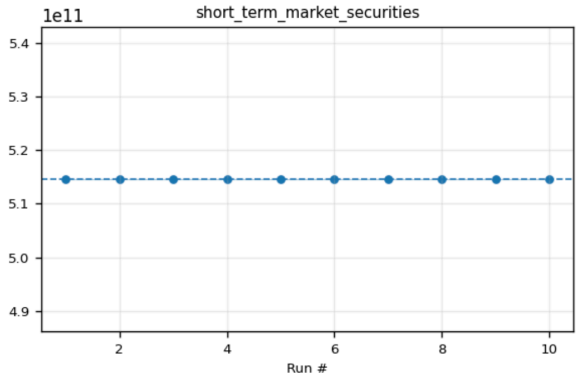
```
"marketable_securities": [
  {"Federal funds sold and securities purchased under resale agreements":
   295001.0},
  {"Securities borrowed": 219546.0}
],
```

Then we updated our naming and definition from "marketable_securities" to "short_term_market_securities":

For short_term_market_securities, these are liquid, unrestricted debt or equity investments intended to be sold in the short term, e.g., U.S. Treasuries, commercial paper, money market funds, publicly traded equities, short-term investments.

We see that the hallucination rate practically goes to zero:




After obtaining the correct field values, calculating the various financial ratios is straightforward. We write a Python script for the calculation, which gives us definite results. The ratio calculation runs automatically after normalization via `run_ratio_calculation.py` (using `RatioCalculator` from `financial_forecast/extraction/statement_ratios.py`)

Calculated fields and financial ratios for GM are as follows for 2022 and 2023 in the 2023 annual report:

```
{
  "year": "2022",
  "currency": "USD",
  "field_values": {
    "cash_and_cash_equivalents": 19153000000.0,
    "short_term_market_securities": 12150000000.0,
    "net_accounts_receivable": 46956000000.0,
    "total_current_liabilities": 91173000000.0,
    "total_debt_short_term_and_long_term": 114699000000.0,
    "total_equity": 71927000000.0,
    "total_assets": 264037000000.0,
    "total_revenue": 156735000000.0,
    "total_operating_cost": 146421000000.0,
    "net_income": 9708000000.0,
    "income_tax_expense": 1888000000.0,
    "interest_expenses": 987000000.0,
    "depreciation_and_amortization": 11290000000.0
  },
  "derived_values": {
    "ebit": 12583000000.0,
    "ebitda": 23873000000.0,
```

```
    "quick_assets": 78259000000.0,
    "debt_plus_equity": 186626000000.0
  },
  "ratios": {
    "cost_to_income_ratio": 0.9341946597760551,
    "quick_ratio": 0.8583571890801005,
    "debt_to_equity_ratio": 1.5946584731741904,
    "debt_to_assets_ratio": 0.43440502656824614,
    "debt_to_capital_ratio": 0.6145928220076516,
    "debt_to_ebitda_ratio": 4.804549072173585,
    "interest_coverage_ratio": 12.74873353596758
  }
},
{
  "year": "2023",
  "currency": "USD",
  "field_values": {
    "cash_and_cash_equivalents": 18853000000.0,
    "short_term_market_securities": 7613000000.0,
    "net_accounts_receivable": 51454000000.0,
    "total_current_liabilities": 94445000000.0,
    "total_debt_short_term_and_long_term": 121741000000.0,
    "total_equity": 68189000000.0,
    "total_assets": 273064000000.0,
    "total_revenue": 171842000000.0,
    "total_operating_cost": 162544000000.0,
    "net_income": 9840000000.0,
    "income_tax_expense": 563000000.0,
    "interest_expenses": 911000000.0,
    "depreciation_and_amortization": 11888000000.0
  },
  "derived_values": {
    "ebit": 11314000000.0,
    "ebitda": 23202000000.0,
    "quick_assets": 77920000000.0,
    "debt_plus_equity": 189930000000.0
  },
  "ratios": {
    "cost_to_income_ratio": 0.9458921567486412,
    "quick_ratio": 0.8250304409974059,
    "debt_to_equity_ratio": 1.7853466101570634,
    "debt_to_assets_ratio": 0.44583321126182873,
    "debt_to_capital_ratio": 0.640978255146633,
    "debt_to_ebitda_ratio": 5.247004568571675,
    "interest_coverage_ratio": 12.419319429198683
  }
}
```


### 3.6 h, i) Applicability to Other Annual Reports

Our script works for other annual reports as well, including LVMH, Tencent, Alibaba, JPMorgan Chase, ExxonMobil, Volkswagen, Microsoft, and Google:

LVMH:


```
{
  "year": "2022",
  "currency": "EUR",
```

```
    "field_values": {
      "cash_and_cash_equivalents": 7301000000.0,
      "short_term_market_securities": 3552000000.0,
      "net_accounts_receivable": 4258000000.0,
      "total_current_liabilities": 31543000000.0,
      "total_debt_short_term_and_long_term": 19739000000.0,
      "total_equity": 56604000000.0,
      "total_assets": 134646000000.0,
      "total_revenue": 79184000000.0,
      "total_operating_cost": 58220000000.0,
      "net_income": 14084000000.0,
      "income_tax_expense": 5362000000.0,
      "interest_expenses": 271000000.0,
      "depreciation_and_amortization": 6226000000.0
    },
    "derived_values": {
      "ebit": 19717000000.0,
      "ebitda": 25943000000.0,
      "quick_assets": 15111000000.0,
      "debt_plus_equity": 76343000000.0
    },
    "ratios": {
      "cost_to_income_ratio": 0.7352495453626995,
      "quick_ratio": 0.4790603303427068,
      "debt_to_equity_ratio": 0.3487209384495795,
      "debt_to_assets_ratio": 0.14659923057498922,
      "debt_to_capital_ratio": 0.2585567766527383,
      "debt_to_ebitda_ratio": 0.7608603476853101,
      "interest_coverage_ratio": 72.75645756457564
    }
  },
  {
    "year": "2023",
    "currency": "EUR",
    "field_values": {
      "cash_and_cash_equivalents": 7774000000.0,
      "short_term_market_securities": 3490000000.0,
      "net_accounts_receivable": 4728000000.0,
      "total_current_liabilities": 33145000000.0,
      "total_debt_short_term_and_long_term": 21907000000.0,
      "total_equity": 62701000000.0,
      "total_assets": 143694000000.0,
      "total_revenue": 86153000000.0,
      "total_operating_cost": 63600000000.0,
      "net_income": 15174000000.0,
      "income_tax_expense": 5673000000.0,
      "interest_expenses": 760000000.0,
      "depreciation_and_amortization": 7177000000.0
    },
    "derived_values": {
      "ebit": 21607000000.0,
      "ebitda": 28784000000.0,
      "quick_assets": 15992000000.0,
      "debt_plus_equity": 84608000000.0
    },
    "ratios": {
      "cost_to_income_ratio": 0.7382215361043725,
      "quick_ratio": 0.4824860461608086,
```

```
    "debt_to_equity_ratio": 0.3493883670116904,
    "debt_to_assets_ratio": 0.1524559132601222,
    "debt_to_capital_ratio": 0.2589235060514372,
    "debt_to_ebitda_ratio": 0.7610825458588104,
    "interest_coverage_ratio": 28.430263157894736
  }
},
{
  "year": "2024",
  "currency": "EUR",
  "field_values": {
    "cash_and_cash_equivalents": 9631000000.0,
    "short_term_market_securities": 3956000000.0,
    "net_accounts_receivable": 4731000000.0,
    "total_current_liabilities": 33696000000.0,
    "total_debt_short_term_and_long_term": 22942000000.0,
    "total_equity": 69287000000.0,
    "total_assets": 149190000000.0,
    "total_revenue": 84683000000.0,
    "total_operating_cost": 65804000000.0,
    "net_income": 12550000000.0,
    "income_tax_expense": 5157000000.0,
    "interest_expenses": 952000000.0,
    "depreciation_and_amortization": 7796000000.0
  },
  "derived_values": {
    "ebit": 18659000000.0,
    "ebitda": 26455000000.0,
    "quick_assets": 18318000000.0,
    "debt_plus_equity": 92229000000.0
  },
  "ratios": {
    "cost_to_income_ratio": 0.7770626926301619,
    "quick_ratio": 0.5436253561253561,
    "debt_to_equity_ratio": 0.3311155050731017,
    "debt_to_assets_ratio": 0.15377706280581807,
    "debt_to_capital_ratio": 0.24875039304340282,
    "debt_to_ebitda_ratio": 0.8672084672084672,
    "interest_coverage_ratio": 19.599789915966387
  }
}
```


Tencent:


```
{
  "year": "2023",
  "currency": "CNY",
  "field_values": {
    "cash_and_cash_equivalents": 172320000000.0,
    "short_term_market_securities": 14903000000.0,
    "net_accounts_receivable": 46606000000.0,
    "total_current_liabilities": 352157000000.0,
    "total_debt_short_term_and_long_term": 348618000000.0,
    "total_equity": 873681000000.0,
    "total_assets": 1577246000000.0,
    "total_revenue": 609015000000.0,
    "total_operating_cost": 453642000000.0,
    "net_income": 118048000000.0,
```

```
    "income_tax_expense": 43276000000.0,
    "interest_expenses": 12268000000.0,
    "depreciation_and_amortization": 59008000000.0
  },
  "derived_values": {
    "ebit": 173592000000.0,
    "ebitda": 232600000000.0,
    "quick_assets": 233829000000.0,
    "debt_plus_equity": 1222299000000.0
  },
  "ratios": {
    "cost_to_income_ratio": 0.744878204970321,
    "quick_ratio": 0.6639907768410113,
    "debt_to_equity_ratio": 0.3990220686955536,
    "debt_to_assets_ratio": 0.22102956672579926,
    "debt_to_capital_ratio": 0.285214992403659,
    "debt_to_ebitda_ratio": 1.4987876182287188,
    "interest_coverage_ratio": 14.149983697424194
  }
},
{
  "year": "2024",
  "currency": "CNY",
  "field_values": {
    "cash_and_cash_equivalents": 132519000000.0,
    "short_term_market_securities": 12913000000.0,
    "net_accounts_receivable": 48203000000.0,
    "total_current_liabilities": 396909000000.0,
    "total_debt_short_term_and_long_term": 338615000000.0,
    "total_equity": 1053896000000.0,
    "total_assets": 1780995000000.0,
    "total_revenue": 660257000000.0,
    "total_operating_cost": 460160000000.0,
    "net_income": 196467000000.0,
    "income_tax_expense": 45018000000.0,
    "interest_expenses": 11981000000.0,
    "depreciation_and_amortization": 56213000000.0
  },
  "derived_values": {
    "ebit": 253466000000.0,
    "ebitda": 309679000000.0,
    "quick_assets": 193635000000.0,
    "debt_plus_equity": 1392511000000.0
  },
  "ratios": {
    "cost_to_income_ratio": 0.6969407367131284,
    "quick_ratio": 0.4878574181991338,
    "debt_to_equity_ratio": 0.3212983064742631,
    "debt_to_assets_ratio": 0.19012686728486042,
    "debt_to_capital_ratio": 0.24316863565171118,
    "debt_to_ebitda_ratio": 1.093438689740021,
    "interest_coverage_ratio": 21.155663133294382
  }
}
```


Alibaba:


```
{
```

```
  "year": "2024",
  "currency": "CNY",
  "field_values": {
    "cash_and_cash_equivalents": 248125000000.0,
    "short_term_market_securities": 262955000000.0,
    "net_accounts_receivable": 29362000000.0,
    "total_current_liabilities": 421507000000.0,
    "total_debt_short_term_and_long_term": 170776000000.0,
    "total_equity": 1101871000000.0,
    "total_assets": 1764829000000.0,
    "total_revenue": 941168000000.0,
    "total_operating_cost": 827818000000.0,
    "net_income": 71332000000.0,
    "income_tax_expense": 22529000000.0,
    "interest_expenses": 7947000000.0,
    "depreciation_and_amortization": 44504000000.0
  },
  "derived_values": {
    "ebit": 101808000000.0,
    "ebitda": 146312000000.0,
    "quick_assets": 540442000000.0,
    "debt_plus_equity": 1272647000000.0
  },
  "ratios": {
    "cost_to_income_ratio": 0.8795645410808697,
    "quick_ratio": 1.282166132472296,
    "debt_to_equity_ratio": 0.1549872898007117,
    "debt_to_assets_ratio": 0.0967663156033814,
    "debt_to_capital_ratio": 0.13418960638731714,
    "debt_to_ebitda_ratio": 1.1672043304718684,
    "interest_coverage_ratio": 12.810872027180068
  }
},
{
  "year": "2025",
  "currency": "CNY",
  "field_values": {
    "cash_and_cash_equivalents": 145487000000.0,
    "short_term_market_securities": 228826000000.0,
    "net_accounts_receivable": 30652000000.0,
    "total_current_liabilities": 435346000000.0,
    "total_debt_short_term_and_long_term": 230703000000.0,
    "total_equity": 1078393000000.0,
    "total_assets": 1804227000000.0,
    "total_revenue": 996347000000.0,
    "total_operating_cost": 856203000000.0,
    "net_income": 125976000000.0,
    "income_tax_expense": 35445000000.0,
    "interest_expenses": 9596000000.0,
    "depreciation_and_amortization": 42459000000.0
  },
  "derived_values": {
    "ebit": 171017000000.0,
    "ebitda": 213476000000.0,
    "quick_assets": 404965000000.0,
    "debt_plus_equity": 1309096000000.0
  },
  "ratios": {
```

```
    "cost_to_income_ratio": 0.8593421769724805,
    "quick_ratio": 0.9302141285322479,
    "debt_to_equity_ratio": 0.21393221209707408,
    "debt_to_assets_ratio": 0.12786805651395305,
    "debt_to_capital_ratio": 0.17623077299143836,
    "debt_to_ebitda_ratio": 1.0806975959826866,
    "interest_coverage_ratio": 17.821696540225094
  }
}
```


JPMorgan Chase:


```
{
  "year": "2023",
  "currency": "USD",
  "field_values": {
    "cash_and_cash_equivalents": 624151000000.0,
    "short_term_market_securities": 476588000000.0,
    "net_accounts_receivable": 107363000000.0,
    "total_current_liabilities": 3155690000000.0,
    "total_debt_short_term_and_long_term": 436537000000.0,
    "total_equity": 327878000000.0,
    "total_assets": 3875393000000.0,
    "total_revenue": 239425000000.0,
    "total_operating_cost": 177813000000.0,
    "net_income": 49552000000.0,
    "income_tax_expense": 12060000000.0,
    "interest_expenses": 81321000000.0,
    "depreciation_and_amortization": 7512000000.0
  },
  "derived_values": {
    "ebit": 142933000000.0,
    "ebitda": 150445000000.0,
    "quick_assets": 1208102000000.0,
    "debt_plus_equity": 764415000000.0
  },
  "ratios": {
    "cost_to_income_ratio": 0.7426668058891093,
    "quick_ratio": 0.38283291451314927,
    "debt_to_equity_ratio": 1.3314007039203606,
    "debt_to_assets_ratio": 0.11264328546808026,
    "debt_to_capital_ratio": 0.5710733044223361,
    "debt_to_ebitda_ratio": 2.90163847253149,
    "interest_coverage_ratio": 1.7576394781175835
  }
},
{
  "year": "2024",
  "currency": "USD",
  "field_values": {
    "cash_and_cash_equivalents": 469317000000.0,
    "short_term_market_securities": 514547000000.0,
    "net_accounts_receivable": 101223000000.0,
    "total_current_liabilities": 3256638000000.0,
    "total_debt_short_term_and_long_term": 454311000000.0,
    "total_equity": 344758000000.0,
    "total_assets": 4002814000000.0,
    "total_revenue": 278906000000.0,
```

```
    "total_operating_cost": 203825000000.0,
    "net_income": 58471000000.0,
    "income_tax_expense": 16610000000.0,
    "interest_expenses": 101350000000.0,
    "depreciation_and_amortization": 7938000000.0
  },
  "derived_values": {
    "ebit": 176431000000.0,
    "ebitda": 184369000000.0,
    "quick_assets": 1085087000000.0,
    "debt_plus_equity": 799069000000.0
  },
  "ratios": {
    "cost_to_income_ratio": 0.7308017755085943,
    "quick_ratio": 0.33319239043455245,
    "debt_to_equity_ratio": 1.3177678255471954,
    "debt_to_assets_ratio": 0.11349790422437815,
    "debt_to_capital_ratio": 0.5685504005286152,
    "debt_to_ebitda_ratio": 2.464139849974779,
    "interest_coverage_ratio": 1.740809077454366
  }
}
```


ExxonMobil:


```
{
  "year": "2023",
  "currency": "USD",
  "field_values": {
    "cash_and_cash_equivalents": 31568000000.0,
    "short_term_market_securities": 0.0,
    "net_accounts_receivable": 38015000000.0,
    "total_current_liabilities": 65316000000.0,
    "total_debt_short_term_and_long_term": 41573000000.0,
    "total_equity": 212538000000.0,
    "total_assets": 376317000000.0,
    "total_revenue": 344582000000.0,
    "total_operating_cost": 290950000000.0,
    "net_income": 36010000000.0,
    "income_tax_expense": 15429000000.0,
    "interest_expenses": 849000000.0,
    "depreciation_and_amortization": 20641000000.0
  },
  "derived_values": {
    "ebit": 52288000000.0,
    "ebitda": 72929000000.0,
    "quick_assets": 69583000000.0,
    "debt_plus_equity": 254111000000.0
  },
  "ratios": {
    "cost_to_income_ratio": 0.8443563505928923,
    "quick_ratio": 1.065328556555821,
    "debt_to_equity_ratio": 0.19560266869924436,
    "debt_to_assets_ratio": 0.11047335092488514,
    "debt_to_capital_ratio": 0.163601733100889,
    "debt_to_ebitda_ratio": 0.5700475805235229,
    "interest_coverage_ratio": 61.58775029446407
  }
```

```
},
{
  "year": "2024",
  "currency": "USD",
  "field_values": {
    "cash_and_cash_equivalents": 23187000000.0,
    "short_term_market_securities": 0.0,
    "net_accounts_receivable": 43681000000.0,
    "total_current_liabilities": 70307000000.0,
    "total_debt_short_term_and_long_term": 41710000000.0,
    "total_equity": 270606000000.0,
    "total_assets": 453475000000.0,
    "total_revenue": 349585000000.0,
    "total_operating_cost": 299716000000.0,
    "net_income": 33680000000.0,
    "income_tax_expense": 13810000000.0,
    "interest_expenses": 996000000.0,
    "depreciation_and_amortization": 23442000000.0
  },
  "derived_values": {
    "ebit": 48486000000.0,
    "ebitda": 71928000000.0,
    "quick_assets": 66868000000.0,
    "debt_plus_equity": 312316000000.0
  },
  "ratios": {
    "cost_to_income_ratio": 0.8573479983408899,
    "quick_ratio": 0.951085951612215,
    "debt_to_equity_ratio": 0.1541355328411048,
    "debt_to_assets_ratio": 0.09197860962566845,
    "debt_to_capital_ratio": 0.1335506346136605,
    "debt_to_ebitda_ratio": 0.5798854409965521,
    "interest_coverage_ratio": 48.68072289156626
  }
}
```


Volkswagen:


```
{
  "year": "2023",
  "currency": "EUR",
  "field_values": {
    "cash_and_cash_equivalents": 43449000000.0,
    "short_term_market_securities": 26772000000.0,
    "net_accounts_receivable": 88230000000.0,
    "total_current_liabilities": 206036000000.0,
    "total_debt_short_term_and_long_term": 232799000000.0,
    "total_equity": 189186000000.0,
    "total_assets": 600649000000.0,
    "total_revenue": 322284000000.0,
    "total_operating_cost": 314907000000.0,
    "net_income": 17861000000.0,
    "income_tax_expense": 5237000000.0,
    "interest_expenses": 3640000000.0,
    "depreciation_and_amortization": 27566000000.0
  },
  "derived_values": {
    "ebit": 26738000000.0,
```

```
    "ebitda": 54304000000.0,
    "quick_assets": 158451000000.0,
    "debt_plus_equity": 421985000000.0
  },
  "ratios": {
    "cost_to_income_ratio": 0.9771102505864393,
    "quick_ratio": 0.7690452153992506,
    "debt_to_equity_ratio": 1.2305297432156714,
    "debt_to_assets_ratio": 0.3875791019380703,
    "debt_to_capital_ratio": 0.5516760074410227,
    "debt_to_ebitda_ratio": 4.286958603417796,
    "interest_coverage_ratio": 7.345604395604395
  }
},
{
  "year": "2024",
  "currency": "EUR",
  "field_values": {
    "cash_and_cash_equivalents": 40296000000.0,
    "short_term_market_securities": 27326000000.0,
    "net_accounts_receivable": 89985000000.0,
    "total_current_liabilities": 217039000000.0,
    "total_debt_short_term_and_long_term": 254081000000.0,
    "total_equity": 196731000000.0,
    "total_assets": 632905000000.0,
    "total_revenue": 324656000000.0,
    "total_operating_cost": 320570000000.0,
    "net_income": 12394000000.0,
    "income_tax_expense": 4411000000.0,
    "interest_expenses": 3446000000.0,
    "depreciation_and_amortization": 31346000000.0
  },
  "derived_values": {
    "ebit": 20251000000.0,
    "ebitda": 51597000000.0,
    "quick_assets": 157607000000.0,
    "debt_plus_equity": 450812000000.0
  },
  "ratios": {
    "cost_to_income_ratio": 0.9874143709033562,
    "quick_ratio": 0.7261690295292551,
    "debt_to_equity_ratio": 1.2915148095622957,
    "debt_to_assets_ratio": 0.40145203466555013,
    "debt_to_capital_ratio": 0.5636074461194467,
    "debt_to_ebitda_ratio": 4.924336686241448,
    "interest_coverage_ratio": 5.876668601276843
  }
}
```


Microsoft:


```
{
  "year": "2024",
  "currency": "USD",
  "field_values": {
    "cash_and_cash_equivalents": 18315000000.0,
    "short_term_market_securities": 57228000000.0,
    "net_accounts_receivable": 56924000000.0,
```

```
    "total_current_liabilities": 125286000000.0,
    "total_debt_short_term_and_long_term": 51630000000.0,
    "total_equity": 268477000000.0,
    "total_assets": 512163000000.0,
    "total_revenue": 245122000000.0,
    "total_operating_cost": 135689000000.0,
    "net_income": 88136000000.0,
    "income_tax_expense": 19651000000.0,
    "interest_expenses": 2935000000.0,
    "depreciation_and_amortization": 22287000000.0
  },
  "derived_values": {
    "ebit": 110722000000.0,
    "ebitda": 133009000000.0,
    "quick_assets": 132467000000.0,
    "debt_plus_equity": 320107000000.0
  },
  "ratios": {
    "cost_to_income_ratio": 0.5535570042672628,
    "quick_ratio": 1.0573168590265472,
    "debt_to_equity_ratio": 0.19230697601656752,
    "debt_to_assets_ratio": 0.10080775065750552,
    "debt_to_capital_ratio": 0.16128981871686654,
    "debt_to_ebitda_ratio": 0.38816922163161893,
    "interest_coverage_ratio": 37.72470187393527
  }
},
{
  "year": "2025",
  "currency": "USD",
  "field_values": {
    "cash_and_cash_equivalents": 30242000000.0,
    "short_term_market_securities": 64323000000.0,
    "net_accounts_receivable": 69905000000.0,
    "total_current_liabilities": 141218000000.0,
    "total_debt_short_term_and_long_term": 43151000000.0,
    "total_equity": 343479000000.0,
    "total_assets": 619003000000.0,
    "total_revenue": 281724000000.0,
    "total_operating_cost": 153196000000.0,
    "net_income": 101832000000.0,
    "income_tax_expense": 21795000000.0,
    "interest_expenses": 2385000000.0,
    "depreciation_and_amortization": 34153000000.0
  },
  "derived_values": {
    "ebit": 126012000000.0,
    "ebitda": 160165000000.0,
    "quick_assets": 164470000000.0,
    "debt_plus_equity": 386630000000.0
  },
  "ratios": {
    "cost_to_income_ratio": 0.5437804375914015,
    "quick_ratio": 1.1646532311744962,
    "debt_to_equity_ratio": 0.12562922332951942,
    "debt_to_assets_ratio": 0.06971048605580264,
    "debt_to_capital_ratio": 0.11160799731008975,
    "debt_to_ebitda_ratio": 0.2694159148378235,
```

```
    "interest_coverage_ratio": 52.835220125786165
  }
}
```


Google:


```
{
  "year": "2023",
  "currency": "USD",
  "field_values": {
    "cash_and_cash_equivalents": 24048000000.0,
    "short_term_market_securities": 86868000000.0,
    "net_accounts_receivable": 47964000000.0,
    "total_current_liabilities": 81814000000.0,
    "total_debt_short_term_and_long_term": 11870000000.0,
    "total_equity": 283379000000.0,
    "total_assets": 402392000000.0,
    "total_revenue": 307394000000.0,
    "total_operating_cost": 223101000000.0,
    "net_income": 73795000000.0,
    "income_tax_expense": 11922000000.0,
    "interest_expenses": 308000000.0,
    "depreciation_and_amortization": 11946000000.0
  },
  "derived_values": {
    "ebit": 86025000000.0,
    "ebitda": 97971000000.0,
    "quick_assets": 158880000000.0,
    "debt_plus_equity": 295249000000.0
  },
  "ratios": {
    "cost_to_income_ratio": 0.7257818955477335,
    "quick_ratio": 1.941965922702716,
    "debt_to_equity_ratio": 0.04188736638918198,
    "debt_to_assets_ratio": 0.02949859838167757,
    "debt_to_capital_ratio": 0.040203353779352344,
    "debt_to_ebitda_ratio": 0.12115830194649437,
    "interest_coverage_ratio": 279.30194805194805
  }
},
{
  "year": "2024",
  "currency": "USD",
  "field_values": {
    "cash_and_cash_equivalents": 23466000000.0,
    "short_term_market_securities": 72191000000.0,
    "net_accounts_receivable": 52340000000.0,
    "total_current_liabilities": 89122000000.0,
    "total_debt_short_term_and_long_term": 10883000000.0,
    "total_equity": 325084000000.0,
    "total_assets": 450256000000.0,
    "total_revenue": 350018000000.0,
    "total_operating_cost": 237628000000.0,
    "net_income": 100118000000.0,
    "income_tax_expense": 19697000000.0,
    "interest_expenses": 268000000.0,
    "depreciation_and_amortization": 15311000000.0
  },
```

```
  "derived_values": {
    "ebit": 120083000000.0,
    "ebitda": 135394000000.0,
    "quick_assets": 147997000000.0,
    "debt_plus_equity": 335967000000.0
  },
  "ratios": {
    "cost_to_income_ratio": 0.678902227885423,
    "quick_ratio": 1.6606112968739482,
    "debt_to_equity_ratio": 0.033477501199689924,
    "debt_to_assets_ratio": 0.02417069400518816,
    "debt_to_capital_ratio": 0.03239306241386803,
    "debt_to_ebitda_ratio": 0.08038022364358834,
    "interest_coverage_ratio": 448.07089552238807
  }
}
```


## 4 Software Architecture

The codebase follows a modular, object-oriented architecture. All forecast logic lives in the `financial_forecast/` Python package, organized into sub-packages:

- `models/` – composable financial model with pluggable policy modules (`BaseFinancialModel`, `TrainableFinancialModel`), each module typed against its abstract base class (e.g., `OpExModule`, `LiquidityPolicy`, `DividendPolicy`, `DebtPolicy`, `SimpleTax`)
- `training/` – `ForecastPipeline` orchestrator, `PolicyTrainer`, `StructuralTrainer`
- `inference/` – `DeterministicSimulator`, `MonteCarloSimulator`, forecast driver models, plotting
- `extraction/` – LLM-based PDF extraction, normalization, ratio calculation, hallucination analysis
- `reporting/` – `TableFormatter` ABC with `MarkdownTableFormatter`, and `Advisor` ABC with `DeepseekCEOAdvisor` for LLM-based CEO recommendations
- `data/` – `HistoricalDataLoader` and per-company data modules
- `serialization/` – `.npz` parameter save/load
- `clients/` – `AzureLLMClient` for Azure-hosted LLM endpoints

The `ForecastPipeline` class automatically exports a self-contained JSON report (`forecast_report.json`) containing historical and forecast markdown tables, model metadata, and a reference to the trained parameters file. This JSON artifact is consumed by `run_recommendation_to_ceo.py` for downstream LLM analysis, decoupling the training/forecast stage from the LLM recommendation stage.

All method signatures throughout the package use Python type hints, and each entry-point `run_*.py` script is a thin orchestrator that wires together the modular components.

# 5 Testing and Validation

This section describes the validation strategy used to verify the correctness of both the structural financial forecast model (Part 1) and the LLM-based extraction and analysis pipelines (Part 2). The test suite is implemented in Python using `pytest` and comprises 22 test files organized into a layered architecture of unit, integration, and end-to-end tests.

## 5.1 Balance Sheet Identity as Core Invariant

The "no-plug" design of the forecast model means the fundamental accounting identity (*Assets = Liabilities + Equity*) is not enforced by a balancing variable; it must *emerge* from the correctness of every cash flow, accrual, and valuation equation. This makes the identity a powerful test oracle: at every forecast step, we compute

$$\Delta_t = |\text{Total Assets}_t - (\text{Total Liabilities}_t + \text{Equity}_t)|$$

and assert $\Delta_t < 10^{-4}$. A nonzero delta pinpoints a specific structural error (e.g., an expense recorded without a corresponding cash deduction), rather than being silently absorbed by a plug. This check is executed across all forecast horizons and all model variants (deterministic, SimpleOpEx, BayesianOpEx) as the primary integration test.

## 5.2 Test Suite Architecture

The test suite is organized by the module under test, with each layer building on the one below it:

- **Type contract tests** (`test_types.py`): Validate the `RecurrentState` (14-key) and `ForecastInputs` (3-key) TypedDict schemas, ensuring missing, extra, or misspelled keys raise `KeyError` with descriptive context labels.
- **Decomposed unit tests** (`test_cash_budget_decomposed.py`, `test_policy_trainer_decompo` Verify individual computations against hand-calculated values using concrete tensors (e.g., operating net liquidity balance, loss scale computation under STD and NONE modes).
- **Model variant tests** (`test_simple_model_forecast.py`, `test_trainable_financial_model_` `test_trainable_financial_model_bayesianopex.py`): Each model configuration is tested for balance sheet identity, finite output values, determinism (for non-stochastic variants), and serialization round-trip correctness.
- **Simulation interface tests** (`test_simulator_interface.py`): Verify that `DeterministicSimulator` and `MonteCarloSimulator` expose consistent interfaces and correct default parameters.
- **Regression tests** (`test_inflation_alignment.py`): Guard against a previously discovered bug where inflation data of different length than financial history caused silent misalignment. Tests verify tail-alignment, exact-length pass-through, and `ValueError` on insufficient data.
- **Extraction pipeline tests** (`test_extraction.py`, `test_tax_anomalies.py`, `test_statement_normalization.py`, `test_llm_forecast.py`): Test the LLM-based financial statement extraction, tax anomaly detection, statement normaliza-

tion, and LLM balance sheet forecasting pipelines. These use `unittest.mock` to isolate tests from external API calls.

- **Risk warning pipeline tests** (`tests/risk/`—8 files): End-to-end integration tests for the risk extraction pipeline, including PDF page flagging, category extraction, risk memo synthesis, entity anonymization, and Pydantic model validation for domain types.

### 5.3 Testing Strategies

The following strategies are applied across the test suite:

- **Balance sheet identity invariant**: Every model test asserts $|\text{Assets} - \text{Claims}| < 10^{-4}$ at each forecast step, across all time horizons and model variants.
- **Finite-value enforcement**: All TensorFlow tensor outputs are checked with `tf.debugging.assert_all_finite` or equivalent assertions to detect NaN or Inf propagation.
- **Determinism checks**: Non-stochastic model variants are called multiple times with identical inputs; outputs must be bit-identical across runs.
- **Stochastic correctness**: The BayesianOpEx variant is tested for variational sampling sanity, bijector-implied economic bounds (ensuring sampled parameters remain in economically meaningful ranges), and Monte Carlo trajectory finiteness across all samples.
- **Numerical precision**: All financial computations use `tf.float64` (enforced by dtype assertions in tests). Floating-point comparisons use `pytest.approx()` with explicit tolerances. Loss scale computations handle degenerate cases with $\epsilon = 10^{-12}$.
- **Mock isolation**: LLM and external API interactions are replaced with `unittest.mock` patches, ensuring tests run deterministically without network access. Control flow, JSON output structure, and file naming conventions are verified against mocked responses.
- **Serialization round-trips**: Trainable model parameters are saved and reloaded, with assertions that restored values match originals within floating-point tolerance.
- **Pydantic validation**: Risk domain models are tested for type coercion (e.g., string-to-int page numbers), optional field handling, union type acceptance, and rejection of invalid inputs.

### 5.4 Parameter Recovery (Identifiability Testing)

The preceding tests verify *structural* correctness: that the accounting identity holds, outputs are finite, and parameters survive serialization. However, they do not verify that the gradient-based training pipeline converges to the *correct* parameter values. A model can produce finite, balanced forecasts while its learned parameters sit at a spurious local minimum far from the true data-generating process.

**Parameter recovery testing** (also known as *simulation-based calibration*) closes this gap. The procedure is:

1. Define a set of known ground-truth parameter values.
2. Create a "generator" model and assign those values.
3. Run `forecast_step` in a loop from a cash-poor initial balance sheet, producing perfectly consistent synthetic financial statements, including the full cash budget circularity (new ST/LT borrowing, equity financing, interest feedback).
4. Train a fresh "recovery" model on the synthetic data.
5. Assert that recovered parameters match ground truth within tolerance.

We test four scenarios of increasing complexity: (1) all simple (static-ratio) policies, (2) advanced policies with logit-linear trends and Lintner dividends, (3) advanced policies with Bayesian variational OpEx, and (4) advanced policies with Bayesian OpEx and one-time tax anomalies. The most involved scenario (4) uses the following OpEx and tax formulations:

$$\text{OpEx}_t = \underbrace{\beta_0 \cdot \Pi_t^{\text{cum}}}_{\text{baseline}} + \underbrace{\beta_1 \cdot (\text{Sales}_t - \bar{S})}_{\text{variable}} + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, \sigma^2) \tag{57}$$

$$\text{Tax}_t = \text{EBT}_t \cdot \tau + \delta_t, \qquad \delta_t = \begin{cases} a_{\text{onetime}} & \text{if year } t \text{ has anomaly} \\ 0 & \text{otherwise} \end{cases} \tag{58}$$

where $\Pi_t^{\text{cum}} = \prod_{s=1}^{t}(1 + \text{Inflation}_s)$ is the cumulative inflation factor (so that $\beta_0 \cdot \Pi_t^{\text{cum}}$ is the closed-form equivalent of the recursive baseline $\text{OB}_{t-1} \cdot (1 + \text{Inflation}_t)$ from (21)), $\bar{S}$ is the mean historical sales (used to center the variable component), $\beta_0, \beta_1$ have Normal variational posteriors with trainable location and scale, $\sigma$ is the learned aleatoric noise, $\tau$ is the effective tax rate, and $\delta_t$ are known one-time payments (e.g., a \$2B settlement in 2020).

```
Parameter Recovery — Scenario 4 (BayesianOpEx + Tax Anomalies)
---------------------------------------------------------------
Parameter                      True     Recovered   Status
---------------------------------------------------------------
Depreciation Rate               0.0550   ~0.055      PASS (rel 10%)
Asset Growth                    0.0080   ~0.008      PASS (rel 15%)
Accounts Receivable %           0.1600   ~0.160      PASS (rel 10%)
Cost Ratio Alpha                0.3200   ~0.320      PASS (abs 0.10)
Cost Ratio Beta                -0.0200   ~-0.020     PASS (abs 0.02)
BayesianOpEx Var. Loc           0.1000   ~0.100      PASS (rel 20%)
BayesianOpEx Base Scale         1.0(i)   < 1.0       PASS (shrinks)
BayesianOpEx Var. Scale         1.0(i)   < 1.0       PASS (shrinks)
Noise Sigma                     0.0050   < 0.1       PASS
Income Tax Rate                 0.1500   ~0.150      PASS (rel 10%)
Lintner Payout Ratio            0.1600   ~0.160      PASS (rel 10%)
Lintner Adj. Speed              0.3000   ~0.300      PASS (rel 20%)
Avg LT Interest Rate            0.0350   ~0.035      PASS (rel 20%)
Avg Maturity Years              5.0000   ~5.000      PASS (rel 20%)
---------------------------------------------------------------
All 82 tests pass across 4 scenarios (simple, advanced, Bayesian,
Bayesian + tax anomalies). (i) = initial value before training.
```

The results confirm three properties: (a) the model is *identifiable*: ground-truth parameters are uniquely recoverable from data generated by those same parameters; (b) the Adam optimizer converges to the correct minimum rather than a spurious local optimum; and (c) the no-plug accounting framework correctly propagates all financial flows through the cash budget circularity, even under stochastic OpEx and one-time tax anomalies. Parameters trained indirectly via `forecast_step` transitions (interest rates, debt maturity, equity financing mix) require wider tolerances due to the smaller effective sample size of state transitions, consistent with the theoretical expectation that indirect estimation paths have lower Fisher information.

Beyond the numerical tolerance checks above, we also verify the fit *visually* by taking the trained recovery model and applying its own one-step-ahead prediction (`forecast_step`) to every historical transition, then overlaying the resulting fitted values on the synthetic observations. This is the same one-step-ahead diagnostic that `ForecastPipeline` uses for real historical data. Figure 1 shows the result for Scenario 4 (the most involved scenario, with Bayesian OpEx and tax anomalies). Across twelve key metrics—including the net income, cost of goods sold, operating expenses (with injected noise), tax (with the 2020 anomaly spike and 2022 dip), non-current assets, cash, working-capital components, and the full capital structure (effective short-term debt, non-current liabilities, equity)—the red fitted series tracks the black synthetic truth essentially perfectly. This confirms that the recovered parameters reproduce the data-generating process not only in isolation (passing the per-parameter tolerance tests) but also jointly through the full forward model evolution, including the cash budget circularity.

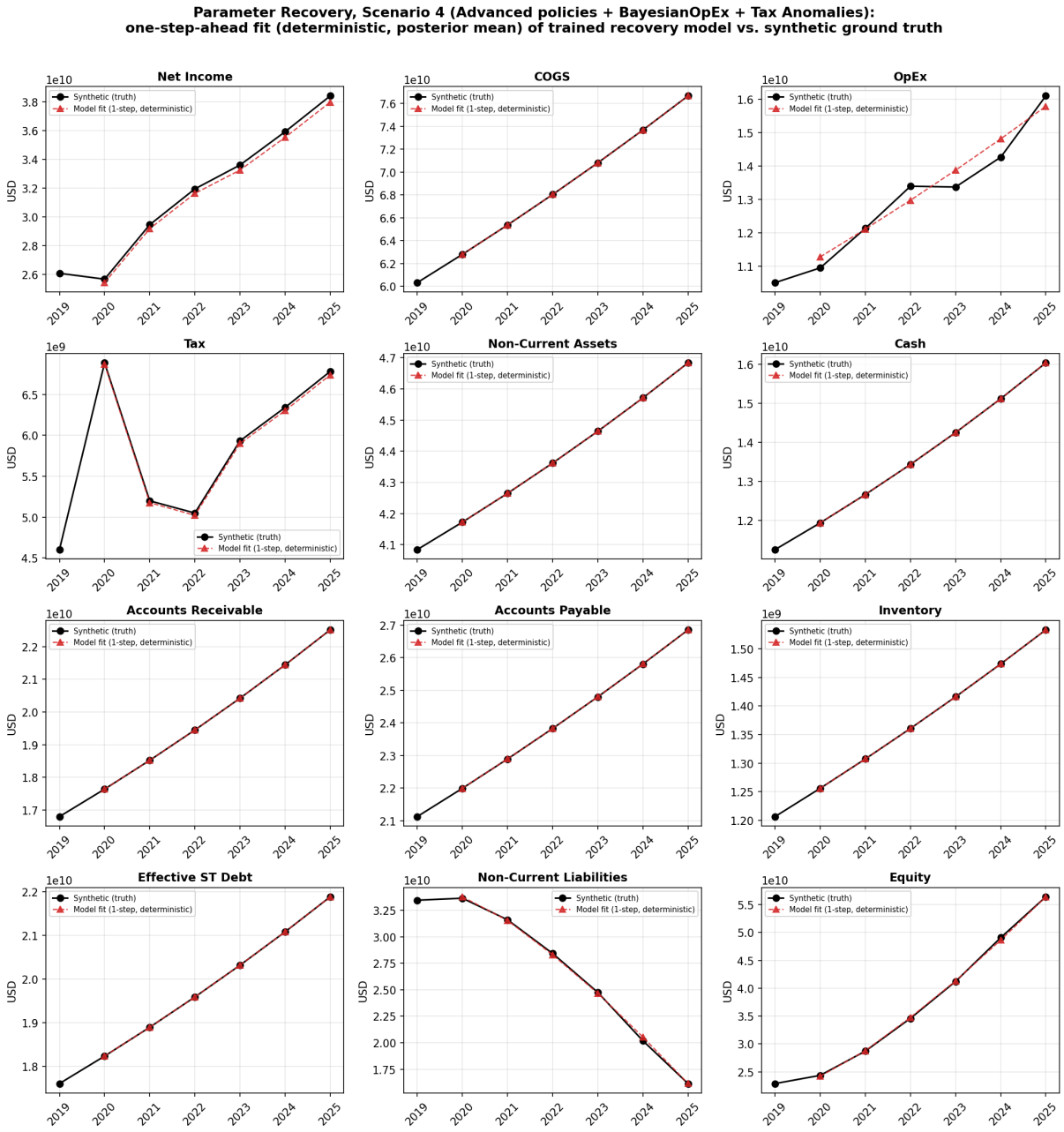



Figure 1: Parameter recovery, Scenario 4 (advanced policies + Bayesian OpEx + tax anomalies): *deterministic* one-step-ahead fit of the trained recovery model against the synthetic ground truth it was trained on. Black markers are the synthetic observations produced by the generator model; red markers are the trained model's `forecast_step` predictions with `use_mean_opex=True` (posterior mean, no sampling), initialized from each preceding state. The 2020 spike and 2022 dip in the Tax panel correspond to the known one-time anomalies $\delta_{2020} = \$2\text{B}$ and $\delta_{2022} = -\$0.5\text{B}$, which the `TaxWithAnomalies` module reproduces exactly. The slight noise in the OpEx panel reflects the injected aleatoric noise $\varepsilon_t \sim \mathcal{N}(0, \sigma^2)$.

The deterministic fit in Figure 1 uses the posterior *mean* of the Bayesian OpEx variational parameters and suppresses aleatoric noise. To also visualize the *predictive*

*uncertainty* implied by the trained posterior, we repeat the same one-step-ahead fit but instead draw $N = 500$ Monte Carlo samples from the OpEx posterior and noise distributions at each transition, then plot the sample mean together with a 95% credible band. The only code difference from the deterministic variant is passing `use_mean_opex=False` to `forecast_step_compiled` on a state tiled to $[N, 14]$ after a single `opex_module.prepare_mc` call. Figure 2 shows the result. As expected, the band is widest on OpEx itself and propagates into Net Income, Tax, Non-Current Liabilities, and Equity (which depend on OpEx through the income statement and cash budget), while COGS, cash, working-capital items, and non-current assets—which are deterministic functions of sales, cost ratio, and working-capital ratios at the 1-step horizon—collapse to a near-zero band. This is the expected topology of uncertainty in the model and further confirms the recovered posterior behaves correctly.

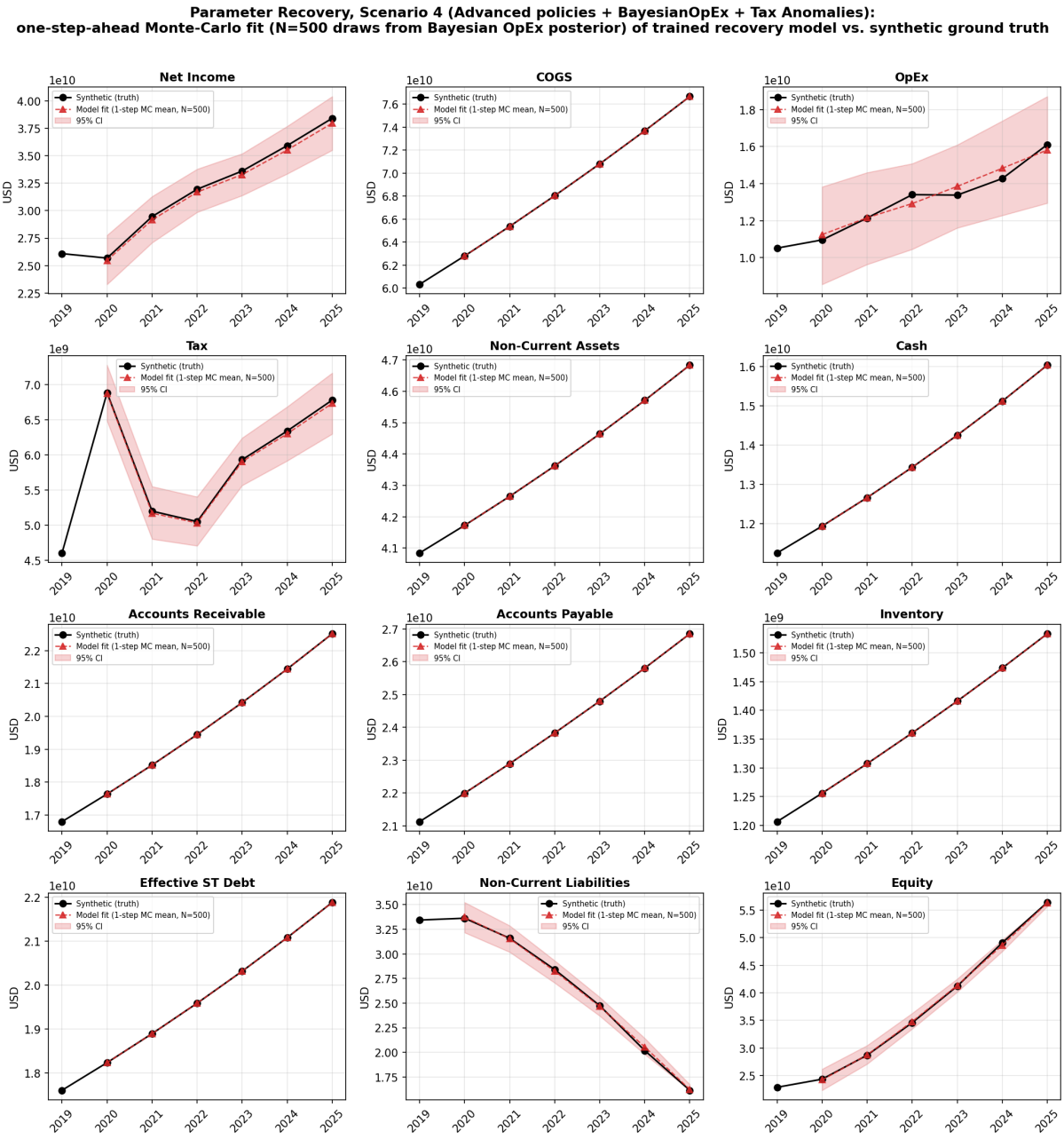



Figure 2: Parameter recovery, Scenario 4: *Monte Carlo* one-step-ahead fit. Same setup as Figure 1 except the trained model is evaluated with `use_mean_opex=False` on a state tiled to $N = 500$ samples, drawing from the Bayesian OpEx posterior $q(\beta_0), q(\beta_1)$ and the aleatoric noise $\mathcal{N}(0, \sigma^2)$. The red line is the MC mean and the shaded red region is the 95% credible band (2.5–97.5 percentiles across samples). Panels whose value depends on OpEx at this 1-step horizon (OpEx, Net Income, Tax, Non-Current Liabilities, Equity) display the propagated uncertainty; panels that do not (COGS, NCA, Cash, AR, AP, Inventory, Effective ST Debt) have near-zero bands, as they are deterministic functions of the (exact) state and exogenous sales.

# 6 References

[Almeida(14)] Almeida, Heitor et al., (2014), "Corporate Liquidity Management: A Conceptual Framework and Survey", Available at SSRN: http://ssrn.com/abstract_id=2336367

[Brav(05)] Brav, Alon, Graham, John R., Harvey, Campbell R., & Michaely, Roni (2005). "Payout Policy in the 21st Century," *Journal of Financial Economics*, 77(3), 483–527.

[Dürr(20)] Dürr, O., Sick, B., & Murina, E. (2020). Probabilistic deep learning: With Python, Keras and TensorFlow probability. Manning.

[IASB(21)] International Accounting Standards Board (IASB). (2021). IAS 2: Inventories. https://www.ifrs.org/content/dam/ifrs/publications/pdf-standards/english/2024/issued/part-a/ias-2-inventories.pdf?bypass=on

[Koller(20)] Koller, T., Goedhart, M., & Wessels, D. (2020). *Valuation: Measuring and Managing the Value of Companies* (7th ed.). McKinsey & Company.

[Lintner(56)] Lintner, John (1956). "Distribution of Incomes of Corporations Among Dividends, Retained Earnings, and Taxes," *American Economic Review*, 46(2), 97–113.

[Liu(23)] Liu, Nelson F. et al, (2023), "Lost in the Middle: How Language Models Use Long Contexts", https://arxiv.org/abs/2307.03172

[Merton(74)] Merton, Robert C. (1974). "On the Pricing of Corporate Debt: The Risk Structure of Interest Rates," *Journal of Finance*, 29(2), 449–470.

[Myers(84)] Myers, Stewart C., & Majluf, Nicholas S. (1984). "Corporate Financing and Investment Decisions When Firms Have Information That Investors Do Not Have," *Journal of Financial Economics*, 13(2), 187–221.

[Pareja(07)] Vélez Pareja, Ignacio, (2007), "Forecasting Financial Statement with No Plugs and No Circularity", Available at SSRN: http://ssrn.com/abstract=1031735

[Pareja(09)] Vélez Pareja, Ignacio, (2009), "Constructing Consistent Financial Planning Models for Valuation" (August 15), Available at SSRN: http://ssrn.com/abstract=1455304

[Wei(22)] Wei, Jason et al., (2022), "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models", https://arxiv.org/abs/2201.11903

[Zhang(25)] Zhang, Haichao et al., (2025), "Research on Financial Statement Checking Relationship Recognition System Based on Large Language Models", https://dl.acm.org/doi/pdf/10.1145/3745238.3745291

# A Reviewer's Feedback

## A.1 Feedback on Part 1

*(1) You identified the principle of "no plug" and some accounting identities, such as non-current assets (NCA) being a function of depreciation and capex:*

*NCA_t = NCA_t-1 – Depreciation_t + CapEx_t*

*However, later, while formulating the time-series model, you consider the non-current asset at time t to be driven by a ratio:*

*NCA_t = NCA_t-1 * (1 + %AG)*

*Is it possible to design a workflow that fully respects the accounting identities (like the first equation) and can be driven by drivers?*

**Response:** Thank you for catching this mistake. This was a typo in the report; the actual script had the correct relationship of CapEx_t = %AG * Sales_t . We fixed the typo in the report

*(2) What is the difference between the FIFO and LIFO principles in accounting? Under what circumstances will these two accounting choices make a difference? Is it possible to avoid using COGS as a plug?*

**Response:** FIFO (First-In, First-Out): Under FIFO, the oldest inventory costs are matched against current sales revenues.

LIFO (Last-In, First-Out): Under LIFO, the most recent inventory costs are matched against current sales revenues.

The most difference can arise from these two accounting standards when there is a rapid inflationary (deflationary) environment. Under FIFO, the cost ratio will be smoother and slightly lower (higher) than the actual current cost of replacement. Under LIFO, the cost ratio will be more volatile since the recent inventory cost is used, and it will tightly track the inflation (deflation) and will squeeze (grow) the margin immediately. Under inflation = 0%, FIFO and LIFO should equal each other because old cost = new cost. Under high inflation, FIFO profit > LIFO profit.

In our model, we avoid using COGS as a plug by updating it so that we learn the cost ratio ($\mathrm{CR}_t$) directly from the historical COGS and sales data by modeling cost ratio as a logit and learning the underlying parameters $\alpha$ and $\beta$ to ensure positivity :

$\mathrm{logit}(\mathrm{CR}_t) = \ln(\mathrm{CR}_t/(1-\mathrm{CR}_t)) = \alpha + \beta \cdot t$ (from (5))

$\mathrm{CR}_t = \mathrm{COGS}_t/\mathrm{Sales}_t$ (from (4))

(3) Similar to (1), can you list out all the entities (rows) in the income statement, cash budget, and balance sheet (simplified versions are fine)? Then analyze how these three statements connect with each other. You may start with revenue on the income statement.

Response: We now include the 10-year forecasted Balance Sheet, Income Statement, and Cash Budget in the *Projected Balance Sheet, Income Statement, and Cash Budget* subsection.

*(4) Advance payment is defined as: AdvPP_t = %AdvPP * Purchase_t+1*

*Is it possible to remove the future dependency on this variable? It may break causal timing and introduce leakage during training. Are there alternative approaches to model this term?*

**Response:** We updated our model such that the advance payments for purchases (AdvPP) are proportional to this year's purchases instead of next year, with the justification:

- From *Advance Payments for Purchases*: "Advance payments to suppliers for purchases ($AdvPP$) can be modeled as strictly proportional to the total purchases made during the current year. This approach relies on the simplifying assumption that current purchasing volumes serve as a reliable proxy for near-term future demand."

Same is applied for advance payments from customers for sales (AdvPS):

- From *Advance Payments for Future Sales*: "Advance payments received from customers for future sales ($AdvPC$) are modeled as a direct proportion of the current year's total sales. This assumes that the volume of upfront customer payments scales linearly with overall revenue generation."

*(5) For CapEx, it's great that you split this term into maintenance capex and growth capex:*

*CapEx_t = MaintenanceCapex_t + GrowthCapex_t*

*You further assume maintenance capex is tied to depreciation. This is a common approach when forecasting future cash flows by assuming a firm is in a steady state. What if the firm is not in a steady state?*

**Response:** We updated our model to handle the case where the asset replacement rate is not unitary (= 1.0). We reflect that change in the report as well:

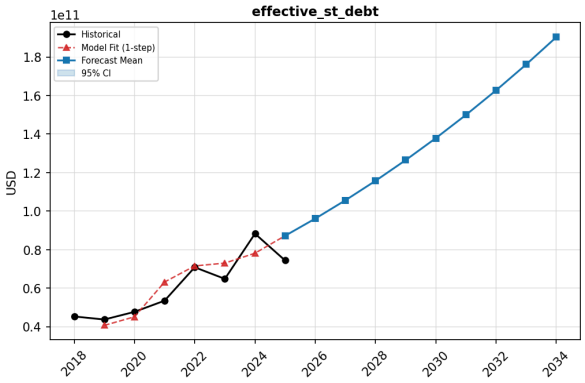
- From *Non-Current Assets (NCA):* "To model CapEx, we diverge from a rigid steady-state assumption where maintenance perfectly offsets depreciation. Instead, we decompose CapEx into a maintenance component (scaled by a trainable parameter, %AM, to reflect partial, full, or inflationary replacement) and a growth component driven by sales expansion:

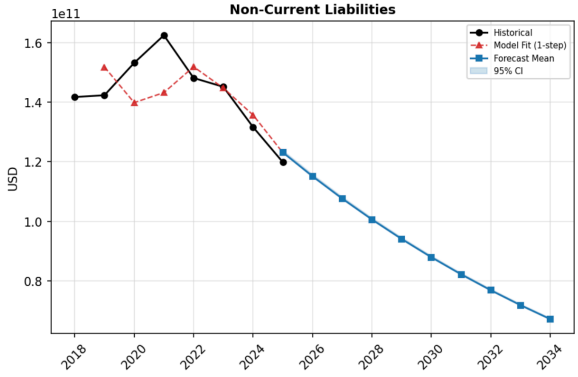
$$\text{CapEx}_t = \%\text{AM} \cdot \text{Depreciation}_t + \%\text{AG} \cdot \text{Sales}_t \quad (3)$$"

*(6) In your report, the financing dynamics on debt are as follows:*

*Debt_t-1 => Interest_t, PrincipalDue_t => deficit_t => Debt_t*

*Can you plot the amount of debt and see whether the outcome is sensible?*




**Response:** We have plotted the forecasted debt levels (see above). The outcome is sensible and mathematically stable. We verified that the model does not fall into an explosive 'debt spiral' The financing dynamics converge realistically.

Specifically for Apple, the model accurately captures two key trends:

1. Short-Term Debt: The model predicts short-term debt scaling proportionally with Sales. This correctly reflects Apple's real-world strategy of utilizing revolving credit and commercial paper for working capital needs, despite possessing a large cash balance.
2. Long-Term Debt: The forecast accurately continues the historical trend of Apple slowly paying down its non-current liabilities.Ultimately, the plot confirms that the endogenous deficit calculation handles debt rollover gracefully without compounding errors.

*(7) Some rules seem a bit ad hoc. Is it possible to tie this to terms with a stronger financial relationship?*

**Response:** We can redefine our training variables from arbitrary percentages to these time-based efficiency metrics.

Receivables: Instead of %AR, we can train a target DSO (Days Sale Outstanding) parameter.

$AR_t$ = DSO / 365 * $Sales_t$

Inventory: Inventory is a function of the cost to produce goods, not the final sales price. Tie it to COGS using DSI (Days Sales of Inventory)

$Inv_t$ = DSI / 365 * $COGS_t$

Payables: We can use DPO (Days Purchases Outstanding) instead of %AP:

$AP_t$ = DPO / 365 * $Purchases_t$

(8) Can you specify the possible constraints for the variables and equations? Please add them to the report and implement them in the code.

**Response:** Thank you. We have added a new subsection, *Parameter Constraints and Bounds*.

*(9) After you have trained the model, can you also perform inference and simulate a firm's financial statements for several years?*

**Response:** We now include the 10-year forecasted Balance Sheet, Income Statement, and Cash Budget in the *Projected Balance Sheet, Income Statement, and Cash Budget* subsection.

(10) Remember to write test cases.

Response: Thank you for the reminder. We wrote 87 test cases across 8 test files covering all modules in the codebase:

- `test_simple_model_forecast.py` – forward simulation model
- `test_trainable_financial_model_simpleopex.py` – trainable model with deterministic OpEx
- `test_trainable_financial_model_bayesianopex.py` – trainable model with Bayesian OpEx
- `test_llm_forecast.py` – LLM-only balance sheet forecasting

- `test_reasoning_prompt.py` – CEO/CFO recommendation prompt
- `test_extraction.py` – financial statement PDF extraction
- `test_statement_normalization.py` – LLM field normalization
- `test_tax_anomalies.py` – tax anomaly extraction and model integration

### A.2 Feedback on Part 2

*(1) In the guideline, we have asked you to "you must submit your code in TensorFlow and Python." TensorFlow is not only for deep learning and training, but also for vectorization management. That means you need to rewrite your codes by using TensorFlow in place of NumPy with the existing framework. NumPy are not strictly forbidden, as they can still be applied when downloading data from Yahoo Finance, data preprocessing and plotting.*

**Response:** We refactored all core modules (training, inference, model) to use TensorFlow exclusively. All trainable parameters are `tf.Variable` instances, the forecast loop and training steps are compiled into TensorFlow graphs via `@tf.function`, and gradient updates use `GradientTape` for automatic differentiation. NumPy is retained only for data serialization (`np.savez`/`np.load`) and preprocessing. We observed a performance improvement of over 100x from graph compilation alone.

*(2) Specifically, what is the merit of TensorFlow compared to NumPy?*

**Response:** In the context of this codebase, TensorFlow provides six concrete advantages over NumPy:

1. **Automatic differentiation.** `GradientTape` computes gradients for over 50 coupled financial parameters without manual calculus. A NumPy implementation would require finite-difference approximation or hand-derived gradients for every parameter, which is impractical given the interdependencies of the financial statements.
2. **Graph compilation.** The `@tf.function` decorator compiles the forecast loop and training step into optimized computation graphs, enabling over 100x speedup compared to eager Python/NumPy loops across 25,000+ training epochs.
3. **Constrained trainable variables.** `tfp.util.TransformedVariable` with bijectors (Softplus, Sigmoid, Chain) automatically enforces economic constraints (such as positivity for growth rates, or $[0, 1]$ bounds for payout ratios) during gradient updates. NumPy has no equivalent; manual clamping after each optimization step would be required.
4. **Vectorized Monte Carlo simulation.** `tf.TensorArray` and `tf.while_loop` inside `@tf.function` efficiently batch 1,000+ stochastic trajectories. NumPy would require explicit Python loops or complex pre-allocated array management that cannot be graph-compiled.
5. **Variational inference.** TensorFlow Probability enables Bayesian OpEx modeling with automatic KL divergence gradients through sampling operations. Computing the Evidence Lower Bound (ELBO) loss requires differentiating through stochastic nodes, which is not possible natively in NumPy.

6. **Module system.** `tf.Module` provides automatic variable discovery and tracking across a hierarchy of policy submodules (CapEx, working capital, liquidity, debt, dividends, buyback, tax). This eliminates manual parameter bookkeeping that a NumPy implementation would require.

*(3) Codebase structure: Ultimately, we are delivering a Python package to our client. Please refactor it to an OOP style, with proper codebase hierarchy.*

**Response:** We refactored the entire codebase following four guiding principles:

1. OOP principles: proper encapsulation, inheritance, polymorphism, and abstraction
2. Clean Code principles (Robert Martin): meaningful naming, single-responsibility functions and classes, SOLID principles
3. PEP 8 and PEP 20 compliance
4. Pythonic style per Fluent Python: Python idioms over Java-style patterns

We used Claude Code's Plan Mode to conduct a systematic code review before refactoring. The following prompt was used to leverage the tool effectively:

```
Enter planning mode. Do not modify any files yet.

I am a finance/quant intern candidate at JP Morgan and need a thorough,
honest code review of this financial forecasting codebase before I submit it.
The interviewer specifically referenced these three books as the standard
I should meet:

1. Clean Code (Robert C. Martin)
2. Clean Code in Python (Mariano Anaya)
3. Fluent Python (Luciano Ramalho)

Please review the entire codebase systematically across these dimensions:
1. OOP Design & SOLID Principles
2. Module Cohesion & Coupling
3. Pythonic Style (Fluent Python lens)
4. Naming & Readability (Clean Code lens)
5. Type Hints & Contracts
6. Test Coverage Gaps

After completing the review, produce a structured report with:
- A severity rating per issue (Critical / Major / Minor)
- The specific file and line range for each issue
- A one-line explanation of which principle it violates
- Do NOT suggest fixes yet
```

*(4) Add docstring and comment: please ensure your code is well-documented in English and follow established docstring conventions (such as PEP257 for Python).*

**Response:** All public interfaces are now documented with PEP 257-compliant docstrings as part of the refactoring effort.

*(5) Attempting LaTeX: Can you try illustrating your ideas in a LaTeX-generated PDF?*

**Response:** The report has been rewritten in LaTeX.

*(6) Double Entry Principle: Can you start with a section to discuss in detail what double entry principle means by providing with some examples? An example from items on income statement and one from statement of cashflow will help the reader understanding why it can keep the balance sheet balanced.*

**Response:** We have added a dedicated subsection in the Introduction (see Section 1.2, *The Double-Entry Principle*), which explains the principle with two worked examples: a credit sale (income statement to balance sheet) and a receivable collection (cash budget to balance sheet).